



ΔΗΜΟΚΡΙΤΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΡΑΚΗΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΛΟΓΙΣΜΙΚΟΥ ΚΑΙ ΑΝΑΠΤΥΞΗΣ ΕΦΑΡΜΟΓΩΝ

**Μελέτη αξιοπιστίας (π.χ. stability, recovery time, responsiveness
etc.) του λογισμικού Kubernetes**

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Ιατρού Κωνσταντίνος, ΑΕΜ: 58071

Επιβλέπων Καθηγητής: Βασίλειος Τσαουσίδης, Καθηγητής, Τμήμα
Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών, Δ.Π.Θ.

Ξάνθη, 2026



ΔΗΜΟΚΡΙΤΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΡΑΚΗΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΛΟΓΙΣΜΙΚΟΥ ΚΑΙ ΑΝΑΠΤΥΞΗΣ ΕΦΑΡΜΟΓΩΝ

Μελέτη αξιοπιστίας (π.χ. stability, recovery time, responsiveness etc.) του λογισμικού Kubernetes

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Ιατρού Κωνσταντίνος, ΑΕΜ: 58071

ΕΞΕΤΑΣΤΙΚΗ ΕΠΙΤΡΟΠΗ

Επιβλέπων Καθηγητής: Βασίλειος Τσαουσίδης, Καθηγητής, Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών, Δ.Π.Θ.

2ο Μέλος: Ελευθερία Κατσίρη, Επίκουρη Καθηγήτρια, Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών, Δ.Π.Θ.

3ο Μέλος: Ιωάννης Μπούταλης, Καθηγητής, Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών, Δ.Π.Θ.

Ξάνθη, 2026



DEMOCRITUS UNIVERSITY OF THRACE
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL & COMPUTER
ENGINEERING
SECTOR OF SOFTWARE AND APPLICATION DEVELOPMENT

**Reliability study (e.g., stability, recovery time,
responsiveness, etc.) of the Kubernetes software**

DIPLOMA THESIS

Iatrou Konstantinos, Registration Number: 58071

COMMITTEE OF EXAMINERS

Supervisor: Vassilios Tsaousidis, Professor, Department of Electrical & Computer Engineering, DUTH

Member 2: Eleftheria Katsiri, Assistant Professor, Department of Electrical & Computer Engineering, DUTH

Member 3: Ioannis Boutalis, Professor, Department of Electrical & Computer Engineering, DUTH

Xanthi, 2026

ΑΝΑΦΟΡΑ ΣΤΗΝ ΤΗΡΗΣΗ ΤΩΝ ΑΚΑΔΗΜΑΪΚΩΝ ΑΡΧΩΝ ΔΕΟΝΤΟΛΟΓΙΑΣ

Η παρούσα εργασία με τίτλο «Μελέτη αξιοπιστίας (π.χ. stability, recovery time, responsiveness etc.) του λογισμικού Kubernetes» είναι πρωτότυπη και πραγματοποιήθηκε από τον φοιτητή του Τμήματος Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών με Αρ. Μητρώου ΑΕΜ 58071 στο Εργαστήριο Εργαστήριο Προγραμματισμού και Επεξεργασίας Πληροφοριών του τομέα Τομέας Λογισμικού και Ανάπτυξης Εφαρμογών, υπό την επίβλεψη του Τσαουσίδα Βασίλειου. Η συγγραφή της εργασίας/διατριβής πραγματοποιήθηκε εξολοκλήρου από τον φοιτητή Ιατρού Κωνσταντίνο, υπό την καθοδήγηση και τις υποδείξεις του επιβλέποντά του, και με την αναγνώριση της συμβολής του Κουκή Γεώργιου.

Βεβαιώνεται ότι, κατά την εκπόνηση και τη συγγραφή της εργασίας του, ο φοιτητής τήρησε τα προβλεπόμενα από τον νόμο και τον αντίστοιχο εσωτερικό κανονισμό του Τμήματος, σεβάστηκε πλήρως τις Αρχές της Ακαδημαϊκής Ηθικής και του Κώδικα Δεοντολογίας, οι οποίες απαγορεύουν την παραποίηση των ερευνητικών/πειραματικών αποτελεσμάτων, την αναφορά ψευδών στοιχείων, την κατάχρηση της διανοητικής ιδιοκτησίας τρίτων και τη λογοκλοπή και ότι έγινε με σεβασμό στις αρχές.

Περίληψη

Η παρούσα διπλωματική εργασία έχει ως κύριο στόχο την αξιολόγηση και τη διασφάλιση της αξιοπιστίας και της ανθεκτικότητας κατανεμημένων συστημάτων σε περιβάλλοντα νέφους που χρησιμοποιούν την πλατφόρμα διαχείρισης περιεκτών (containers) Kubernetes. Η μέθοδος με την οποία γίνεται αυτή η αξιολόγηση είναι η μηχανική χάους (chaos engineering), η οποία δίνει τη δυνατότητα να εντοπιστούν και να αξιολογηθούν ευπάθειες του συστήματος μέσω ελεγχόμενης προσθήκης σφαλμάτων σε πραγματικές συνθήκες λειτουργίας. Ως πεδίο εφαρμογής επιλέχθηκε η εφαρμογή μικροϋπηρεσιών Online Boutique, η οποία προσομοιώνει ένα ηλεκτρονικό κατάστημα αγορών και χρησιμοποιείται συχνά ως πρότυπο συγκρίσεων (benchmarking). Για την εκτέλεση των πειραμάτων αξιοποιείται το εργαλείο Chaos Mesh, ενώ η μελέτη εστιάζει σε τρεις κρίσιμους άξονες για την απόδοση του συστήματος: τη σταθερότητα (stability), τον χρόνο ανάκαμψης (recovery time) και την ικανότητα απόκρισης (responsiveness). Τα σενάρια περιλαμβάνουν αστοχίες σε επίπεδο pods, υποβάθμιση του δικτύου και εξάντληση των πόρων του συστήματος, κατά την διάρκεια των οποίων παρατηρείται η συμπεριφορά του συστήματος πριν και μετά την εφαρμογή βελτιωτικών μέτρων.

Τα αποτελέσματα των πειραμάτων έδειξαν ότι οι προεπιλεγμένες ρυθμίσεις των μηχανισμών του Kubernetes, αν και βοηθητικές, δεν είναι πλήρως επαρκείς για τη διατήρηση της διαθεσιμότητας ενός συστήματος κατά τη διάρκεια ισχυρών καταπονήσεων, ενώ μετά από μία κατάρρευση του συστήματος δε δρουν άμεσα για την ανάκαμψή του, με αποτέλεσμα να προκύπτουν διαστήματα μη διαθεσιμότητας, γεγονός που επηρεάζει την εμπειρία των χρηστών. Μετά την εφαρμογή εξειδικευμένων παραμετροποιήσεων και την ενεργοποίηση μηχανισμών που προσφέρει το περιβάλλον του Kubernetes, όπως η αυτόματη οριζόντια κλιμάκωση, η αλλαγή των παραμέτρων των ανιχνευτών ετοιμότητας και ζωτικότητας και ο ορθός καταμερισμός των πόρων, το σύστημα παρουσίασε υψηλότερη διαθεσιμότητα, μείωση στον χρόνο απόκρισης υπό συνθήκες συμφόρησης και γρηγορότερη ανάκαμψη μετά από καταστροφικά σφάλματα.

Λέξεις-Κλειδιά: Kubernetes, Μηχανική Χάους, Μικροϋπηρεσίες, Αξιοπιστία συστήματος, Chaos Mesh, Ανθεκτικότητα, Κατανεμημένα Συστήματα.

Abstract

The main objective of this thesis is to evaluate and ensure the reliability and resilience of distributed systems in cloud environments using the Kubernetes container management platform. The method used for this evaluation is chaos engineering, which enables system administrators to identify and calibrate hidden vulnerabilities in the system by introducing controlled errors under real operating conditions. The application used is the Online Boutique microservices application, which simulates an online store. The Chaos Mesh tool is used to perform experiments, while the study focuses on three critical performance indicators: stability, recovery time, and responsiveness. The scenarios included pod-level failures, network degradation, and system resource exhaustion. The behavior of the system was observed before and after the implementation of improvement measures.

The results of the research showed that although the inherent mechanisms of Kubernetes are helpful, they are not entirely sufficient to maintain system availability during heavy loads, and after a system crash, they do not act immediately to recover it, leaving it unavailable to users for a significant period of time. After implementing new settings and mechanisms, such as automatic horizontal scaling, the configuration of readiness and vitality detectors, and the proper allocation of resources, the system showed higher availability, reduced response time under congestion conditions, and faster recovery after catastrophic failures.

Keywords: Kubernetes, Chaos Engineering, Microservices, System Reliability, Chaos Mesh, Resilience, Distributed Systems.

Πίνακας Περιεχομένων

1. Εισαγωγή	16
1.1. Στόχος και Εφαρμογή Διπλωματικής	16
1.2. Ερευνητικά Ερωτήματα	16
1.3. Δομή Εργασίας	17
1.4. Σχετική Βιβλιογραφία	18
2. Βασικές Έννοιες	20
2.1. Μονολιθική Αρχιτεκτονική	20
2.2. Εικονικές Μηχανές, Επόπτες, Περιέκτες και Μικροϋπηρεσίες	21
2.2.1. Εικονικές Μηχανές και Επόπτες	21
2.2.2. Περιέκτες	22
2.2.3. Μικροϋπηρεσίες	23
2.3. Docker και Kubernetes	24
2.3.1. Αρχιτεκτονική Kubernetes	25
2.3.1.1. Επίπεδο Ελέγχου	25
2.3.1.2. Κόμβοι Εκτέλεσης	25
2.3.1.3. Pods	26
2.3.1.4. Εξυπηρετητής Διεπαφής Προγραμματισμού Εφαρμογών	26
2.3.1.5. Χρονοπρογραμματιστής	27
2.3.1.6. Kubelet	27
2.3.1.7. Διαμεσολαβητής Υπηρεσιών	27
2.3.1.8. etcd	28
2.3.1.9. Μηχανή Εκτέλεσης Περιεκτών	28
2.3.2. Αντικείμενα του Περιβάλλοντος Kubernetes	28
2.4. Δοκιμές Καταπόνησης/Φορτίου	30
2.4.1. Εργαλεία Δοκιμών Καταπόνησης	30
2.5. Μηχανική Χάους	31
2.6. Διαφορές Δοκιμών Καταπόνησης και Μηχανικής Χάους	33
2.6.1. Γενικές Διαφορές και Στόχοι	33

2.6.2. Πειράματα Μηχανικής Χάους Πίεσης των Πόρων.....	33
2.7. Εργαλεία Μηχανικής Χάους	34
2.7.1. Chaos Monkey	34
2.7.2. Chaos Mesh.....	35
2.7.3. Chaos Toolkit.....	36
2.7.4. Litmus / LitmusChaos	37
2.7.5. Gremlin	37
2.7.6. Steadybit	37
2.7.7. ChaosBlade	38
2.7.8. Krkn	38
2.7.9. Σύγκριση Εργαλείων Μηχανικής Χάους	39
2.8. Εργαλεία Παρακολούθησης και Οπτικοποίησης	42
2.8.1. Prometheus	42
2.8.2. Grafana	43
2.8.3. Πίνακες ελέγχου των εργαλείων	43
3. Πειραματική Προσέγγιση και Αποτελέσματα	44
3.1. Πειραματική Διάταξη	44
3.1.1. Χαρακτηριστικά Κόμβων.....	44
3.1.2. Τοπολογία Συστάδας.....	45
3.1.3. Χρήση Εφαρμογής	45
3.1.4. Σύστημα Παρακολούθησης.....	46
3.2. Σύστημα Παρακολούθησης.....	46
3.2.1. Γενικές Μετρικές	46
3.2.2. Μετρικές σε επίπεδο Pods	47
3.2.3. Μετρικές Δικτύου	48
3.2.4. Μετρικές κατανάλωσης πόρων σε επίπεδο αντικειμένων ανάπτυξης	49
3.2.5. Μετρικές κατανάλωσης πόρων σε επίπεδο εικονικών μηχανών	51
4. Πειράματα Μηχανικής Χάους	52
4.1. Το περιβάλλον δημιουργίας πειραμάτων	52
4.2. Πειράματα σε επίπεδο Pod.....	53
4.2.1. Αποτυχία των pods της ιστοσελίδας για 10 λεπτά	54

4.3. Πειράματα σε επίπεδο Δικτύου	56
4.3.1. Περιορισμός του εύρους ζώνης στα 25 Kbps για 20 λεπτά	56
4.3.2. Περιορισμός του εύρους ζώνης στα 12 Kbps για 20 λεπτά	59
4.3.3. Περιορισμός του εύρους ζώνης στα 1 Kbps για 20 λεπτά	61
4.3.4. Προσθήκη 80ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά	64
4.3.5. Προσθήκη 150ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά	66
4.3.6. Προσθήκη 300ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά	68
4.4. Πειράματα σε επίπεδο πόρων	70
4.4.1. Μέτρια καταπόνηση πόρων επεξεργαστή και μνήμης	71
4.4.2. Βαριά καταπόνηση πόρων επεξεργαστή και μνήμης	74
4.5. Προσομοίωση αύξησης αριθμού χρηστών για 20 λεπτά	77
4.6. Βελτιώσεις Συστήματος	80
4.6.1. Στρατηγική κατανομής και υψηλής διαθεσιμότητας	80
4.6.2. Διαχείριση των πόρων και δυναμική κλιμάκωση	83
4.6.3. Ανθεκτικότητα δικτύου και μηχανισμοί ελέγχου συστήματος	84
5. Εφαρμογή μεθοδολογίας μηχανικής χάους	88
5.1. Ήπια υποβάθμιση της υπηρεσίας ολοκλήρωσης αγορών	88
5.2. Επιθετική υποβάθμιση της υπηρεσίας ολοκλήρωσης αγορών	93
5.3. Εφαρμογή Βελτιώσεων Ανθεκτικότητας	98
5.4. Επανεκτέλεση ήπιου ολοκληρωμένου σεναρίου	99
5.5. Επανεκτέλεση επιθετικού ολοκληρωμένου σεναρίου	103
6. Συμπεράσματα	108
6.1. Αξιολόγηση Ευρημάτων	108
6.1.1. Εξάντληση πόρων	108
6.1.2. Ανεπάρκεια αυτοϊασης σε επίπεδο pods	109
6.1.3. Μη γραμμική φύση της δικτυακής συμπεριφοράς	109
6.2. Αξιολόγηση Βελτιώσεων	109
6.3. Μελλοντικές Προοπτικές	110
6.3.1. Ενσωμάτωση eBPF	110
6.3.2. Μηχανική Χάους σε επίπεδο ασφάλειας	111
6.3.3. Αυτοματοποίηση μέσω Τεχνητής Νοημοσύνης	111

6.3.4. Ανθεκτικότητα σε σύνολα διατήρησης κατάστασης.....	111
Αναφορές.....	112

Λίστα Εικόνων

Εικόνα 1. Μικροϋπηρεσίες και Μονολιθική Αρχιτεκτονική. (Oracle Cloud Infrastructure (OCI), 2024)	21
Εικόνα 2. Οι δύο τύποι εποπτών. (Ganesh, 2024)	22
Εικόνα 3. Εφαρμογή με Αρχιτεκτονική Μικροϋπηρεσιών. (Prasath).....	24
Εικόνα 4. Απεικόνιση Αρχιτεκτονικής Kubernetes και των βασικών στοιχείων των κόμβων. (Kiprotich, 2023).....	25
Εικόνα 5. Η οργάνωση των περιεκτών και των Pods στους κόμβους. (Luksa, 2018).....	26
Εικόνα 6. Ο Εξυπηρετητής ΔΠΕ ως σημείο συμφόρησης. (Badawy, 2022)	27
Εικόνα 7. Παράδειγμα χώρων ονομάτων σε έναν κόμβο. (Extio Technology, 2023)	29
Εικόνα 8. Αντικείμενα ανάπτυξης και σύνολα αντιγράφων. (Maharjan, 2020)	29
Εικόνα 9. Σημείο σύνδεσης χρήστη σε υπηρεσία. (Ahamed, 2022).....	30
Εικόνα 10. Η μεθοδολογία των πειραμάτων μηχανικής χάους. (Johari, 2023)	32
Εικόνα 11. Η δομή του Chaos Mesh. (Chaos Mesh Authors, 2024).....	35
Εικόνα 12. Η αρχική σελίδα του Πίνακα Ελέγχου του Chaos Mesh.	36
Εικόνα 13. Συμβατότητα του Chaos Toolkit με Περιβάλλοντα και Επεκτάσεις. (Chaos Toolkit Contributors, 2024)	36
Εικόνα 14. Αρχιτεκτονική λογισμικού ChaosBlade. (The ChaosBlade Authors, 2025)	38
Εικόνα 15. Δομή της εφαρμογής Krkn με τις υποστηριζόμενες επεκτάσεις. (Krkn Contributors, 2025)	38
Εικόνα 16. Διάγραμμα Πειραματικής Διάταξης	44
Εικόνα 17. Οι γενικές μετρικές σε σταθερή κατάσταση.....	47
Εικόνα 18. Οι μετρικές σε επίπεδο Pods σε σταθερή κατάσταση.	48
Εικόνα 19. Δικτυακές μετρικές όταν το σύστημα βρίσκεται σε σταθερή κατάσταση (α).	49
Εικόνα 20. Δικτυακές μετρικές όταν το σύστημα βρίσκεται σε σταθερή κατάσταση (β).	49
Εικόνα 21. Μετρικές κατανάλωσης πόρων σε επίπεδο αντικειμένων ανάπτυξης και pods.	50
Εικόνα 22. Μετρικές κατανάλωσης πόρων σε επίπεδο κόμβων σε σταθερή κατάσταση λειτουργίας.....	51
Εικόνα 23. Η σελίδα δημιουργίας πειράματος στο Chaos Mesh.	52
Εικόνα 24. Επιλογή εμβέλειας, ονόματος και διάρκεια πειράματος.	53
Εικόνα 25. Χαρακτηριστικά και ορισμός του πειράματος αποτυχίας του pod της ιστοσελίδας. ...	54
Εικόνα 26. Οι επανεκκινήσεις του pod της ιστοσελίδας και η αναλογία έτοιμων pod ανά αντικείμενο ανάπτυξης.	54
Εικόνα 27. Διαμεταγωγή του χώρου ονομάτων και ρυθμός εισερχόμενων και εξερχόμενων πακέτων του δικτύου ανά υπηρεσία.	55
Εικόνα 28. Διαθεσιμότητα και καθυστερήσεις αιτημάτων.	55
Εικόνα 29. Ορισμός του πειράματος περιορισμού του εύρους ζώνης στα 25 Kbps.....	57
Εικόνα 30. Μετρικές για εύρος ζώνης 25 Kbps.....	58

Εικόνα 31. Ορισμός του πειράματος περιορισμού του εύρους ζώνης στα 12 Kbps.....	59
Εικόνα 32. Μετρικές Δικτύου για περιορισμό του εύρους ζώνης στα 12 Kbps.	60
Εικόνα 33. Ποσοστό χρήσης επεξεργαστικής ισχύος ανά αντικείμενο ανάπτυξης (12 Kbps).....	60
Εικόνα 34. Ορισμός του πειράματος περιορισμού του εύρους ζώνης στα 1 Kbps.....	62
Εικόνα 35. Μετρικές Δικτύου για περιορισμό του εύρους ζώνης στα 1 Kbps.	62
Εικόνα 36. Ποσοστό χρήσης επεξεργαστικής ισχύος ανά αντικείμενο ανάπτυξης (1 Kbps).....	63
Εικόνα 37. Ορισμός του πειράματος αύξησης καθυστέρησης κατά 80ms.	64
Εικόνα 38. Μετρικές δικτύου για αύξηση της καθυστέρησης κατά 80ms.	65
Εικόνα 39. Ορισμός του πειράματος αύξησης καθυστέρησης κατά 150ms.	66
Εικόνα 40. Μετρικές δικτύου για αύξηση της καθυστέρησης κατά 150ms.	67
Εικόνα 41. Ορισμός του πειράματος αύξησης καθυστέρησης κατά 300ms.	68
Εικόνα 42. Μετρικές δικτύου για αύξηση της καθυστέρησης κατά 300ms.	69
Εικόνα 43. Ποσοστό χρήσης επεξεργαστή για αύξηση της καθυστέρησης κατά 300ms.	69
Εικόνα 44. Ορισμός του πειράματος μέτριας καταπόνησης των πόρων.	71
Εικόνα 45. Μετρικές γεγονότων εξάντλησης μνήμης.	72
Εικόνα 46. Μετρικές χρήσης πόρων από το σύστημα κατά την μέτρια κατανάλωση πόρων (α). 72	
Εικόνα 47. Μετρικές χρήσης πόρων από το σύστημα κατά την μέτρια κατανάλωση πόρων (β). 72	
Εικόνα 48. Κατανάλωση πόρων ανά εικονική μηχανή.	73
Εικόνα 49. Ορισμός του πειράματος μεγάλης καταπόνησης των πόρων.	74
Εικόνα 50. Μετρικές επανεκκινήσεων και τερματισμών λόγω εξάντλησης μνήμης.	74
Εικόνα 51. Καθυστερήσεις p95 και p99.	75
Εικόνα 52. Ποσοστό κατανάλωσης διαθέσιμης μνήμης ανά μικροϋπηρεσία.	75
Εικόνα 53. Ποσοστό κατανάλωσης υπολογιστικής ισχύος και περιορισμός διεργασιών.	76
Εικόνα 54. Συνολική κατανάλωση πόρων του συστήματος ανά κόμβο.	76
Εικόνα 55. Εντολές για την αύξηση των εικονικών χρηστών.	78
Εικόνα 56. Ρυθμός μετάδοσης δεδομένων ανά υπηρεσία.	78
Εικόνα 57. Καθυστερήση p99 των αιτημάτων.	79
Εικόνα 58. Κατανάλωση διαθέσιμης υπολογιστικής ισχύος σε επίπεδο μικροϋπηρεσιών (αριστερά) και κόμβων (δεξιά).	79
Εικόνα 59. Περιορισμός χρήσης υπολογιστικής χρήσης ανά υπηρεσία.	80
Εικόνα 60. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.	90
Εικόνα 61. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.	90
Εικόνα 62. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.	91
Εικόνα 63. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.	91
Εικόνα 64. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.	95
Εικόνα 65. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.	95
Εικόνα 66. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.	95
Εικόνα 67. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.	96
Εικόνα 68. Διάγραμμα περιορισμού υπολογιστικής ισχύος ανά μικροϋπηρεσία.	96
Εικόνα 69. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.	99

Εικόνα 70. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.	99
Εικόνα 71. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.	100
Εικόνα 72. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.	100
Εικόνα 73. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.	103
Εικόνα 74. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.	103
Εικόνα 75. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.	104
Εικόνα 76. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.	104

Λίστα Πινάκων

Πίνακας 1. Σύγκριση Εργαλείων βάσει των περιβαλλόντων λειτουργίας και επεκτάσεών τους. .39	39
Πίνακας 2. Σύγκριση εργαλείων βάσει της ευκολίας εκμάθησης.....40	40
Πίνακας 3. Σύγκριση εργαλείων βάσει των δυνατοτήτων τους.41	41
Πίνακας 4. Χαρακτηριστικά των εικονικών μηχανών.44	44
Πίνακας 5. Οι μικροϋπηρεσίες του Online Boutique με την λειτουργία τους και την γλώσσα προγραμματισμού που χρησιμοποιήθηκε για την ανάπτυξή τους.45	45
Πίνακας 6. Επιπτώσεις ανά μετρική για σφάλμα του rod της ιστοσελίδας.56	56
Πίνακας 7. Ακριβείς μεταβολές των μετρικών λόγω του περιορισμού στα 25 Kbps.....58	58
Πίνακας 8. Ακριβείς μεταβολές των μετρικών λόγω του περιορισμού στα 12 Kbps.....61	61
Πίνακας 9. Ακριβείς μεταβολές των μετρικών λόγω του περιορισμού στα 1 Kbps.....63	63
Πίνακας 10. Αναλυτικές μετρήσεις των μετρικών δικτύου για αύξηση της καθυστέρησης κατά 80ms.....65	65
Πίνακας 11. Αναλυτικές μετρήσεις των μετρικών δικτύου για αύξηση της καθυστέρησης κατά 150ms.....67	67
Πίνακας 12. Αναλυτικές μετρήσεις του δικτύου για αύξηση της καθυστέρησης κατά 300ms.....70	70
Πίνακας 13. Αναλυτική μεταβολή κατανάλωσης πόρων κατά την μέτρια καταπόνηση τους.73	73
Πίνακας 14. Αναλυτική μεταβολή κατανάλωσης πόρων κατά την μεγάλη καταπόνηση τους.76	76
Πίνακας 15. Αναλυτικές μεταβολές των μετρικών κατά το πρώτο σενάριο μηχανικής χάους.92	92
Πίνακας 16. Αναλυτικές μεταβολές των μετρικών κατά το δεύτερο σενάριο μηχανικής χάους. ..97	97
Πίνακας 17. Αναλυτικές μεταβολές των μετρικών κατά την επανεκτέλεση του ήπιου σεναρίου μηχανικής χάους.....100	100
Πίνακας 18. Σύγκριση μετρικών κατά την διάρκεια της ήπιας ροής εργασιών πριν και μετά τις βελτιώσεις.101	101
Πίνακας 19. Αναλυτικές μεταβολές των μετρικών κατά την επανεκτέλεση του επιθετικού σεναρίου μηχανικής χάους.....104	104
Πίνακας 20. Σύγκριση μετρικών κατά την διάρκεια της επιθετικής ροής εργασιών πριν και μετά τις βελτιώσεις.106	106

Συντομογραφίες

ΑΠΠ – Ανοιχτή Πρωτοβουλία Περιεκτών

ΒΔ – Βάση Δεδομένων

ΔΠΕ – Διεπαφή Προγραμματισμού Εφαρμογών

ΕΔΧ – Εντελώς Δίκαιος Χρονοπρογραμματιστής

ΛΣ – Λειτουργικό Σύστημα

ΕΠΒΡ – Έλεγχος Πρόσβασης Βάσει Ρόλου

ΣΕ/ΣΔ – Συνεχούς Ενσωμάτωσης/Συνεχούς Διάθεσης

1. Εισαγωγή

1.1. Στόχος και Εφαρμογή Διπλωματικής

Ο κύριος στόχος της παρούσας διπλωματικής εργασίας είναι η εφαρμογή και η αξιολόγηση των τεχνικών μηχανικής χάους ως εργαλείο για τη διασφάλιση της αξιοπιστίας σε περιβάλλοντα περιεκτών και συγκεκριμένα διαχειριζόμενα από την πλατφόρμα Kubernetes. Η μηχανική χάους, σε αντίθεση με παραδοσιακές μεθόδους δοκιμής, στοχεύει στον εντοπισμό και βαθμονόμηση άγνωστων αδυναμιών του συνολικού συστήματος μέσω ελεγχόμενης εισαγωγής καταπονήσεων και σφαλμάτων. Συγκεκριμένα, η εργασία εστιάζει στην αξιολόγηση τριών σημαντικών δεικτών για την απόδοση ενός συστήματος:

1. **Σταθερότητα (stability):** Η ικανότητα του συστήματος να παραμένει λειτουργικό υπό συνθήκες πίεσης των πόρων ή αστάθειας του δικτύου.
2. **Χρόνος ανάκαμψης (recovery time):** Ο χρόνος που χρειάζεται το σύστημα για να επανέλθει σε σταθερή κατάσταση λειτουργίας (steady state) μετά από κάποιο καταστροφικό σφάλμα.
3. **Ικανότητα απόκρισης (responsiveness):** Η ικανότητα του συστήματος να διατηρεί αποδεκτούς χρόνους απόκρισης / καθυστέρησης (latency) για τους χρήστες κατά την διάρκεια σφαλμάτων ή καταπονήσεων.

Ως πεδίο εφαρμογής επιλέχθηκε η εφαρμογή Online Boutique, η οποία βασίζεται σε αρχιτεκτονική μικροϋπηρεσιών και αποτελείται από 11 διαφορετικές διασυνδεδεμένες υπηρεσίες. Το πειραματικό μέρος θα εκτελεστεί μέσω του εργαλείου Chaos Mesh, το οποίο δίνει στους διαχειριστές την δυνατότητα επιλογής σεναρίων μηχανικής χάους σε διάφορα επίπεδα όπως υποδομής, δικτύου και πόρων.

1.2. Ερευνητικά Ερωτήματα

Οι δοκιμές και τα πειράματα που πραγματοποιήθηκαν στα πλαίσια της εργασίας δομούνται γύρω από τέσσερα βασικά ερωτήματα:

Ερώτημα 1: Ποια είναι τα όρια υποβάθμισης του καταναμεμένου συστήματος (συστάδας) σε επίπεδο υποδομής, δικτύου και πόρων τα οποία αν ξεπεραστούν μία τυπική μικροϋπηρεσιακή εφαρμογή καθίσταται μη λειτουργική;

Ερώτημα 2: Επαρκούν οι προεπιλεγμένοι παράμετροι των μηχανισμών του Kubernetes για την εξασφάλιση της συνεχούς λειτουργίας της εφαρμογής σε περίπτωση απώλειας στοιχείων υποδομής (pods), ή απαιτούνται επιπλέον αρχιτεκτονικές ρυθμίσεις από τους διαχειριστές του;

Ερώτημα 3: Πώς μπορεί να επηρεάσει ένας θορυβώδης γειτονικός κόμβος (noisy neighbor) τη σταθερότητα της συστάδας λόγω έλλειψης πόρων και πώς μπορούν τα σενάρια μηχανικής χάους σε επίπεδο πόρων να εντοπίσουν αυτή την ευπάθεια;

Ερώτημα 4: Σε τι βαθμό μπορούν να βελτιώσουν την σταθερότητα και την αυτοΐαση του συστήματος επιπλέον μηχανισμοί βελτίωσης όπως η αυτόματη οριζόντια κλιμάκωση και η προσαρμογή των παραμέτρων των ανιχνευτών (probes) κατά την διάρκεια εκτέλεσης πειραμάτων μηχανικής χάους;

1.3. Δομή Εργασίας

Η οργάνωση της εργασίας γίνεται στα εξής κεφάλαια:

Κεφάλαιο 2 – Βασικές έννοιες: Σε αυτό το κεφάλαιο παρουσιάζονται το θεωρητικό υπόβαθρο της αρχιτεκτονικής μικροϋπηρεσιών και Kubernetes, οι θεμελιώδεις αρχές της Μηχανικής Χάους καθώς και διαθέσιμα εργαλεία πειραμάτων μηχανικής χάους, δοκιμών καταπόνησης και παρακολούθησης του συστήματος.

Κεφάλαιο 3 – Πειραματική Προσέγγιση: Εδώ περιγράφεται η υποδομή της συστάδας, τα εργαλεία παρακολούθησης και μηχανικής χάους που χρησιμοποιούνται για την αξιολόγηση της σταθερής κατάστασης και των επιπτώσεων των πειραμάτων.

Κεφάλαιο 4 – Πειράματα Μηχανικής Χάους: Σε αυτό το κεφάλαιο παρουσιάζονται τα πειράματα, τα αποτελέσματα τους και καταγράφονται χωρίς την εφαρμογή βελτιώσεων.

Κεφάλαιο 5 – Εφαρμογή Μεθοδολογίας Μηχανικής Χάους: Εδώ δίνεται ο ορισμός σύνθετων σεναρίων μηχανικής χάους και εφαρμόζεται η μεθοδολογία της μηχανικής χάους, η οποία αναλύεται στο κεφάλαιο 2.

Κεφάλαιο 6 – Συμπεράσματα: Συνοψίζονται τα ευρήματα και δίνεται τελική απάντηση στα ερευνητικά ερωτήματα που τέθηκαν στο προηγούμενο κεφάλαιο (1.2).

1.4. Σχετική Βιβλιογραφία

Η διασφάλιση της αξιοπιστίας και της ανθεκτικότητας (resilience) συστημάτων με αρχιτεκτονική μικροϋπηρεσιών αποτελεί σημαντικό πεδίο έρευνας, καθώς η πολυπλοκότητα των καταναεμμένων συστημάτων καθιστά τις συμβατικές μεθόδους αναποτελεσματικές. Σε αυτή την διπλωματική εργασία, γίνεται εφαρμογή πειραμάτων μηχανικής χάους, σύμφωνα με την μεθοδολογία της, για την αξιολόγηση και την βελτίωση τέτοιων συστημάτων. Σύμφωνα με την πρόσφατη ανασκόπηση των Mailewa et al. [24], η μηχανική χάους αποτελεί «μια αυστηρή διαδικασία που επικεντρώνεται στην ενίσχυση της ανθεκτικότητας των συστημάτων λογισμικού μέσω εκτέλεσης πειραμάτων σε περιβάλλοντα παραγωγής (in production)». Η βασική αρχή της μεθοδολογίας, σύμφωνα με τους Mailewa et al., είναι η «ελεγχόμενη αποτυχία» (controlled failure). Η παρούσα διπλωματική εργασία δεν παρουσιάζει την μηχανική χάους μόνο σε θεωρητικό βαθμό, αλλά γίνεται και πρακτική εφαρμογή της.

Σε πρακτικό επίπεδο, στην εργασία του Akuthota [3], η μηχανική χάους εφαρμόζεται ως πειραματική διαδικασία, η οποία στοχεύει στην ανάδειξη αδυναμιών ενός συστήματος μέσω ελεγχόμενης δημιουργίας σφαλμάτων. Η συγκεκριμένη μελέτη εστιάζει σε εργαλεία που εκτελούνται σε επίπεδο κώδικα και βιβλιοθηκών, όπως είναι το Simmy¹, το οποίο λειτουργεί συνεργατικά με το Polly² σε περιβάλλον .NET. Η προσέγγιση επιτρέπει την εκτέλεση πειραμάτων μηχανικής χάους στην λογική της εφαρμογής, εκμεταλλευόμενη αδυναμίες όπως οι εξαιρέσεις (exceptions) (Akuthota, 2023). Αντίθετα, στην παρούσα διπλωματική υπάρχει σημαντική διαφοροποίηση στην προσέγγιση, καθώς τα πειράματα γίνονται χωρίς να υπάρχει γνώση ως προς την γλώσσα ανάπτυξης της εφαρμογής, αλλά με χρήση του εργαλείου Chaos Mesh.

Παράλληλα, η μελέτη του Nandigam [25] παρέχει μία ενδελεχή επισκόπηση για τις μετρικές αξιοπιστίας, όπως η διαθεσιμότητα και ο χρόνος ανάκαμψης, τονίζοντας την σημασία του σωστού καθορισμού της σταθερής κατάστασης (steady state) του συστήματος. Ο Nandigam προσφέρει σημαντικές πληροφορίες για το θεωρητικό υπόβαθρο της μεθοδολογίας της μηχανικής χάους (Nandigam, 2024). Συμπληρωματικά σε αυτή την μελέτη η έρευνα των Ahmad et al. [2] εξετάζει την επίδραση της μηχανικής χάους υπό μεταβαλλόμενο φορτίο από την πλευρά των ενεργών χρηστών κατά την διάρκεια των πειραμάτων (Ahmad, Al-Qora'n, & Zayed, 2024). Τα ευρήματα αυτών των δύο άρθρων υποδεικνύουν την βαρύτητα του φορτίου που παράγουν οι χρήστες στα συστήματα που εκτελούνται σε περιβάλλοντα νέφους.

Για την συγγραφή, λήφθηκε υπόψη και εργασία «Automated Testing and Resilience of Microservice's Network-link using Istio Mesh» [18] η οποία χρησιμοποιεί ως εφαρμογή αναφοράς, επίσης, ένα ηλεκτρονικό κατάστημα αγορών. Η διαφοροποίηση έγκειται στον μηχανισμό διαχείρισης των σφαλμάτων που εντοπίζονται για την ενίσχυση της ανθεκτικότητας και της αξιοπιστίας του συστήματος. Στην εργασία των Karn et al. το εργαλείο το οποίο χρησιμοποιείται είναι το Istio Service Mesh το οποίο αποτελεί ένα πλέγμα υπηρεσιών (Service Mesh) όπως αναφέρεται και στο όνομά του. Το εργαλείο αυτό υλοποιεί τους μηχανισμούς ανθεκτικότητας και κάνει την διαχείριση κίνησης (Karn, Das, Pant, Heikkonen, & Kanth, 2022). Αντίθετα στην πειραματική διαδικασία της διπλωματικής εργασίας δεν χρησιμοποιούνται τρίτες

¹ Simmy: Βιβλιοθήκη με σενάρια μηχανικής χάους για την εισαγωγή σφαλμάτων (fault-injection) σε οικοσύστημα .NET. <https://github.com/Polly-Contrib/Simmy>

² Polly: Βιβλιοθήκη με εργαλεία ανθεκτικότητας .NET εφαρμογών και χειρισμού παροδικών σφαλμάτων. <https://www.pollydocs.org/index.html>

εφαρμογές για την εφαρμογή βελτιώσεων, αλλά δυνατότητες και μηχανισμοί που ήδη υπάρχουν στο Kubernetes, αλλά μπορεί να μην χρησιμοποιούνται εγγενώς ή να χρειάζονται παραμετροποίηση.

Για την επιλογή των πειραμάτων, μελετήθηκε το άρθρο των Flora et al. με τίτλο «Study on the Aging and Fault Tolerance of Microservices in Kubernetes» [12], του οποίου το αντικείμενο είναι η γήρανση του λογισμικού (software aging) και η σταδιακή εξάντληση των πόρων σε εφαρμογές που εκτελούνται σε περιβάλλοντα Kubernetes. Το συμπέρασμα του παραπάνω άρθρου είναι ότι οι προεπιλεγμένοι ανιχνευτές (probes) του περιβάλλοντος Kubernetes πολλές φορές κρίνονται αναποτελεσματικοί στον εντοπισμό της σταδιακής υποβάθμισης και δεν προλαβαίνουν την κατάρρευση του συστήματος (Flora, Gonçalves, Teixeira, & Antunes, 2022). Η παρούσα εργασία επιβεβαιώνει τα ευρήματα του άρθρου μέσω σεναρίων χάους σε επίπεδο πόρων, αλλά δεν περιορίζεται σε αυτό. Σε επόμενα βήματα γίνεται εφαρμογή βελτιώσεων και αξιολόγηση αυτών. Για την επιλογή αυτών των βελτιώσεων λήφθηκαν υπόψη τα ευρήματα των Wang et al. [38], τα οποία υποδεικνύουν την αδυναμία απλών μεθόδων για τον εντοπισμό ορισμένων σφαλμάτων και προτείνουν μία μέθοδο που λαμβάνει υπόψη την αρχιτεκτονική μικροϋπηρεσιών του συστήματος (Wang, και συν., 2024). Κάποιες από τις βελτιώσεις είναι η παραμετροποίηση των χρονικών ορίων των ανιχνευτών και η πιο αυστηρή κατανομή των πόρων στις απαιτήσεις του λογισμικού, μειώνοντας τον χρόνο ανάκαμψης και μετριάζοντας τις επιπτώσεις που αναφέρονται στο άρθρο.

2. Βασικές Έννοιες

2.1. Μονολιθική Αρχιτεκτονική

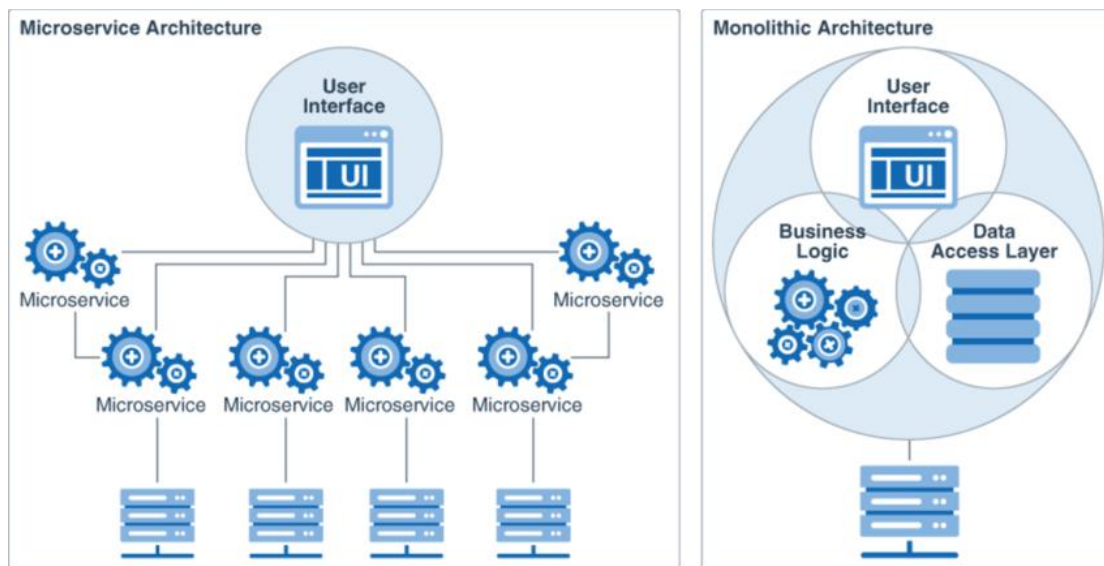
Τα προηγούμενα χρόνια, η πλειοψηφία των εφαρμογών εκτελείτο ως μία μεγάλη περίπλοκη διεργασία, όπως φαίνεται και στην Εικόνα 1. Η κάθε εφαρμογή είχε ολόκληρα αντίγραφα της μοιρασμένα σε πολλούς υπολογιστές οι οποίοι λειτουργούσαν ως εξυπηρετητές (servers). Όλες οι μονολιθικής προσέγγισης εφαρμογές αποτελούνται από πολλά τμήματα που είναι αυστηρά αλληλεξαρτώμενα μεταξύ τους. Αυτά τα τμήματα αναπτύσσονται, εκτελούνται και γίνεται η διαχείρισή τους, από την εκάστοτε ομάδα, ως ενιαία οντότητα, αφού όλα τους εκτελούνται ως μία διεργασία από την πλευρά του λειτουργικού συστήματος. Αυτή η μονολιθική προσέγγιση, όπως ορίζεται στην βιβλιογραφία, χρησιμοποιείται ακόμα σε μεγάλο αριθμό εμπορικών εφαρμογών που βρίσκονται αυτήν την στιγμή στην αγορά.

Οποιοσδήποτε τροποποιήσεις σε ένα συγκεκριμένο τμήμα της εφαρμογής απαιτούν επανεκκίνηση ολόκληρης της εφαρμογής. Αυτό συμβαίνει λόγω της άμεσης αλληλεξάρτησης όλων των υπολειτουργιών της εφαρμογής, αλλά και της αλληλεξάρτησης όλων των απαραίτητων βιβλιοθηκών. Εξαιτίας της πολυπλοκότητας της δομής του μονολιθικού συστήματος και της ροής εργασιών του, κάθε έκδοση της εφαρμογής έχει μεγάλους κύκλους ζωής και οι ενημερώσεις είναι σπάνιες (Luksa, 2018). Οι μικρές και οι απλές εφαρμογές με μονολιθική αρχιτεκτονική έχουν αρκετά πλεονεκτήματα αφού είναι εύκολες στην ανάπτυξη, την δοκιμή, την εκτέλεση και την κλιμάκωση (scale) τους. Αυτά τα πλεονεκτήματα μετριάζονται ή και χάνονται, με την πάροδο του χρόνου, σε εφαρμογές που είναι επιτυχημένες και αναβαθμίζονται, καθώς γίνονται πιο περίπλοκες και καταλήγουν να γίνονται ένας μεγάλος «μονόλιθος». Αυτό συμβαίνει γιατί η εφαρμογή παύει να είναι απλή (Chen, Li, & Li, 2017).

Επιπλέον, όταν οι απαιτήσεις σε φόρτο εργασίας αυξάνονται, με πιο απλό παράδειγμα την αύξηση του αριθμού χρηστών της εφαρμογής, τότε πρέπει να γίνει κατάλληλη κλιμάκωση του συστήματος για να μπορεί το σύστημα να ανταπεξέλθει στις απαιτήσεις. Οι επιλογές είναι δύο με πρώτη την κατακόρυφη κλιμάκωση του συστήματος, δηλαδή την ενίσχυση των ήδη υπάρχοντων εξυπηρετητών με προσθήκη περισσότερων ή ισχυρότερων επεξεργαστών, περισσότερης μνήμης ή άλλων εξαρτημάτων που απαιτεί η εφαρμογή (scale up). Η δεύτερη επιλογή είναι η αύξηση του αριθμού των εξυπηρετητών, δηλαδή η οριζόντια κλιμάκωση του συστήματος (scale out). Αν και απλές στην σύλληψη, και οι δύο προσεγγίσεις παρουσιάζουν σημαντικά μειονεκτήματα. Η κατακόρυφη κλιμάκωση μπορεί να γίνει αρκετά ακριβή από οικονομικής πλευράς, ενώ η οριζόντια συνήθως απαιτεί αλλαγές στον κώδικα της εφαρμογής, οι οποίες πιθανώς να είναι δύσκολα εφαρμόσιμες ή και σε περιπτώσεις αδύνατες. Αυτό γιατί κάποιες από τις υπολειτουργίες της εφαρμογής μπορεί να μην υποστηρίζουν οριζόντια κλιμάκωση, με συνηθέστερο παράδειγμα τις σχεσιακές βάσεις (MySQL, PostgreSQL, Oracle Database). Αν έστω και μία λειτουργία δεν μπορεί να κλιμακωθεί, τότε όλη η εφαρμογή δεν έχει την δυνατότητα να κλιμακωθεί (Luksa, 2018).

Τα πλεονεκτήματα που συνεχίζουν να διατηρούν οι εφαρμογές μονολιθικής προσέγγισης είναι η χρήση κοινού χώρου μνήμης για όλες τις λειτουργίες και τις συναρτήσεις που καλούν. Αυτό ισοδυναμεί με μεγάλη ταχύτητα και μηδενικές δικτυακές καθυστερήσεις. Η αύξηση, όμως, της πολυπλοκότητας ενισχύει το γεγονός ότι ο μοναδικός αυτός χώρος μνήμης αποτελεί και μοναδικό σημείο αποτυχίας (Single Point of Failure, SPoF). Με άλλα λόγια ένα σφάλμα μνήμης μπορεί να οδηγήσει σε κατάρρευση ολόκληρου του συστήματος. Σε τέτοιου είδους συστήματα η μηχανική χάους είναι δύσκολο έως αδύνατο να εφαρμοστεί

γιατί είναι δύσκολος ο εντοπισμός της πηγής του προβλήματος σε ένα πείραμα, αφού δεν υπάρχουν σαφή όρια μεταξύ των διαφορετικών λειτουργιών-υπηρεσιών του συστήματος.



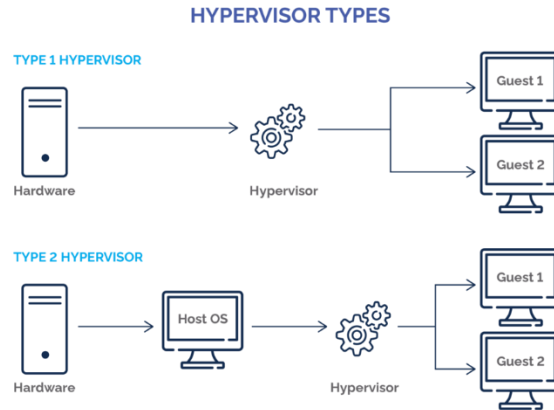
Εικόνα 1. Μικροϋπηρεσίες και Μονολιθική Αρχιτεκτονική. (Oracle Cloud Infrastructure (OCI), 2024)

2.2. Εικονικές Μηχανές, Επόπτες, Περιέκτες και Μικροϋπηρεσίες

2.2.1. Εικονικές Μηχανές και Επόπτες

Εικονική μηχανή (virtual machine, VM) είναι ένα υπολογιστικό περιβάλλον το οποίο λειτουργεί σαν ξεχωριστό και αυτόνομο σύστημα με δική του επεξεργαστική ισχύ, μνήμη, περιβάλλον δικτύου και αποθηκευτικό χώρο. Η εικονική μηχανή δημιουργείται από ένα μέρος των υπολογιστικών πόρων του υλικού (hardware) του φυσικού υπολογιστή.

Το φυσικό μηχάνημα πάνω στο οποίο εκτελούνται οι εικονικές μηχανές ονομάζεται σύστημα φιλοξενίας (host), ενώ όλες οι εικονικές μηχανές που χρησιμοποιούν τους πόρους ονομάζονται φιλοξενούμενα συστήματα (guests). Ο επόπτης (hypervisor) διαχειρίζεται και κατανέμει τους πόρους (επεξεργαστική ισχύ, μνήμη, αποθηκευτικό χώρο) ώστε να μοιράζονται αποδοτικά μεταξύ των υπάρχοντων συστημάτων φιλοξενίας και των νέων που μπορεί να δημιουργηθούν από κάποιον χρήστη (Red Hat Authors, 2024). Οι επόπτες χωρίζονται σε δύο τύπους (βλ. Εικόνα 2). Ο τύπος 1 αναφέρεται σε αυτούς που εκτελούνται απευθείας πάνω στο υλικό ενός υπολογιστή. Οι εικονικές μηχανές, σε αυτή την περίπτωση, προγραμματίζονται άμεσα στο διαθέσιμο υλικό. Οι πιο γνωστοί επόπτες αυτής της κατηγορίας είναι οι Kernel-based Virtual Machine, Hyper-V και VMware ESXi. Στον τύπο 2 ανήκουν οι επόπτες που εκτελούνται πάνω σε κάποιο λειτουργικό σύστημα όπως κάθε τυπικό πρόγραμμα. Οι δημοφιλέστεροι είναι τα VirtualBox και VMware Workstation.



Εικόνα 2. Οι δύο τύποι εποπτών. (Ganesh, 2024)

2.2.2. Περιέκτες

Οι περιέκτες (containers) είναι περιορισμένες σε μέγεθος και ελαφριά περιβάλλοντα εκτέλεσης τα οποία μοιράζονται τον πυρήνα του λειτουργικού συστήματος, αλλά λειτουργούν ανεξάρτητα το ένα από το άλλο. Ο κάθε περιέκτης περιέχει τον κώδικα και όλες τις απαραίτητες βιβλιοθήκες στις οποίες βασίζεται η εφαρμογή ώστε να εκτελείται γρήγορα και αξιόπιστα στο υπολογιστικό περιβάλλον (Jadhav, Sable, Suresh, & Hanawal, 2023). Τα πλεονεκτήματα που παρέχουν οι περιέκτες είναι:

- **Ευκολία στη μεταφορά:** Μπορούν να εκτελεστούν σε έναν υπολογιστή, σε εξυπηρετητή και σε περιβάλλοντα νέφους (cloud).
- **Απομόνωση:** Κάθε περιέκτης είναι απομονωμένος από τους υπόλοιπους, μειώνοντας τις πιθανότητες συγκρούσεων και τα προβλήματα εξάρτησης που μπορεί να προκύψουν.
- **Αυτοματισμός και ευελιξία:** Επιτρέπουν την δημιουργία αντιγράφων εύκολα και γρήγορα, καθώς και την ανάπτυξη εφαρμογών σε μεγάλη κλίμακα.
- **Ελαφριά:** Σε σύγκριση με τις εικονικές μηχανές, οι περιέκτες έχουν μικρότερες απαιτήσεις σε υπολογιστικούς πόρους κάνοντας τους πιο αποδοτική λύση, αφού δεν απαιτούν ξεχωριστό λειτουργικό σύστημα για την κάθε μία.

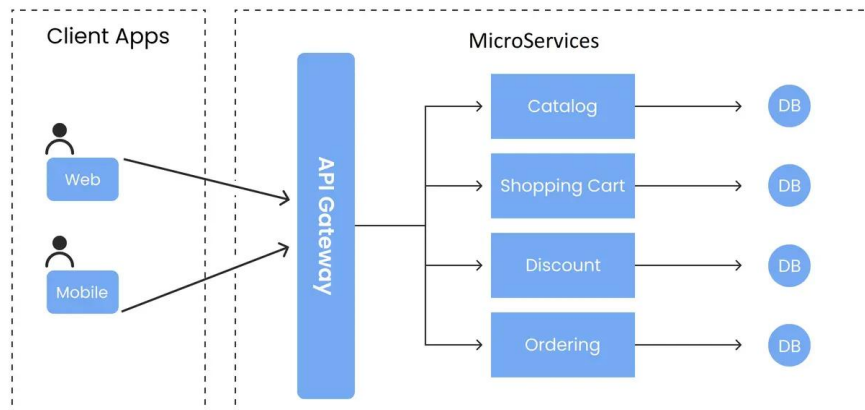
Η σημαντική διαφορά μεταξύ των εικονικών μηχανών και των περιεκτών είναι το γεγονός ότι οι δεύτεροι δεν είναι πραγματικά αντικείμενα, αλλά αποτελούν έναν εικονικό διαχωρισμό που δημιουργείται από δύο χαρακτηριστικά του Linux Kernel, τους χώρους ονομάτων (namespaces) και τις ομάδες ελέγχου (cgroups). Οι χώροι ονομάτων αποτελούν βασικό αντικείμενο του Kubernetes και θα εξηγηθούν στο αντίστοιχο κεφάλαιο. Οι ομάδες ελέγχου δίνουν στους διαχειριστές την δυνατότητα ελέγχου των πόρων, περιορίζοντας την επεξεργαστική ισχύ και την μνήμη που διατίθεται σε έναν περιέκτη. Όταν κάποια διεργασία ξεπεράσει ορισμένα όρια χρήσης ενεργοποιεί τον τερματιστή εξάντλησης μνήμης (Out of Memory Killer) και την τερματίζει άμεσα. Επιπλέον ο χρόνος εκκίνησης ενός περιέκτη είναι αρκετές τάξεις μεγέθους μικρότερος από αυτόν των εικονικών μηχανών, καθώς δεν έχουν τόσο μεγάλη λειτουργική επιβάρυνση (overhead) λόγω έλλειψης λειτουργικού συστήματος. Οπότε κατά την εκκίνηση χρειάζεται απλά εκκίνηση της υπηρεσίας που στεγάζεται σε αυτόν.

2.2.3. Μικροϋπηρεσίες

Οι μικροϋπηρεσίες (microservices) – γνωστές και ως αρχιτεκτονική μικροϋπηρεσιών (microservice architecture) —είναι η δόμηση μίας εφαρμογής ως ένα σετ από υπηρεσίες οι οποίες είναι ανεξάρτητα εκτελέσιμες και χαλαρά εξαρτώμενες μεταξύ τους. Η κάθε υπηρεσία ενθυλακώνει μία ξεχωριστή λειτουργία του συστήματος και είναι προσβάσιμη από τις άλλες μέσω ελαφρών πρωτοκόλλων επικοινωνίας. Η αρχιτεκτονική μικροϋπηρεσιών είναι προσανατολισμένη στις υπηρεσίες της εφαρμογής και τις διαχωρίζει με ξεκάθαρα όρια για να υπάρχει η ανεξάρτητη δυνατότητα ανάπτυξης της κάθε μιας από το υπόλοιπο περιβάλλον (Newman, 2021). Οι μικροϋπηρεσίες διευκολύνουν τις επί μέρους ομάδες να εκτελούν τις απαραίτητες εργασίες ανεξάρτητα η μία από την άλλη μειώνοντας την εξάρτηση του κώδικα μεταξύ των τμημάτων της εφαρμογής, αυξάνοντας την αναγνωσιμότητα του κώδικα και προσφέροντας την δυνατότητα κλιμάκωσης των τμημάτων που την υποστηρίζουν ανεξάρτητα από το υπόλοιπο σύστημα.

Συχνά μία μονολιθική εφαρμογή παρουσιάζει δυσκολίες στην κλιμάκωσή της. Μία λύση σε αυτό το πρόβλημα είναι ο διαχωρισμός της σε επί μέρους μικρότερα τμήματα τα οποία θα εκτελούνται ανεξάρτητα μεταξύ τους. Αυτά τα τμήματα είναι οι μικροϋπηρεσίες και η κάθε μία από αυτές εκτελεί μία ανεξάρτητη διεργασία και επικοινωνεί με τις υπόλοιπες μέσω απλών και καλά ορισμένων διεπαφών προγραμματισμού (APIs), όπως φαίνεται και στην Εικόνα 3. Οι μέθοδοι επικοινωνίας μπορεί να είναι είτε σύγχρονοι (π.χ. με χρήση του πρωτοκόλλου HTTP), είτε ασύγχρονοι (π.χ. με χρήση του πρωτοκόλλου AMQP), ενώ η γλώσσα κάθε μικροϋπηρεσίας μπορεί να επιλεγεί ξεχωριστά ώστε να διευκολύνει τον προγραμματιστή και να είναι πιο αποδοτική η λειτουργία της, ενώ αλλαγές στην ανάπτυξη μίας μικροϋπηρεσίας δεν απαιτούν ανάλογες αλλαγές στις υπόλοιπες. Γενικά είναι μεταξύ τους τεχνολογικά ανεξάρτητες τόσο στον τρόπο ανάπτυξης όσο και στον τρόπο αποθήκευσης. (Luksa, 2018)

Όπως η μονολιθική αρχιτεκτονική, έτσι και η αρχιτεκτονική μικροϋπηρεσιών έχει, αντίστοιχα, κάποια μειονεκτήματα. Το γεγονός ότι η εφαρμογή θα είναι μοιρασμένη σε διαφορετικά τμήματα που το καθένα θα επιτελεί την δική του λειτουργία, σημαίνει πως για να ολοκληρωθούν αιτήματα θα πρέπει να επικοινωνούν μεταξύ τους. Το δίκτυο όμως δεν είναι πάντοτε αξιόπιστο, τα πακέτα μεταξύ των υπηρεσιών θα χρειάζονται χρόνο μετακίνησης (latency) από μικροϋπηρεσία σε μικροϋπηρεσία και το εύρος ζώνης (bandwidth) δεν είναι άπειρο. Σε αυτό το σημείο εισάγεται η μηχανική χάους, η οποία μέσω των πειραμάτων, βοηθάει στον εντοπισμό αδυναμιών και σφαλμάτων του συστήματος με στόχο την διόρθωση ή τον περιορισμό τους, αυξάνοντας την ανθεκτικότητα και την αξιοπιστία του.



Εικόνα 3. Εφαρμογή με Αρχιτεκτονική Μικροϋπηρεσιών. (Prasath)

2.3. Docker και Kubernetes

Το Docker είναι ένα εργαλείο ανοικτού κώδικα το οποίο παρέχει την δυνατότητα ενός συστηματικού τρόπου αυτοματοποίησης για την γρηγορότερη εκτέλεση εφαρμογών σε περιβάλλοντα Linux μέσα σε περιέκτες (Bernstein, 2014). Χρησιμοποιείται για την ανάπτυξη εφαρμογών και λειτουργιών (DevOps) επειδή επιτρέπει την αποστολή λογισμικού, σε οποιοδήποτε περιβάλλον υποστηρίζει Docker, γρήγορα και αξιόπιστα.

Από την άλλη, το Kubernetes σύμφωνα και με την επίσημη ιστοσελίδα της πλατφόρμας ορίζεται ως εξής:

“Το Kubernetes είναι μία φορητή, επεκτάσιμη και ανοικτού κώδικα πλατφόρμα για την διαχείριση εργασιών και υπηρεσιών σε περιβάλλοντα περιεκτών, η οποία διευκολύνει την ρύθμιση και την αυτοματοποίησή τους. Διαθέτει ένα μεγάλο και γρήγορα αναπτυσσόμενο οικοσύστημα. Οι υπηρεσίες, η υποστήριξη και τα εργαλεία του Kubernetes είναι ευρέως διαθέσιμα.” (The Kubernetes Authors, 2024)

Συνεπώς, το Kubernetes είναι μία πλατφόρμα που χρησιμοποιείται για την δημιουργία, την εκτέλεση και την διαχείριση κατανεμημένων συστημάτων και εφαρμογών. Οι εφαρμογές μπορεί να έχουν διαφορετικά είδη οργάνωσης και μεγέθους, αλλά γενικά αποτελούνται πάντοτε από ένα ή περισσότερα προγράμματα τα οποία εκτελούνται σε ξεχωριστές μηχανές, είτε φυσικές είτε εικονικές. Αυτά τα προγράμματα δέχονται κάποια είσοδο ή κάποιες εισόδους και αφού εκτελέσουν την λειτουργία τους επιστρέφουν τα αποτελέσματα στη μορφή που είναι ρυθμισμένη από τον προγραμματιστή. Την μορφή αυτή την αποφασίζει ο προγραμματιστής λαμβάνοντας υπόψη αν η έξοδος θα δοθεί στον τελικό χρήστη ή αν θα δοθεί ως νέα είσοδος σε κάποιο άλλο πρόγραμμα του συστήματος (Burns, Beda, Hightower, & Evenson, 2022). Οι διαχειριστές επιλέγουν να χρησιμοποιήσουν την συγκεκριμένη πλατφόρμα για να αυτοματοποιήσουν την ανάπτυξη και την κλιμάκωση της εφαρμογής ή των υπολειτουργιών της εφαρμογής (μικροϋπηρεσιών) ώστε να εξασφαλίζουν την διαθεσιμότητα της για τους χρήστες.

2.3.1. Αρχιτεκτονική Kubernetes

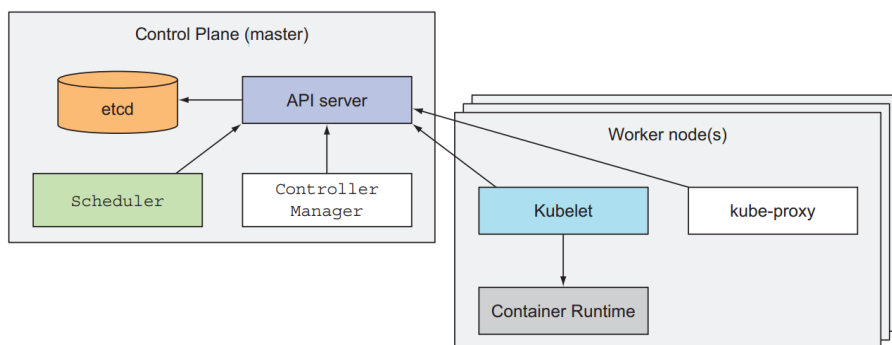
Με τον όρο αρχιτεκτονική Kubernetes γίνεται αναφορά στην διαχείριση των φιλοξενούμενων συστημάτων, τα οποία είναι οργανωμένα σε συστάδες (clusters). Μία συστάδα στο περιβάλλον του Kubernetes αποτελείται από το επίπεδο ελέγχου (control plane) και ένα σύνολο από κόμβους εκτέλεσης (worker nodes). Μέρος της αρχιτεκτονικής είναι και τα απαραίτητα κομμάτια κώδικα που εκτελούνται, ξεχωριστά στα πλαίσια του επιπέδου ελέγχου και των κόμβων εκτέλεσης (The Kubernetes Authors, 2024).

2.3.1.1. Επίπεδο Ελέγχου

Το επίπεδο ελέγχου (control plane) είναι ένας κόμβος, ο οποίος διαχειρίζεται και συντονίζει τον προγραμματισμό και την εκτέλεση των εφαρμογών που βρίσκονται εντός των περιεκτών. Ελέγχει την κατάσταση της συστάδας, την κατανομή των πόρων και παίρνει αποφάσεις για τον προγραμματισμό όλων των εφαρμογών και την κλιμάκωση των μικροϋπηρεσιών τους σε όλους τους κόμβους εκτέλεσης. Όταν ο διαχειριστής του συστήματος υποβάλλει στο επίπεδο ελέγχου μία λίστα μικροϋπηρεσιών ή εφαρμογών, η πλατφόρμα τα αναθέτει στους κόμβους εκτέλεσης. Τόσο για τους προγραμματιστές, όσο και για τον διαχειριστή του συστήματος δεν παίζει κανένα ρόλο σε ποιον συγκεκριμένο κόμβο θα εκτελεστεί η κάθε εφαρμογή. Τα κύρια δομικά στοιχεία του επιπέδου ελέγχου είναι ο εξυπηρετητής διεπαφής προγραμματισμού εφαρμογών (API server), το etcd, ο kube-scheduler, ο kube-controller-manager και ο cloud-controller-manager. Η λειτουργία κάθε στοιχείου θα εξηγηθεί στην συνέχεια.

2.3.1.2. Κόμβοι Εκτέλεσης

Οι κόμβοι εκτέλεσης (worker nodes) είναι τα μηχανήματα στα οποία εκτελούνται οι εγκιβωτισμένες (containerized) εφαρμογές. Οι λειτουργίες των εργατών για την εκτέλεση, την παρακολούθηση και την παροχή υπηρεσιών στο σύνολο του συστήματος γίνονται από τα βασικά δομικά στοιχεία των κόμβων εκτέλεσης τα οποία είναι η μηχανή εκτέλεσης των περιεκτών (συνήθως Docker), το kubelet και το Kubernetes Service Proxy (kube-proxy).

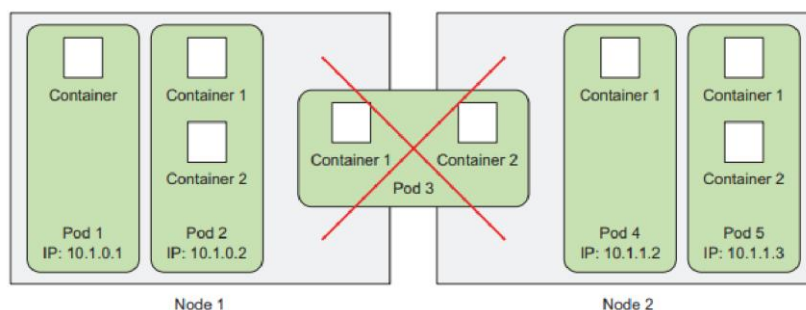


Εικόνα 4. Απεικόνιση Αρχιτεκτονικής Kubernetes και των βασικών στοιχείων των κόμβων. (Kiprotich, 2023)

2.3.1.3. Pods

Το pod αποτελεί τη μικρότερη και απλούστερη μονάδα εκτέλεσης στο περιβάλλον Kubernetes. Αντί να εκτελούνται ελεύθεροι στην συστάδα οι περιέκτες, ενθυλακώνονται εντός των pods. Ένα pod μπορεί να φιλοξενεί είτε μόνο έναν περιέκτη, είτε, σπανιότερα, μία ομάδα περιεκτών των οποίων οι διεργασίες μοιράζονται τους πόρους του. Είναι σημαντικό να διευκρινιστεί ότι ένα pod εκτελείται αυστηρά στα πλαίσια ενός κόμβου εκτέλεσης. Δεν είναι δυνατό οι περιέκτες ενός pod να εκτελούνται σε διαφορετικούς κόμβους (Luksa, 2018). Η αρχιτεκτονική αυτή εξυπηρετεί έναν διπλό στόχο, ο οποίος είναι από την μία η απομόνωση για την διευκόλυνση της διαχείρισης σφαλμάτων και την καταγραφή συμβάντων (logging) και από την άλλη η συνεργασία, παρέχοντας ένα κοινό περιβάλλον για τους αλληλεξαρτώμενους περιέκτες.

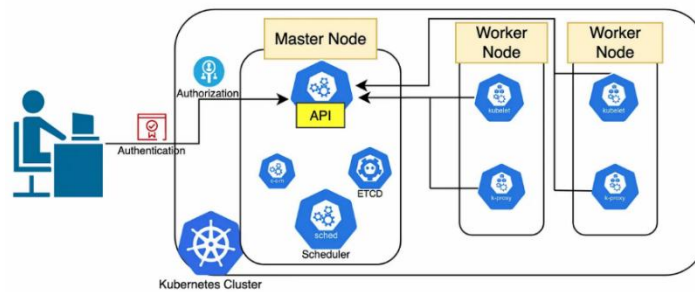
Όλοι οι περιέκτες εντός ενός pod μοιράζονται την ίδια διεύθυνση Διαδικτυακού πρωτοκόλλου (IP) και για αυτό η ανάθεση των θυρών πρέπει να γίνει με προσοχή ώστε να αποφευχθούν συγκρούσεις στα πακέτα που λαμβάνονται και αποστέλλονται από και προς τους περιέκτες.



Εικόνα 5. Η οργάνωση των περιεκτών και των Pods στους κόμβους. (Luksa, 2018)

2.3.1.4. Εξυπηρετητής Διεπαφής Προγραμματισμού Εφαρμογών

Ο εξυπηρετητής ΔΠΕ (API server) είναι το στοιχείο της πλατφόρμας που εκθέτει την ΔΠΕ του Kubernetes, ώστε ο χρήστης να έχει την δυνατότητα διαχείρισης του επιπέδου ελέγχου. Η υλοποίηση του εξυπηρετητή ΔΠΕ είναι το kube-apiserver, το οποίο είναι σχεδιασμένο να κλιμακώνεται οριζόντια ώστε να αυξάνονται ή να μειώνονται σε αριθμό τα στιγμιότυπα που εκτελούνται και να εξισορροπεί την κίνηση (traffic) μεταξύ τους. Πρόκειται για το μοναδικό stateful στοιχείο που επικοινωνεί απευθείας με το etcd. Αυτό τον καθιστά σημείο συμφόρησης του συστήματος (βλ. Εικόνα 6), καθώς σε σενάρια πίεσης πόρων υπάρχει πιθανότητα υπερφόρτωσής του με αιτήματα οδηγώντας τον σε περιορισμό του ρυθμού εξυπηρέτησης (throttling). Όταν ο χρονοπρογραμματιστής (scheduler) και το kubelet δεν μπορούν να επικοινωνήσουν με τον εξυπηρετητή, η συστάδα παγώνει και δεν μπορούν να δημιουργηθούν νέα pods, ούτε να εντοπιστούν καταρρεύσεις τους.



Εικόνα 6. Ο Εξυπηρετητής ΔΠΕ ως σημείο συμφόρησης. (Badawy, 2022)

2.3.1.5. Χρονοπρογραμματιστής

Ο Χρονοπρογραμματιστής (scheduler) είναι το στοιχείο του επιπέδου ελέγχου, το οποίο παρακολουθεί τα pods που δημιουργούνται και τα αναθέτει σε κάποιον κόμβο ώστε να εκτελούνται σε αυτόν. Οι παράγοντες σύμφωνα με τους οποίους γίνεται η ανάθεση είναι τόσο οι απαιτήσεις του ίδιου του pod όσο και ολόκληρης της εφαρμογής. Οι απαιτήσεις αυτές αφορούν το υλικό, αλλά και το λογισμικό το συστήματος. Επίσης, λαμβάνονται υπόψη οι πολιτικές της εφαρμογής, η τοποθεσία των δεδομένων, ο φόρτος εργασιών και οι προθεσμίες για την ολοκλήρωσή τους. Αφού ληφθούν υπόψη όλες οι παράμετροι και εντοπιστούν όλοι οι κατάλληλοι κόμβοι επιλέγεται ένας κόμβος με στόχο την ισότιμη κατανομή σε όλο το σύστημα. Ο Χρονοπρογραμματιστής δεν τροποποιεί τον κόμβο στον οποίο αναθέτει τα νέα pods, αλλά ανανεώνει τον ορισμό του κάθε pod μέσω του εξυπηρετητή ΔΠΕ. Στην συνέχεια ο εξυπηρετητής ΔΠΕ ενημερώνει το kubelet ότι έχουν προγραμματιστεί κάποια pods. Μόλις το kubelet του κόμβου που αναφέρεται στον ανανεωμένο ορισμό εντοπίσει ότι υπάρχουν προγραμματισμένα pods για αυτόν, δημιουργεί και εκτελεί τα containers του pod.

2.3.1.6. Kubelet

Το kubelet στεγάζεται στους κόμβους εκτέλεσης. Η πρώτη εργασία του είναι να εγγράψει τον κόμβο στο σύστημα και στην συνέχεια παρακολουθεί τον εξυπηρετητή ΔΠΕ για νέα pods που ανατίθενται σε αυτόν. Όταν γίνει αυτό δημιουργεί και εκτελεί τον περιέκτη του pod, ενώ παράλληλα παρακολουθεί την κατάσταση και την κατανάλωση των πόρων αναφέροντας τες στο επίπεδο ελέγχου. Ακόμη, εκτελεί ελέγχους ζωτικότητας (liveness probes) και όταν παρουσιάσει αποτυχία, επανεκκινεί τον περιέκτη. Τέλος όταν ένα pod διαγράφεται από την λίστα του εξυπηρετητή ΔΠΕ, το kubelet τερματίζει τον περιέκτη και ενημερώνει τον εξυπηρετητή.

2.3.1.7. Διαμεσολαβητής Υπηρεσιών

Ο Διαμεσολαβητής Υπηρεσιών (Service Proxy) εκτελείται σε κάθε κόμβο εκτέλεσης και ο σκοπός λειτουργίας του είναι να διαβεβαιώνει ότι οι χρήστες της εφαρμογής μπορούν να συνδεθούν στις υπηρεσίες που έχουν οριστεί μέσω της ΔΠΕ του Kubernetes. Στην ουσία, ελέγχει ότι οι συνδέσεις καταλήγουν σε μία διεύθυνση δικτύου και θύρα ενός pod που εκτελεί την επιθυμητή υπηρεσία. Όταν η υπηρεσία εκτελείται

σε περισσότερα από ένα pods κάνει και κατανομή φόρτου μεταξύ αυτών των pods. Η υλοποίηση του, πλέον, γίνεται μέσω κανόνων τύπου iptables για να ανακατευθύνει τα πακέτα στο επιθυμητό pod.

2.3.1.8. etcd

Το etcd είναι αποθηκευτικός χώρος ζευγών κλειδιών και τιμών, υψηλής διαθεσιμότητας όπου φυλάσσεται η συνολική κατάσταση της συστάδας. Υποστηρίζει αποθήκευση ευαίσθητων δεδομένων (secrets) σε κρυπτογραφημένη μορφή, ενώ η φύλαξη κοινών ρυθμίσεων γίνεται σε μορφή ConfigMaps. Το σημαντικότερο χαρακτηριστικό του στοιχείου είναι η ισχυρή συνέπεια, καθώς βασίζεται στον αλγόριθμο Raft Consensus, ο οποίος απαιτεί πλειοψηφία των κόμβων επιπέδου ελέγχου για την εκτέλεση κάθε λειτουργίας.

2.3.1.9. Μηχανή Εκτέλεσης Περιεκτών

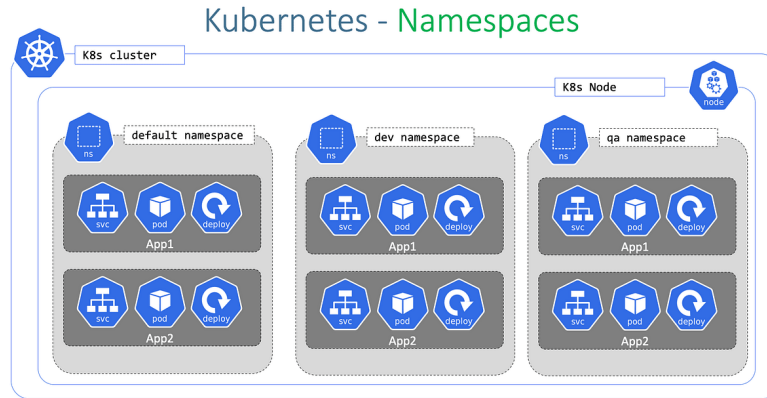
Στο χαμηλότερο επίπεδο λειτουργίας των περιεκτών έχουμε τη μηχανή εκτέλεσης τους (container runtime), η οποία αποτελείται από στοιχεία λογισμικού που διαχειρίζονται την απομόνωση των διεργασιών, αξιοποιώντας βασικούς μηχανισμούς του πυρήνα, όπως τους χώρους ονομάτων και τις ομάδες ελέγχου. Για την αντιμετώπιση ζητημάτων συμβατότητας μεταξύ των διαφορετικών υλοποιήσεων, δημιουργήθηκε η Ανοικτή Πρωτοβουλία Περιεκτών (Open Container Initiative, OCI), μέσω της οποίας ορίστηκαν πρότυπα για την διασφάλιση της φορητότητας και της συμβατότητας των υλοποιήσεων (Stanojevic, Usorac, & Stanojevic, 2021). Τα περιβάλλοντα εκτέλεσης διακρίνονται σε χαμηλού τύπου, τα οποία υλοποιούν άμεσα τις προδιαγραφές της ΑΠΠ και αλληλεπιδρούν με τον πυρήνα, και τα υψηλού τύπου τα οποία παρέχουν πιο ολοκληρωμένη διαχείρισή τους.

2.3.2. Αντικείμενα του Περιβάλλοντος Kubernetes

Τα αντικείμενα στο Kubernetes (Kubernetes Objects) είναι σταθερές οντότητες, οι οποίες χρησιμοποιούνται για την αναπαράσταση της επιθυμητής κατάστασης της συστάδας. Ο διαχειριστής, μέσω αυτών, δηλώνει ποιες εφαρμογές πρέπει να εκτελούνται, σε ποιους πόρους και τις πολιτικές τους (π.χ. ανοχή σφαλμάτων). Το Kubernetes συγκρίνει συνεχώς την κατάσταση της συστάδας με την επιθυμητή και φροντίζει να ταυτίζονται. Ο ορισμός των αντικειμένων γίνεται μέσω αρχείων μορφής JSON στη ΔΠΕ του συστήματος.

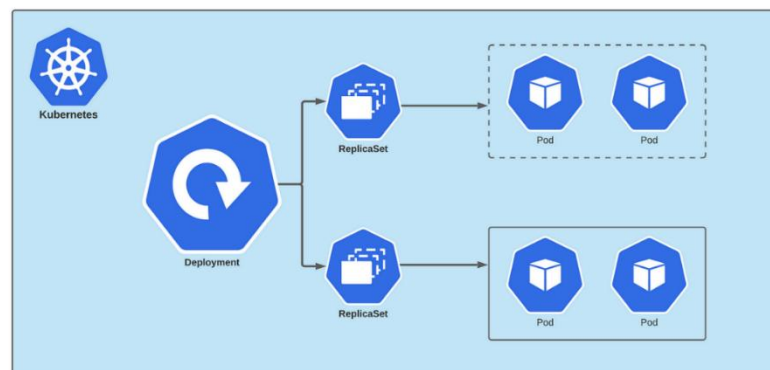
Τα βασικότερα αντικείμενα από τα οποία αποτελείται ένα σύστημα είναι:

- **Οι χώροι ονομάτων (namespaces)**, οι οποίοι αποτελούν έναν μηχανισμό απομόνωσης των πόρων της συστάδας. Λειτουργούν ως ένας τρόπος οργάνωσης σε διαφορετικές ομάδες ή έργα ώστε να μπορούν να συνυπάρχουν χωρίς συγκρούσεις λόγω κοινών ονομασιών ή εκμετάλλευσης κοινών πόρων, αφού δεν μπορούν να επικαλύπτονται μεταξύ τους (The Kubernetes Authors, 2024).



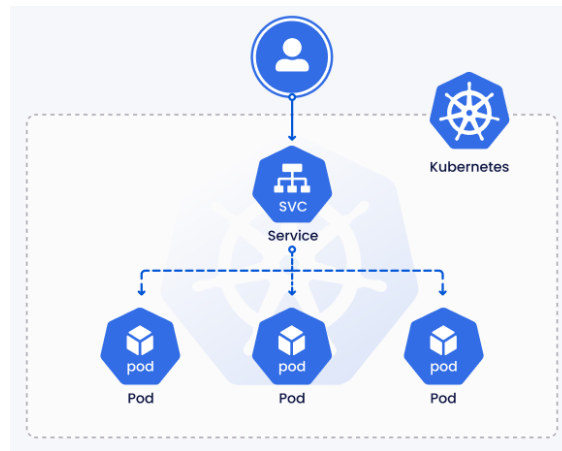
Εικόνα 7. Παράδειγμα χώρων ονομάτων σε έναν κόμβο. (Extio Technology, 2023)

- Τα **σύνολα αντιγράφων** (replicaSet) ορίζουν έναν διαχειριστή όμοιων pods που ονομάζεται ελεγκτής συνόλου αντιγράφων, ο οποίος διασφαλίζει ότι εκτελείται ο καθορισμένος αριθμός αντιγράφων ενός pod. Αυτό προσφέρει υψηλότερη διαθεσιμότητα και ανοχή σε σφάλματα. Αν κάποιο pod καταρρεύσει, ο ελεγκτής το αντικαθιστά άμεσα παρέχοντας το σημαντικότερο χαρακτηριστικό του Kubernetes που είναι η αυτοϊαση (Burns, Beda, Hightower, & Evenson, 2022).
- Τα **αντικείμενα ανάπτυξης** (deployments) είναι υψηλότερου επιπέδου από τα προηγούμενα και διαχειρίζονται τα σύνολα αντιγράφων και τα pods. Ενημερώσεις των εφαρμογών δηλώνονται σε αυτά και παρέχουν αυτοματοποιημένα την δυνατότητα σταδιακής ενημέρωσης το συστήματος (rolling updates), επιστροφής σε προηγούμενες εκδόσεις και δυναμικής κλιμάκωσης των διεργασιών.



Εικόνα 8. Αντικείμενα ανάπτυξης και σύνολα αντιγράφων. (Maharjan, 2020)

- Οι **υπηρεσίες** (services) είναι ένας τρόπος δημιουργίας ενός μοναδικού σταθερού σημείου δικτυακής πρόσβασης σε μία ομάδα pods. Έτσι οι χρήστες δεν χρειάζεται να γνωρίζουν τις δικτυακές διευθύνσεις των pods (εφήμερα) που παρέχουν την υπηρεσία που επιθυμούν (Luksa, 2018). Η λειτουργία τους βασίζεται στο σύστημα ονοματοδοσίας δικτύου (Domain Name System, DNS) της συστάδας.



Εικόνα 9. Σημείο σύνδεσης χρήστη σε υπηρεσία. (Ahamed, 2022)

2.4. Δοκιμές Καταπόνησης/Φορτίου

Οι δοκιμές καταπόνησης (stress testing) είναι μία μορφή ελέγχου του συστήματος μέσω των οποίων αξιολογείται η σταθερότητά του δοκιμάζοντας την λειτουργία του όταν αυτό φτάνει στα όρια χωρητικότητας και των δυνατοτήτων του. Τα σφάλματα που αναζητούνται μέσω των δοκιμών δεν είναι άγνωστα για τους διαχειριστές, αλλά υπάρχει η υποψία ότι μπορεί να προκύψουν. Κατά την διάρκεια της φάσης των δοκιμών, οι υπεύθυνοι αυτών παρατηρούν την συμπεριφορά του συστήματος δίνοντας του εισόδους που προκαλούν υπερφόρτωση (Yang, Jiang, & Huang, 2010). Λόγω της φύσης των συστημάτων που εκτελούνται σε τεχνολογία νέφους, είναι σημαντικός ο άμεσος εντοπισμός προβλημάτων κατά την διάρκεια της λειτουργίας στο μέγιστο των αναμενόμενων δυνατοτήτων του (Arora & Dixit, 2018). Εκτελώντας ένα σύστημα στα όρια των πόρων του δίνει στους διαχειριστές του την ευκαιρία να επιβεβαιώσουν λόγους μείωσης των επιδόσεων, περιορισμούς που μπορεί να προκύψουν λόγω πόρων και σημεία πιθανής εμφάνισης σφαλμάτων. Στην πράξη υπάρχουν αρκετά είδη πειραμάτων για την εκτέλεση δοκιμών που φέρνουν το σύστημα στα όρια του, τα οποία είναι όμως εκτός των ορίων που πραγματεύεται η παρούσα εργασία.

2.4.1. Εργαλεία Δοκιμών Καταπόνησης

Όσον αφορά τις δοκιμές καταπόνησης, υπάρχουν αρκετά εργαλεία τα οποία μπορεί να χρησιμοποιηθούν από τον διαχειριστή ενός συστήματος για την αξιολόγηση του. Αυτό που έχει παρατηρηθεί, όμως, είναι ότι δεν υπάρχει κάποιο εργαλείο το οποίο μπορεί να εκτελέσει πειράματα πολλών κατηγοριών. Με άλλα λόγια, υπάρχουν λογισμικά τα οποία εκτελούν σενάρια που αφορούν διαδικτυακές εφαρμογές, αξιολόγηση δικτύου ενός συστήματος, αξιολόγηση της βάσης δεδομένων, αξιολόγηση κατανεμημένων συστημάτων και μικροϋπηρεσιών, αξιολόγηση κατανομής πόρων, αξιολόγηση της τεχνολογίας νέφους. Τα πιο γνωστά εργαλεία που είναι διαθέσιμα στην αγορά (ανοικτού κώδικα και επί πληρωμή):

- **Apache JMeter**³: Από τα πιο δημοφιλή εργαλεία ανοικτού κώδικα, γραμμένο σε Java. Αρχικά αναπτύχθηκε για διαδικτυακές εφαρμογές, αλλά πλέον υποστηρίζει αρκετά πρωτόκολλα (HTTP, HTTPS, REST, κ.α.) και διαθέτει δικό του γραφικό περιβάλλον για τον σχεδιασμό σεναρίων.
- **Gatling**⁴: Πιο σύγχρονο εργαλείο, βασισμένο στην γλώσσα προγραμματισμού Scala. Παρέχει την δυνατότητα προσομοίωσης μεγάλου αριθμού χρηστών με χαμηλή κατανάλωση πόρων, χρησιμοποιώντας ασύγχρονη αρχιτεκτονική. Τα σενάρια γράφονται σε μορφή κώδικα.
- **Stress-ng**⁵: Σε αντίθεση με τα προηγούμενα, το συγκεκριμένο εργαλείο εστιάζει κυρίως σε σενάρια δικτύου. Λειτουργεί μέσω της γραμμής εντολών και στοχεύει στην δημιουργία εικονικού φορτίου εσωτερικά του συστήματος. Διαθέτει πολλούς μηχανισμούς καταπόνησης για την επεξεργαστική ισχύ, την μνήμη, το σύστημα εισόδων/εξόδων και τον πυρήνα του λειτουργικού συστήματος.
- **wrk**: Πρόκειται για εργαλείο αξιολόγησης, για λόγους σύγκρισης, για το πρωτόκολλο HTTP. Εξαιτίας της πολυνηματικής (multi-thread) επεξεργασίας είναι γρήγορο και ελαφρύ και επιτρέπει την δημιουργία πολύπλοκων αιτημάτων.
- **Siege**⁶: Το συγκεκριμένο εργαλείο καταπόνησης και συγκριτικής αξιολόγησης για HTTP/HTTPS, είναι σχεδιασμένο ν⁷α μετράει την απόδοση κώδικα υπό πίεση. Δίνει την δυνατότητα προσομοίωσης πολλών χρηστών που προσπαθούν να αποκτήσουν πρόσβαση σε μία ή αρκετές ιστοσελίδες.

Εδώ πρέπει να αναφερθεί ότι κάποια από τα παραπάνω δεν αποτελούν επίσημο λογισμικό, αλλά είναι εργαλεία αναπτυγμένα από χρήστες και διατίθενται μέσω πλατφορμών όπως το GitHub.

2.5. Μηχανική Χάους

Ο ορισμός της μηχανικής χάους, σύμφωνα με το βιβλίο «Chaos Engineering: System Resiliency in Practice» είναι:

«Η μηχανική χάους είναι η διενέργεια πειραμάτων με σκοπό τον εντοπισμό αδυναμιών του συστήματος.»

Παρά το όνομα της, η μηχανική χάους γίνεται σε αυστηρά πλαίσια και ακολουθεί κάποιες βασικές αρχές για την εκτέλεση των πειραμάτων, ώστε να αποφεύγεται η δημιουργία προβλημάτων που θα έθεταν το σύστημα εκτός λειτουργίας (Rosenthal & Jones, 2020). Είναι πολύ σημαντικό να μην βγει το σύστημα εκτός λειτουργίας διότι τα πειράματα μηχανικής χάους λαμβάνουν χώρα όταν το σύστημα είναι σε κατάσταση λειτουργίας και διατίθεται για τους χρήστες. Συνεπώς τα πειράματα πρέπει να γίνονται σε περιορισμένη εμβέλεια και ελεγχόμενα για να μην δημιουργηθεί κάποιο

³ <https://jmeter.apache.org/>

⁴ <https://gatling.io/>

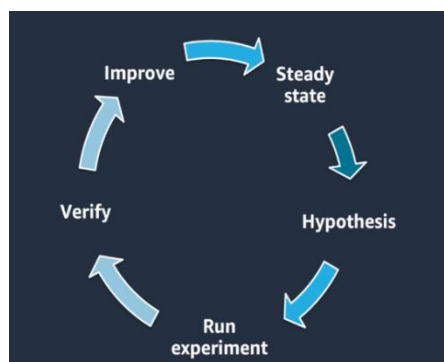
⁵ <https://github.com/ColinIanKing/stress-ng>

⁶ <https://github.com/wg/wrk>

⁷ <https://github.com/JoeDog/siege>

πρόβλημα το οποίο να επηρεάσει σε μεγάλο βαθμό την εμπειρία τους. Η μεθοδολογία εκτέλεσης ενός τέτοιου πειράματος (βλ. Εικόνα 10) είναι η εξής:

1. **Ορισμός σταθερής κατάστασης:** Ορίζεται από τον επιτηρητή του πειράματος η κατάσταση στην οποία θεωρείται ότι το σύστημα λειτουργεί ομαλά. Αυτό πρέπει να δοθεί με κάποιο μετρήσιμο μέγεθος για να μπορεί να συγκριθεί με την κατάσταση του συστήματος όσο εκτελείται το πείραμα.
2. **Διατύπωση υπόθεσης:** Μία υπόθεση του διαχειριστή για το ποια θα ήταν η φυσιολογική απόκριση του συστήματος στο σενάριο του πειράματος, είτε αυτή ταυτίζεται με την σταθερή κατάσταση, είτε είναι μία σειρά από αυτόματες διαδικασίες που θα πρέπει να εκτελεστούν στην περίπτωση του συγκεκριμένου σεναρίου.
3. **Εκτέλεση πειράματος:** Ο διαχειριστής εκτελεί το πείραμα κατά το οποίο γίνεται έγχυση σφαλμάτων στο σύστημα. Τα σφάλματα αυτά μπορεί να αφορούν την απενεργοποίηση κάποιων κόμβων, την εισαγωγή καθυστερήσεων στο δίκτυο, την υπερφόρτωση των πόρων κλπ. Όσο εκτελείται το πείραμα, γίνονται μετρήσεις και αποθηκεύονται για να συγκριθούν με αυτές της σταθερής κατάστασης του συστήματος για να επαληθευθεί ή διαψευστεί η αρχική υπόθεση.
4. **Επαλήθευση αποτελεσμάτων:** Σε περίπτωση διάψευσης της υπόθεσης, είτε γίνεται αναθεώρηση της υπόθεσης και εκτελείται εκ νέου το πείραμα είτε δίνονται τα αποτελέσματα στην ομάδα ανάπτυξης του συστήματος ώστε να γίνουν οι απαραίτητες διορθώσεις και να επανεκτελεστεί το πείραμα, αφού γίνει και πάλι μία αναθεώρηση της υπόθεσης.



Εικόνα 10. Η μεθοδολογία των πειραμάτων μηχανικής χάους. (Johari, 2023)

Πολλές φορές η επιτυχία μίας τέτοιας δοκιμής δεν σημαίνει το τέλος του πειράματος, καθώς όπως αναφέρθηκε παραπάνω, όλα τα πειράματα γίνονται σε περιορισμένη εμβέλεια. Εμβέλεια ενός πειράματος θα ήταν ένα rod μία υπηρεσίας, μια ολόκληρη υπηρεσία ή και ένας χώρος ονομάτων. Συνεπώς όταν μία δοκιμή είναι επιτυχημένη, το επόμενο βήμα είναι να γίνει εκ νέου, μεγαλώνοντας την εμβέλεια του πειράματος, για να πάρουμε ακόμα πιο αντιπροσωπευτικά

αποτελέσματα. Συνήθως αυτά τα πειράματα προγραμματίζονται να γίνουν σε μία μέρα η οποία ονομάζεται «Ημέρα Παιχνιδιού» (Game Day) (Rosenthal & Jones, 2020).

2.6. Διαφορές Δοκιμών Καταπόνησης και Μηχανικής Χάους

Αν και τα πειράματα καταπόνησης του συστήματος και τα πειράματα μηχανικής χάους έχουν αρκετές ομοιότητες και πολλές φορές φαίνεται να ταυτίζονται σαν διεργασίες, οι διαφορές τους είναι πολύ μεγάλες όχι μόνο στον στόχο τους αλλά και στην φύση τους. Πρακτικά οι δύο προσεγγίσεις λειτουργούν συμπληρωματικά στην προσπάθεια βελτίωσης του συστήματος από τους διαχειριστές.

2.6.1. Γενικές Διαφορές και Στόχοι

Η πιο βασική διαφορά μεταξύ των δύο διαδικασιών είναι ο σκοπός της κάθε μίας. Οι δοκιμές καταπόνησης στοχεύουν στην βελτιστοποίηση των επιδόσεων και την εύρεση των ορίων του συστήματος από άποψη χωρητικότητας. Με αυτόν τον τρόπο οι διαχειριστές μπορούν να εντοπίσουν τον λόγο μείωσης όταν το σύστημα λειτουργεί στο μέγιστο των δυνατοτήτων του. Αντίθετα, η μηχανική χάους έχει ως στόχο τον εντοπισμό άγνωστων αδυναμιών του συστήματος και την επιβεβαίωση της ανθεκτικότητάς του. Η παρακολούθηση του συστήματος κατά την διάρκεια των πειραμάτων δεν εστιάζει μόνο στην ικανότητα του συστήματος να εξυπηρετεί αιτήματα, αλλά και στο αν το σύστημα, μετά την πρόκληση μίας βλάβης, μπορεί να ανακάμψει. Ακόμα, οι δοκιμές καταπόνησης διενεργούνται σε περιβάλλοντα δοκιμής, για να μην υπάρχουν δυσάρεστες εμπειρίες για τους χρήστες, όπως μείωσης της ποιότητας υπηρεσιών (Quality of Service, QoS) και της ποιότητας εμπειρίας (Quality of Experience, QoE). Από την άλλη, τα πειράματα μηχανικής χάους εκτελούνται, ελεγχόμενα, σε περιβάλλον παραγωγής, δηλαδή όσο το σύστημα είναι διαθέσιμο σε χρήστες. Ο στόχος είναι η εύρεση σφαλμάτων σε πραγματικές συνθήκες λειτουργίας.

2.6.2. Πειράματα Μηχανικής Χάους Πίεσης των Πόρων

Τα πειράματα μηχανικής χάους, σε αντίθεση με τις δοκιμές καταπόνησης, δεν αφορούν μόνο τους πόρους αλλά μπορεί να εισάγουν προβλήματα ή να προκαλέσουν προβλήματα. Εκεί που συγκλίνουν σαν μεθοδολογίες είναι στα πειράματα πίεσης πόρων. Σε αυτά τα πειράματα προκαλείται εξάντληση πόρων ή υπολογιστικής ισχύος με στόχο την παρακολούθηση της συμπεριφοράς αυτοϊασης. Στα πλαίσια του περιβάλλοντος Kubernetes οι διαφορές μπορούν να διαχωριστούν στους εξής άξονες:

- 1. Πηγή και φύση της πίεσης:** Στις δοκιμές καταπόνησης, η πίεση που εφαρμόζεται στους πόρους του συστήματος προέρχεται συνήθως εξωτερικά από αυτό. Οι χρήστες που προσομοιώνονται να στέλνουν αιτήματα μέσω τρίτων εργαλείων βρίσκονται εξωτερικά της συστάδας. Κατά την διάρκεια των πειραμάτων μηχανικής χάους η πίεση δημιουργείται εσωτερικά στο σύστημα. Τα εργαλεία αξιοποιούν μηχανισμούς του πυρήνα, όπως οι ομάδες ελέγχου, σε κάθε περιέκτη ή pod για να προκληθεί τεχνητή κατανάλωση πόρων.

2. **Σημεία στόχευσης:** Στις δοκιμές καταπόνησης στοχεύονται σημεία διασύνδεσης (endpoints), ΔΠΕ και ροές χρηστών. Στα πειράματα πίεσης πόρων στοχεύονται στοιχεία υποδομής του συστήματος, όπως είναι τα pods, οι περιέκτες, οι κόμβοι και οι χώροι ονομάτων.
3. **Κριτήρια επιτυχίας:** Σαν επιτυχία στις δοκιμές καταπόνησης θεωρούμε την τήρηση των Στόχων του Επιπέδου Υπηρεσιών (Service Level Objectives, SLOs). Σε αυτά περιλαμβάνονται ο χρόνος απόκρισης και ο ρυθμός σφαλμάτων όταν το σύστημα βρίσκεται σε χρήση. Στην μηχανική χάους θεωρούμε επιτυχία όταν το σύστημα παραμένει λειτουργικό κατά την διάρκεια του πειράματος ή όταν έχει την ικανότητα να ανακάμψει αυτόματα χωρίς την παρέμβαση κάποιου διαχειριστή.
4. **Προβλεψιμότητα:** Οι δοκιμές καταπόνησης αποτελούν μία ντετερμινιστική διαδικασία, δηλαδή όταν εκτελώ μία δοκιμή με συγκεκριμένες παραμέτρους θα έχω ένα σταθερό αποτέλεσμα κάθε φορά. Τα πειράματα πίεσης πόρων δεν είναι τόσο προβλέψιμα, καθώς υπάρχουν παράγοντες οι οποίοι επηρεάζουν τα αποτελέσματα του και δεν είναι σταθεροί, οπότε οδηγούν και σε διαφορετικές αποκρίσεις από το σύστημα. Μερικοί από αυτούς τους παράγοντες είναι ο χρονοπρογραμματιστής, ο συλλέκτης σκουπιδιών και οι θορυβώδεις γειτονικοί κόμβοι.

2.7. Εργαλεία Μηχανικής Χάους

Τα εργαλεία μηχανικής χάους είναι λογισμικά που έχουν αναπτυχθεί είτε από ανεξάρτητους χρήστες και διατίθενται δωρεάν είτε από εταιρείες και παρέχονται επί πληρωμή. Τα εργαλεία αυτά επιτρέπουν την προσομοίωση πραγματικών σφαλμάτων στο σύστημα, όπως η απενεργοποίηση εξυπηρετητών, εισαγωγή καθυστερήσεων στο δίκτυο, απώλεια πακέτων, διακοπή πρόσβασης σε βάσεις δεδομένων, κλπ. Όλα αυτά μπορούν να οδηγήσουν στον εντοπισμό αδυναμιών του συστήματος όσο αυτό είναι διαθέσιμο σε χρήση. Υπάρχουν διαθέσιμα πολλά εργαλεία για την δοκιμή της ανθεκτικότητας συστημάτων. Ο τρόπος με τον οποίο επιλέχθηκαν τα εργαλεία που θα αναλυθούν στην συνέχεια είναι αν υποστηρίζονται ακόμα με αναβαθμίσεις και διορθώσεις προβλημάτων και πόσες δυνατότητες προσφέρουν στον μηχανικό που θα τα χρησιμοποιεί, δηλαδή πόσες κατηγορίες πειραμάτων υποστηρίζει, πόσα πειράματα ανά κατηγορία και αν δίνει την επιλογή δημιουργίας προσαρμοσμένων πειραμάτων.

2.7.1. Chaos Monkey⁸

Πρόκειται για ένα από τα πρώτα εργαλεία, και μάλιστα ανοιχτού κώδικα, που αναπτύχθηκε για την εκτέλεση πειραμάτων μηχανικής χάους. Δεν είχε πολλά υλοποιημένα σενάρια, ούτε υποστήριζε την δημιουργία προσαρμοσμένων πειραμάτων από τους προγραμματιστές. Δημιουργήθηκε από την εταιρεία αναπαραγωγής πολυμεσικού περιεχομένου, Netflix, και η λειτουργία του ήταν να απενεργοποιεί διακομιστές με τυχαίο τρόπο χωρίς να διακόπτεται η λειτουργία της υπηρεσίας για τους χρήστες. Στόχος ήταν η παρακολούθηση της απόκρισης του συστήματος σε περίπτωση διακοπών ρεύματος (Netflix OSS, 2023). Λόγω των περιορισμένων δυνατοτήτων του συγκεκριμένου λογισμικού, δεν θα γίνει περαιτέρω ανάλυση. Ο κύριος λόγος αναφοράς ήταν ιστορικός, καθώς μετά την επιτυχία του συγκεκριμένου λογισμικού ακολούθησε η ανάπτυξη περισσότερων εργαλείων μηχανικής χάους τα οποία έχουν την

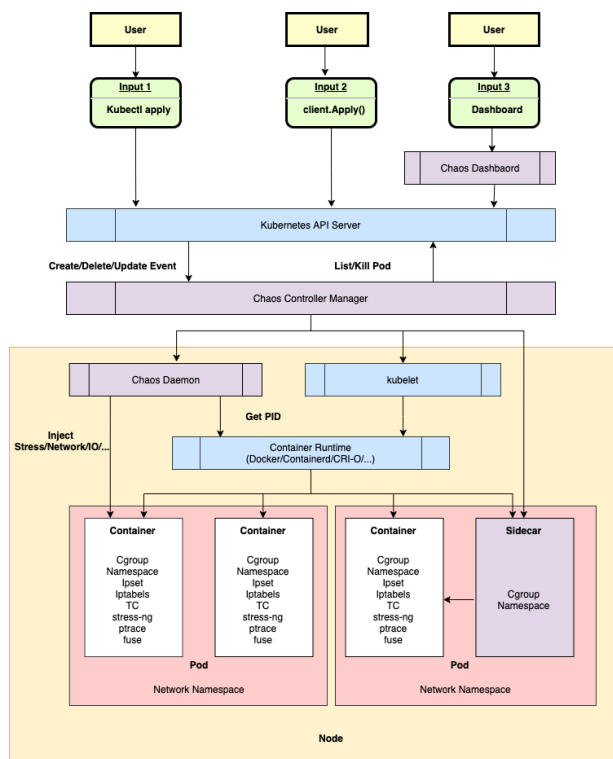
⁸ <https://netflix.github.io/chaosmonkey/>

δυνατότητα εκτέλεσης περισσότερων πειραμάτων δοκιμάζοντας την ανθεκτικότητα των συστημάτων σε περισσότερους τομείς πέρα από την παροδική απώλεια διακομιστών.

2.7.2. Chaos Mesh⁹

Τώως το πιο ευρέως χρησιμοποιούμενο εργαλείο μηχανικής χάους που υπάρχει διαθέσιμο την περίοδο συγγραφής της παρούσας εργασίας. Το Chaos Mesh είναι μία πλατφόρμα ανοιχτού κώδικα σε τεχνολογία νέφους για την εκτέλεση πειραμάτων μηχανικής χάους. Οι κύριοι λόγοι που οδηγούν σε πολλούς χρήστες να το επιλέξουν είναι ότι, πλέον, αποτελεί ένα σταθερό εργαλείο που έχει δοκιμαστεί από μεγάλο αριθμό χρηστών και είναι πιστοποιημένο από πολλές εταιρείες. Ένα σημαντικό χαρακτηριστικό του είναι πως προσφέρει γραφικό περιβάλλον, βασισμένο σε Kubernetes, για τον προγραμματισμό και την αυτοματοποίηση των πειραμάτων. Αυτό διευκολύνει στην χρήση του, αφού εγκατασταθεί δεν είναι υποχρεωτικό να προγραμματίζουν και να εκτελούν τα πειράματα μέσω κάποιας γραμμής εντολών. Ακόμη, διαθέτει μεγάλη ποικιλία έτοιμων σεναρίων προσομοίωσης σφαλμάτων, ενώ αν οι απαιτήσεις του επιθυμητού προς εκτέλεση πειράματος δεν ικανοποιούνται από τα έτοιμα σενάρια που υπάρχουν διαθέσιμα, δίνεται δυνατότητα σχεδιασμού πειραμάτων από τον ίδιο τον μηχανικό. Ακόμη το εργαλείο έχει υλοποιηθεί με πολλαπλά επίπεδα ελέγχου ασφαλείας και έλεγχο πρόσβασης βάσει ρόλου ώστε να δίνεται πρόσβαση στις λειτουργίες που μπορεί να χρειάζεται κάθε χρήστης και να αποφεύγεται η πρόσβαση σε λειτουργίες που μπορεί να επηρεάσουν άλλες ομάδες που συμμετέχουν στον σχεδιασμό, την ανάπτυξη και τον έλεγχο της εφαρμογής.

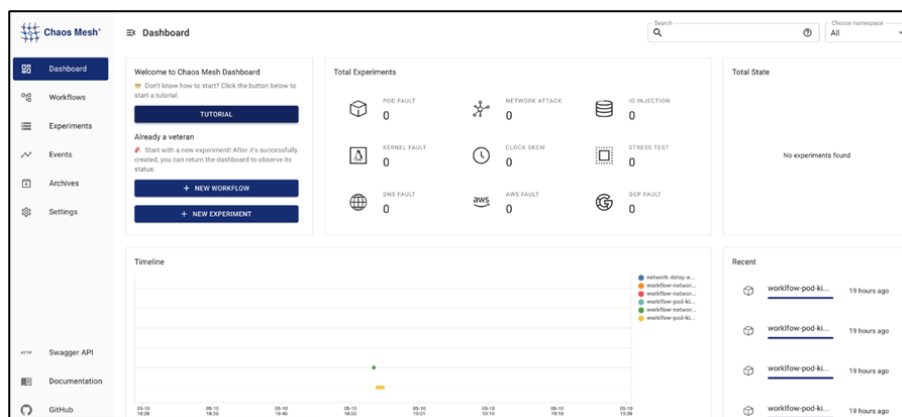
Το Chaos Mesh αποτελείται από τρία κύρια στοιχεία, τον πίνακα ελέγχου χάους (chaos dashboard), τον Διαχειριστή Ελέγχου Χάους (chaos controller) και τον δαίμονα (chaos daemon). Ο πίνακας ελέγχου προσφέρει το γραφικό περιβάλλον για την χρήση του εργαλείου και ταυτόχρονα παρέχει τον μηχανισμό Πρόσβασης Ελέγχου Βάσει Ρόλου (Role Based Access Control, RBAC). Ο διαχειριστής ελέγχου χάους αποτελεί το κεντρικό στοιχείο λογικής του Chaos Mesh και είναι υπεύθυνος για τον χρονοπρογραμματισμό και την διαχείριση των πειραμάτων μέσω ελεγκτών με Προσαρμοσμένους Ορισμούς Πόρων (Custom Resource Definitions, CRDs) όπως οι ροές εργασιών (workflows) και ο χρονοπρογραμματιστής. Τέλος ο δαίμονας είναι το στοιχείο το οποίο εκτελεί τα πειράματα καθώς έχει πρόσβαση στους χώρους ονομάτων των pods (Chaos Mesh Authors, 2024).



Εικόνα 11. Η δομή του Chaos Mesh. (Chaos Mesh Authors, 2024)

⁹ <https://chaos-mesh.org/>

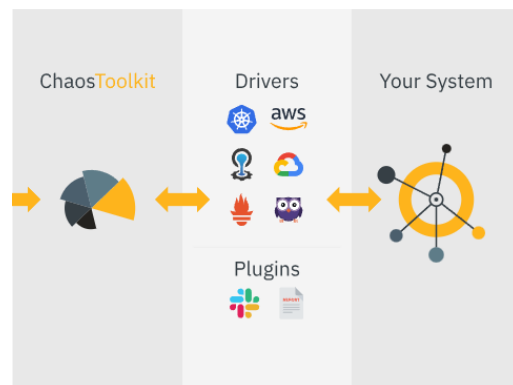
Στο γραφικό περιβάλλον ο διαχειριστής μπορεί να βρει υλοποιημένα σενάρια που επηρεάζουν τους πόρους, την πλατφόρμα του συστήματος ή και την ίδια την εφαρμογή. Συνολικά υπάρχουν 11 διαφορετικές κατηγορίες πειραμάτων και η κάθε μία από αυτές μπορεί να έχει έτοιμα περισσότερα του ενός σενάρια προς προγραμματισμό και εκτέλεση. Μάλιστα μέσω των ροών εργασίας χάους μπορεί να προγραμματιστεί τόσο σειριακή προσομοίωση σφαλμάτων, όσο και παράλληλη.



Εικόνα 12. Η αρχική σελίδα του Πίνακα Ελέγχου του Chaos Mesh.

2.7.3. Chaos Toolkit¹⁰

Το συγκεκριμένο εργαλείο δεν προσφέρει γραφικό περιβάλλον, ούτε έτοιμα σενάρια πειραμάτων μηχανικής χάους στον διαχειριστή, κάνοντας το λιγότερο φιλικό προς τους χρήστες. Ο μοναδικός τρόπος εκτέλεσης πειραμάτων είναι ο σχεδιασμός τους σε μορφή JSON ή YAML. Αυτό μπορεί να το κάνει δυσκολότερο στην χρήση του από άλλα, όμως του δίνει απεριόριστες δυνατότητες στην δημιουργία πειραμάτων, καθώς ο μόνος περιορισμός για τον προγραμματιστή είναι η δική του δημιουργικότητα και ικανότητα μετάφρασης ενός σεναρίου σε μία συμβατή μορφή, δηλαδή είτε JSON είτε YAML. Εφόσον το συγκεκριμένο λογισμικό δίνει την δυνατότητα να δημιουργήσει πειράματα σε μορφή αρχείων, δίνει ταυτόχρονα και την δυνατότητα να οργανώσει και να τα συνδυάσει μεταξύ τους, όπως γίνεται με κάθε κομμάτι κώδικα. Το εργαλείο είναι επεκτάσιμο με οποιοδήποτε άλλο σύστημα χάρη στο Open API και έχει την δυνατότητα αυτοματοποίησης μέσω ενσωμάτωσης σε οποιαδήποτε αλυσίδα ΣΕ/ΣΔ (CI/CD) επιθυμεί ο χρήστης.



Εικόνα 13. Συμβατότητα του Chaos Toolkit με Περιβάλλοντα και Επεκτάσεις. (Chaos Toolkit Contributors, 2024)

Το εργαλείο εκτελείται μόνο μέσω της γραμμής εντολών χάους. Αν η εκτέλεση δεν γίνεται περιορισμένη σε έναν περιέκτη, προτείνεται να είναι ενεργοποιημένο το εικονικό περιβάλλον (virtual environment) για

¹⁰ <https://chaostoolkit.org/>

να μπορούν να εντοπιστούν οι βιβλιοθήκες από τις οποίες εξαρτάται το ίδιο το λογισμικό, το σύστημα, αλλά και οποιοδήποτε συγκεκριμένο πείραμα (Chaos Toolkit Contributors, 2024).

2.7.4. Litmus / LitmusChaos¹¹

Το LitmusChaos είναι μια πλατφόρμα ανοιχτού κώδικα για μηχανική χάους. Είναι συμβατό με Kubernetes, αλλά και με άλλες δημοφιλείς πλατφόρμες όπως τα AWS, GCP και VMWare. Το εργαλείο αυτό είναι χρήσιμο τόσο στη φάση ανάπτυξης της εφαρμογής όσο και περιβάλλοντα κανονικής λειτουργίας της εφαρμογής και μέσω αυτού μπορούν να εντοπιστούν προβλήματα και αδυναμίες του συστήματος. Η προσέγγιση του Litmus είναι εγγενής σε περιβάλλοντα Kubernetes και ο ορισμός των πειραμάτων γίνεται με δηλωτικό τρόπο μέσω Προσαρμοσμένων Ορισμών Πόρων του Kubernetes (LitmusChaos Authors, 2025). Οι κατηγορίες πειραμάτων που υποστηρίζονται από το συγκεκριμένο εργαλείο είναι οι εξής:

- Χάος σε επίπεδο Pods
- Χάος σε επίπεδο Κόμβων
- Χάος σε επίπεδο εφαρμογής
- Ειδικά σφάλματα για δομές νέφους (AWS, GCP, Azure, VMWare)

2.7.5. Gremlin¹²

Το Gremlin δεν διατίθεται δωρεάν, παρέχει, όμως, την δυνατότητα δωρεάν δοκιμής για 30 μέρες. Οι δυνατότητες που προσφέρει το εργαλείο είναι η ασφαλής εισαγωγή σφαλμάτων, μια σταθερή και αξιόπιστη σουίτα πειραμάτων, η συνεργατική διαχείριση κατά τις «Ημέρες Παιχνιδιού» και είναι έτοιμο για χρήσεις από εταιρείες. Τα υποστηριζόμενα πειράματα αφορούν:

- πόρους του συστήματος
- την κατάσταση του συστήματος
- το δίκτυο

Ταυτόχρονα, το εργαλείο είναι συμβατό με πολλά περιβάλλοντα μεταξύ τους διανομές Linux, περιέκτες, περιβάλλοντα Windows (Server και απλά) και πλατφόρμες νέφους.

2.7.6. Steadybit¹³

Το συγκεκριμένο λογισμικό επίσης διατίθεται μόνο επί πληρωμή, αλλά παρέχει μία δοκιμαστική περίοδο 14 ημερών. Ο σχεδιασμός του είναι επικεντρωμένος στην διευκόλυνση των χρηστών ώστε να σχεδιάζουν και να εκτελούν τα πειράματα με τον πιο απλό δυνατό τρόπο. Σε σχέση με τις υπόλοιπες πλατφόρμες μηχανικής χάους αυτό είναι διαθέσιμο ως «Λογισμικό ως Υπηρεσία» (SaaS) και ως λογισμικό που εκτελείται τοπικά για απομονωμένα περιβάλλοντα (περιβάλλοντα εκτός σύνδεσης). Το εργαλείο δίνει στον χρήστη την δυνατότητα οπτικοποίησης των δεδομένων ενός αρχείου καταγραφής γεγονότων μέσω

¹¹ <https://litmuschaos.io/>

¹² <https://www.gremlin.com/>

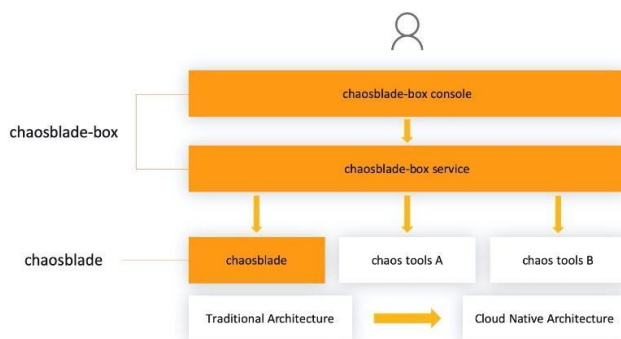
¹³ <https://steadybit.com/>

διάφορων γραφικών στοιχείων (widgets) που δίνονται εντός του σχεδιαστή πειραμάτων. Οι κατηγορίες που υποστηρίζονται αφορούν την εφαρμογή, την πλατφόρμα Kubernetes, το δίκτυο, τους πόρους του συστήματος και την κατάσταση του (Steadybit Authors, 2024).

2.7.7. ChaosBlade¹⁴

Το συγκεκριμένο εργαλείο είναι μία πλατφόρμα εγγενής σε περιβάλλοντα νέφους, η οποία υποστηρίζει πολλαπλούς clusters και γλώσσες. Περιλαμβάνει τις επί μέρους πλατφόρμες chaosblade-box και το chaosblade εργαλείο μηχανικής χάους (The ChaosBlade Authors, 2025). Οι τρεις κατηγορίες πειραμάτων που υποστηρίζονται είναι:

- Φυσικής μηχανής (Πόρων)
- Kubernetes
- Εφαρμογής

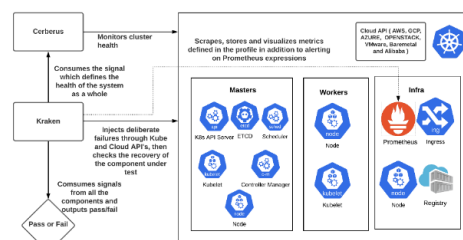


Εικόνα 14. Αρχιτεκτονική λογισμικού ChaosBlade. (The ChaosBlade Authors, 2025)

2.7.8. Krkn¹⁵

Το Krkn (Kraken) είναι ένα εργαλείο που έχει δομηθεί καθαρά στο GitHub και έχει ως στόχο την ενίσχυση της ανθεκτικότητας συστημάτων σε περιβάλλοντα Kubernetes. Το συγκεκριμένο εργαλείο είναι αρκετά πιο απλό από τα προηγούμενα και έχει υλοποιημένα μόνο κάποια βασικά πειράματα μηχανικής χάους που αφορούν:

- Pods, Περιέκτες, Κόμβους
- Πόρους
- Εφαρμογή/Ενέργεια/Ζώνες διακοπών ρεύματος
- Δίκτυο
- Χώρους ονομάτων
- Αποθηκευτικούς χώρους



Εικόνα 15. Δομή της εφαρμογής Krkn με τις υποστηριζόμενες επεκτάσεις. (Krkn Contributors, 2025)

¹⁴ <https://chaosblade.io/en/>

¹⁵ <https://github.com/krkn-chaos/krkn>

2.7.9. Σύγκριση Εργαλείων Μηχανικής Χάους

Επειδή τα εργαλεία που παρουσιάστηκαν παραπάνω είναι αρκετά και με κοινές αλλά και διαφορετικές δυνατότητες σε αυτήν την ενότητα θα γίνει μία άμεση σύγκριση τους στα σημαντικότερα χαρακτηριστικά που μπορεί να αφορούν έναν χρήστη.

Πίνακας 1. Σύγκριση Εργαλείων βάσει των περιβαλλόντων λειτουργίας και επεκτάσεών τους.

Tool\Feature	Ανοιχτό ή Κώδικα	Πλατφόρμες Περιεκτών	Υποστηριζόμενα Λειτουργικά Συστήματα	Πλατφόρμες Διαχείρισης Περιεκτών	Συμβατότητα	Εγγενής σε περιβάλλοντα νέφους
Chaos Monkey	✓		Linux (EC2)	Minimal (AWS Auto Scaling)	Βασική ενσωμάτωση σε AWS	AWS
Chaos Mesh	✓		Linux, macOS, Windows	Kubernetes	Prometheus, Grafana	✓
Chaos Toolkit	✓	Μέσω προσθέτων (plugins)	Linux, Windows, macOS	Any via plugins (K8s, AWS, etc.)	Επεκτάσιμο μέσω πρόσθετων	✓
LitmusChaos	✓		Linux, Windows, macOS	Kubernetes	Prometheus, Grafana, Argo	✓
Steadybit	✗		Linux, macOS, Windows (via WSL)	Kubernetes, ECS	Ενσωματώνεται με εργαλεία παρακολούθησης	✓
ChaosBlade	✓		Linux, macOS	Kubernetes	Prometheus, Grafana	✓
Krkn	✓		Linux	Kubernetes, OpenShift	Red Hat ecosystem, Prometheus	✓
Gremlin	✗		Linux, Windows	Kubernetes, ECS, Nomad	Datadog, Slack, PagerDuty, Prometheus, Grafana και άλλα	✓

Τα χαρακτηριστικά βάσει των οποίων συγκρίνονται τα εργαλεία στον παραπάνω πίνακα είναι:

- Αν είναι ανοιχτού κώδικα, δηλαδή κατά πόσο διατίθεται δωρεάν το λογισμικό, αλλά και ο πηγαίος κώδικας του.
- Είναι η δυνατότητα του λογισμικού να πακετάρεται ο κώδικας του λογισμικού μόνο με τις βιβλιοθήκες του λειτουργικού συστήματος, ώστε να είναι ελαφρύ και φορητό (Pahl, 2015).
- Τα λειτουργικά συστήματα που υποστηρίζονται από κάθε εργαλείο.
- Οι πλατφόρμες διαχείρισης των περιεκτών που υποστηρίζονται από κάθε εργαλείο.
- Η συμβατότητα των εργαλείων με άλλα εργαλεία, με επεκτάσεις και περιβάλλοντα λειτουργίας.
- Αν το εργαλείο είναι εγγενές σε περιβάλλοντα νέφους, δηλαδή αν έχει υλοποιηθεί έτσι ώστε να ακολουθεί τις απαιτήσεις του Ιδρύματος Εγγενούς Υπολογιστικής Νέφους (Cloud Native Computing Foundation), να είναι ελαφρώς διασυνδεδεμένο, να είναι ασφαλές, ανθεκτικό, να έχει απλή δομή διαχείρισης για τους χρήστες και να υπάρχει δυνατότητα παρακολούθησης του.

Πίνακας 2. Σύγκριση εργαλείων βάσει της ευκολίας εκμάθησης.

Tool\Feature	Υποστήριξη από την κοινότητα	Ποιότητα Έγγραφης Τεκμηρίωσης	Οπτική Παρακολούθηση	ΕΠΒΡ
Chaos Monkey	Ελάχιστη (Έχει εγκαταληφθεί)	Βασική Ελάχιστοι οδηγοί για AWS	Μέσω επεκτάσεων	x
Chaos Mesh	Υψηλή	Λεπτομερής Ευανάγνωστη Οργανωμένη	✓	✓
Chaos Toolkit	Υψηλή	Εκτενής Δυσανάγνωστη	Μέσω επεκτάσεων	x
LitmusChaos	Μέση	Λεπτομερής Οργανωμένη	✓	✓
Steadybit	Χαμηλή/Επί πληρωμή	Ελλιπής	✓	x
ChaosBlade	Χαμηλή	Ελλιπής Μετάφραση	Ενσωματωμένο Prometheus	x
Krkn	Χαμηλή	Ελλιπής Πολλά μεμονωμένα αρχεία στο GitHub	Ενσωματωμένο Cerberus	✓

Gremlin	Επί πληρωμή	Λεπτομερής Οργανωμένη	Ενσωματωμένο Datadog	✓
----------------	-------------	----------------------------------	-------------------------	---

- Η υποστήριξη από τις κοινότητες σε αυτά τα εργαλεία είναι αρκετά περιορισμένη σε σχέση με άλλα πεδία.
- Η έγγραφη τεκμηρίωση κρίνεται σε σχέση με το αν είναι αρκετά αναλυτική, καλά οργανωμένη και κατανοητή. Ακόμη κάποια εργαλεία έχουν μοιρασμένη τη τεκμηρίωση σε μία ιστοσελίδα του εργαλείου, αλλά και στο GitHub.
- Οι ενσωματώσεις αναφέρονται στην δυνατότητα ενσωμάτωσης εξωτερικών εργαλείων ή επεκτάσεων στο κάθε εργαλείο.

Πίνακας 3. Σύγκριση εργαλείων βάσει των δυνατοτήτων τους.

Tool\Feature	Τύποι Σφαλμάτων	Εξειδικευμένες Δυνατότητες / Στόχευση	Δυνατότητα Δοκιμής της Ασφάλειας	Αυτοματοποίηση
Chaos Monkey	- Τερματισμός Στιγμιότυπων (AWS EC2) - Διακοπή Αυτόματης Κλιμάκωσης	✗	✗	Περιορισμένος (Μέσω κώδικα)
Chaos Mesh	- Σφάλματα Pod/Δικτύου/Συστήματος Ονοματοδοσίας - Δοκιμές καταπόνησης (Επεξεργαστική Ισχύ/Μνήμη /Σύστημα Αρχείων) - Αποσυγχρονισμός Ρολογιού	✓	✗	Υψηλή
Chaos Toolkit	- Δυνατότητα προσαρμογής μέσω προσθέτων (Δίκτυο, Νέφος, Διεργασίες) - Ευρύ φάσμα μετά από προετοιμασία	✗	✗	Ευέλικτη
LitmusChaos	- Σε Kubernetes: Σφάλματα pods/κόμβων - Χάος σε επίπεδο δικτύου - Σφάλματα παρόχων νέφους (AWS,GCP,Azure)	✓	✗	Υψηλή
Steadybit	- Σφάλματα δομής/δικτύου/επιπέδου εφαρμογής - Διακοπές υπηρεσίας νέφους - Επιθέσεις σε περιβάλλον εφαρμογών Java	✗	✓	Υψηλή

ChaosBlade	-Σφάλματα ΛΣ/Περιεκτών/ Kubernetes/ Νέφους -Καταπόνηση ΒΔ (MySQL, Redis) -Εξάντληση πόρων συστήματος	x	✓	Μόνο μέσω προσθέτων
Krkn	-Διακοπές σε επίπεδο συστάδας -Σφάλματα κόμβων -Διακοπές σε περιβάλλοντα νέφους	✓	x	Μεσαία
Gremlin	-Εξάντληση πόρων, Καθυστέρηση στο δίκτυο, Τερματισμοί, κλπ. (20+ είδη σφαλμάτων) -Χάος σε επίπεδο ασφάλειας (Σύστημα ονοματοδοσίας, Λανθασμένες ρυθμίσεις σε ομάδες ασφαλείας)	✓	✓	Υψηλή

Στον Πίνακα 3 συγκρίνονται και πάλι τα 8 εργαλεία σε 4 βασικούς άξονες:

- Τις βασικές κατηγορίες σφαλμάτων που υποστηρίζονται από τα εργαλεία.
- Την δυνατότητα σε εξειδικευμένες καταπονήσεις που χρειάζονται ειδικά δικαιώματα πρόσβασης, όπως είναι αυτές των βάσεων δεδομένων.
- Την δυνατότητα δοκιμών που αφορούν την ασφάλεια, όπως τον αποσυγχρονισμό ρολογιού ή επιθέσεις σε περιβάλλοντα Java.
- Την δυνατότητα του εργαλείου να αυτοματοποιήσει διαδικασίες και πειράματα.

2.8. Εργαλεία Παρακολούθησης και Οπτικοποίησης

Το σημαντικότερο μέρος της μηχανικής χάους είναι η παρακολούθηση της συστάδας με στόχο την ανάλυση της συμπεριφοράς του συστήματος γενικά, αλλά και συγκεκριμένα κατά την διάρκεια των πειραμάτων μηχανικής χάους. Σε αυτό το κεφάλαιο περιγράφονται τα βασικά εργαλεία συλλογής μετρικών, ιχνών (traces) και αρχείων καταγραφής, καθώς και οπτικοποίησής τους.

2.8.1. Prometheus¹⁶

Το Prometheus είναι το ευρέως χρησιμοποιούμενο εργαλείο συλλογής και οπτικοποίησης μετρικών συνήθως σε μορφή χρονοσειρών ή πινάκων. Είναι ειδικά σχεδιασμένο για χρήση σε περιβάλλοντα νέφους (Prometheus Authors 2014-2025, 2025). Ο τρόπος που λειτουργεί είναι να τραβάει μετρικές μέσω εξαγωγέων ή σημείων διασύνδεσης τις οποίες αποθηκεύει σε βάσεις δεδομένων και τις εκθέτει μέσω PromQL ερωτημάτων και ειδοποιήσεων.

¹⁶ <https://prometheus.io/>

2.8.2. Grafana¹⁷

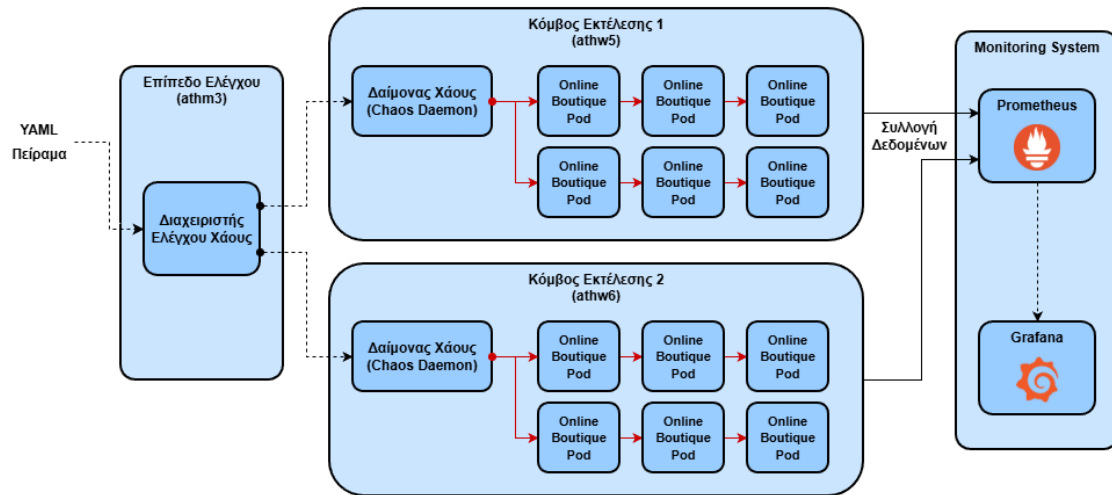
Το Grafana παρέχει ένα επίπεδο οπτικοποίησης πάνω από το Prometheus (ή και άλλες πηγές δεδομένων). Χρησιμοποιείται για την δημιουργία πινάκων ελέγχου και ειδοποιήσεων. Παρέχει δυνατότητα οπτικοποίησης και μετατροπής των δεδομένων σε μορφές που διευκολύνουν την εποπτεία του συστήματος και είναι αντιπροσωπευτικές για αυτό, ειδικά κατά την διάρκεια πειραμάτων μηχανικής χάους (Grafana Labs, 2025).

2.8.3. Πίνακες ελέγχου των εργαλείων

Πολλά από τα εργαλεία μηχανικής χάους παρέχουν δικά τους γραφικά περιβάλλοντα για την παρακολούθηση της κατάστασης των πειραμάτων, της σειράς γεγονότων και δράσεων του εργαλείου κατά το πείραμα και των αντικειμένων-στόχων του πειράματος. Ενσωματώνουν και αρχεία καταγραφών του κάθε πειράματος και αρκετά από αυτά μπορούν να συνδεθούν με το Prometheus και το Grafana για την παροχή μετρικών.

¹⁷ <https://grafana.com/>

3. Πειραματική Προσέγγιση και Αποτελέσματα



Εικόνα 16. Διάγραμμα Πειραματικής Διάταξης

3.1. Πειραματική Διάταξη

Για τις ανάγκες των πειραμάτων και των σεναρίων μηχανικής χάους που θα παρουσιαστούν στη συνέχεια χρησιμοποιήθηκε ένα καταναμημένο σύστημα σε περιβάλλον Kubernetes, το οποίο αποτελείται από 3 κόμβους (nodes), οι οποίοι εκτελούνται σε περιβάλλον Ubuntu 22.04.

3.1.1. Χαρακτηριστικά Κόμβων

Τα χαρακτηριστικά των εικονικών μηχανών δίνονται στον παρακάτω Πίνακα 4:

Πίνακας 4. Χαρακτηριστικά των εικονικών μηχανών.

Λειτουργικό Σύστημα	Ubuntu 22.04.2 LTS (Jammy Jellyfish)
Πυρήνας (Kernel)	5.15.0-71-generic
Επεξεργαστής (CPU)	Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz (4 vCPUs)
Μνήμη (RAM)	4 GB
Αποθηκευτικός Χώρος	40 GB (Διαθέσιμα 24 GB)
Έκδοση Kubernetes	v1.32.9
Μηχανή Εκτέλεσης Περιεκτών	containerd (v1.7.28 Control Plane / v2.2.0 Workers)

3.1.2. Τοπολογία Συστάδας

Η συστάδα αποτελείται από έναν κόμβο ελέγχου και δύο κόμβους εκτέλεσης, όπως φαίνεται παρακάτω:

Επίπεδο ελέγχου (athm3): Υπεύθυνος για την διαχείριση της συστάδας.

Κόμβοι εκτέλεσης (athw5, athw6): Φιλοξενούν τα pods των εφαρμογών και των εργαλείων μηχανικής χάους και παρακολούθησης και οπτικοποίησης των μετρικών του συστήματος.

3.1.3. Χρήση Εφαρμογής

Ως πεδίο εφαρμογής των πειραμάτων μηχανικής χάους επιλέχθηκε μία εφαρμογή ανοιχτού κώδικα με αρχιτεκτονική μικροϋπηρεσιών που προσομοιώνει ένα ηλεκτρονικό κατάστημα αγορών, το Online Boutique [14]. Έχει αναπτυχθεί από την Google για την παρουσίαση τεχνολογιών νέφους (cloud-native technologies), όπως είναι το Kubernetes. Το Online Boutique περιέχει μία επιπλέον μικροϋπηρεσία προσομοίωσης δικτυακής κίνησης χρηστών, οι οποίοι περιηγούνται σε προϊόντα, προσθέτουν κάποια από αυτά στο καλάθι αγορών και ολοκληρώνουν συναλλαγές. Η επιλογή της συγκεκριμένης εφαρμογής έγινε, λόγω της ευρείας χρήσης της ίδιας ή παρόμοιων σε ήδη υπάρχουσες ερευνητικές εργασίες κάνοντας την ένα καλό πρότυπο σύγκρισης (benchmark) για την αξιολόγηση της ανθεκτικότητας και της αξιοπιστίας της. Αποτελείται από 10 ανεξάρτητες μικροϋπηρεσίες αναπτυγμένες με διαφορετικές τεχνολογίες, επιβεβαιώνοντας την αναφορά στην ευελιξία που δίνει η αρχιτεκτονική μικροϋπηρεσιών στους προγραμματιστές. Η μεταξύ τους επικοινωνία γίνεται αποκλειστικά μέσω του πρωτοκόλλου gRPC, το οποίο εξασφαλίζει χαμηλή καθυστέρηση. Οι μικροϋπηρεσίες και οι λειτουργίες τους αναλύονται στον παρακάτω πίνακα:

Πίνακας 5. Οι μικροϋπηρεσίες του Online Boutique με την λειτουργία τους και την γλώσσα προγραμματισμού που χρησιμοποιήθηκε για την ανάπτυξή τους.

Μικροϋπηρεσία	Γλώσσα προγραμματισμού	Λειτουργία
Frontend	Go	Υλοποιεί την ιστοσελίδα και δέχεται τα HTTP αιτήματα των χρηστών.
Cartservice	C#	Διαχειρίζεται το καλάθι αγορών κάθε χρήστη, χρησιμοποιώντας μία βάση δεδομένων για την αποθήκευση δεδομένων.
Productcatalogservice	Go	Παρέχει στους χρήστες λειτουργίες αναζήτησης και ανάκτησης πληροφοριών για τα προϊόντα.
Currencyservice	Node.js	Μετατρέπει τις τιμές των προϊόντων από ένα νόμισμα σε ένα άλλο.
Paymentservice	Node.js	Προσομοιώνει την πληρωμή των αγορών από χρήστες.
Shippingservice	Go	Υπολογίζει το κόστος αποστολής κάθε παραγγελίας.

Emailservice	Python	Προσομοιώνει την αποστολή ηλεκτρονικών μηνυμάτων για την επιβεβαίωση των παραγγελιών.
Checkoutservice	Go	Εκτελεί την διαδικασία ολοκλήρωσης μίας παραγγελίας.
Recommendationservice	Python	Προτείνει σχετικά προϊόντα με αυτά που επιλέγει ο χρήστης.
Adservice	Java	Εμφανίζει διαφημίσεις σχετικές με την σελίδα.

3.1.4. Σύστημα Παρακολούθησης

Για την δημιουργία, την εκτέλεση και την παρακολούθηση των επιπτώσεων των πειραμάτων εγκαταστάθηκαν στο σύστημα τα παρακάτω εργαλεία:

- **Prometheus:** Συλλέγει δεδομένα από διάφορες μετρικές που αφορούν ολόκληρη την συστάδα.
- **Grafana:** Χρησιμοποιείται για την οπτικοποίηση των δεδομένων.
- **Kube-state-metrics:** Παρέχει δεδομένα για την κατάσταση των αντικειμένων του Kubernetes (Αντικείμενα Ανάπτυξης, Pods)
- **Chaos Mesh:** Για την δημιουργία και την εκτέλεση των πειραμάτων μηχανικής χάους.

3.2. Σύστημα Παρακολούθησης

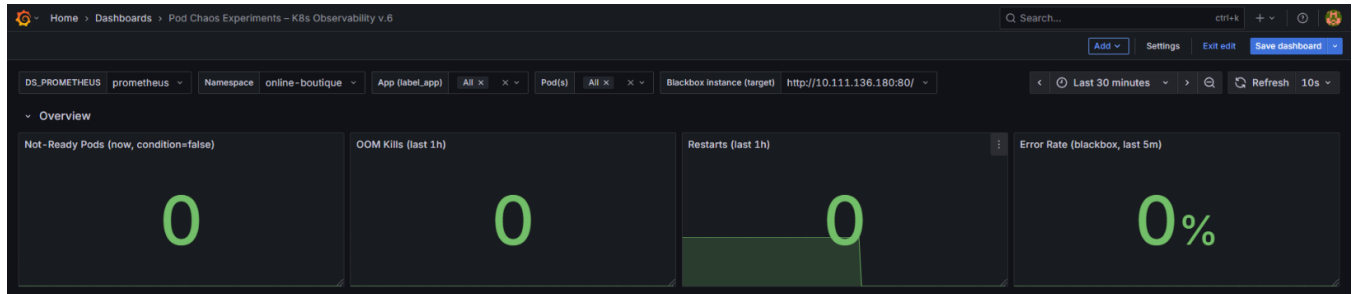
Για την παρατήρηση των επιπτώσεων κάθε πειράματος υλοποιήθηκε ένας πλήρης πίνακας παρακολούθησης με οπτικοποιημένα δεδομένα από το Prometheus στο Grafana. Τα δεδομένα είναι χωρισμένα σε ξεκάθαρα κομμάτια για μετρικές διαφορετικής κατηγορίας.

3.2.1. Γενικές Μετρικές

Σε αυτό το μέρος υπάρχουν κάποιες βασικές μετρικές τις οποίες θα παρακολουθούσε κανείς σε πραγματικό χρόνο για να παρατηρήσει κάποιο πρόβλημα γρήγορα και άμεσα, χωρίς όμως να μπορεί να εντοπίσει την πηγή του προβλήματος. Οι μετρικές αυτές είναι:

- **Not-Ready Pods:** Ο αριθμός των pods που δεν είναι σε κατάσταση ετοιμότητας για να δεχτούν αιτήματα χρηστών.
- **Pod Restarts:** Ο ρυθμός επανεκκινήσεων των containers, το οποίο δείχνει αν υπάρχει κάποια μικροϋπηρεσία που κατέρρευσε. Ένας σταθερός αριθμός επανεκκινήσεων δείχνει αστάθεια κάποιας μικροϋπηρεσίας.
- **OOM Kills:** Τερματισμοί containers εξαιτίας εξάντλησης της διαθέσιμης μνήμης. Είναι ένδειξη κακού περιορισμού μνήμης ή διαφυγής μνήμης.

- **Error Rate:** Δίνει το ποσοστό των αποτυχημένων προσπαθειών να συνδεθεί κάποιος στην ιστοσελίδα (frontend) του συστήματος.



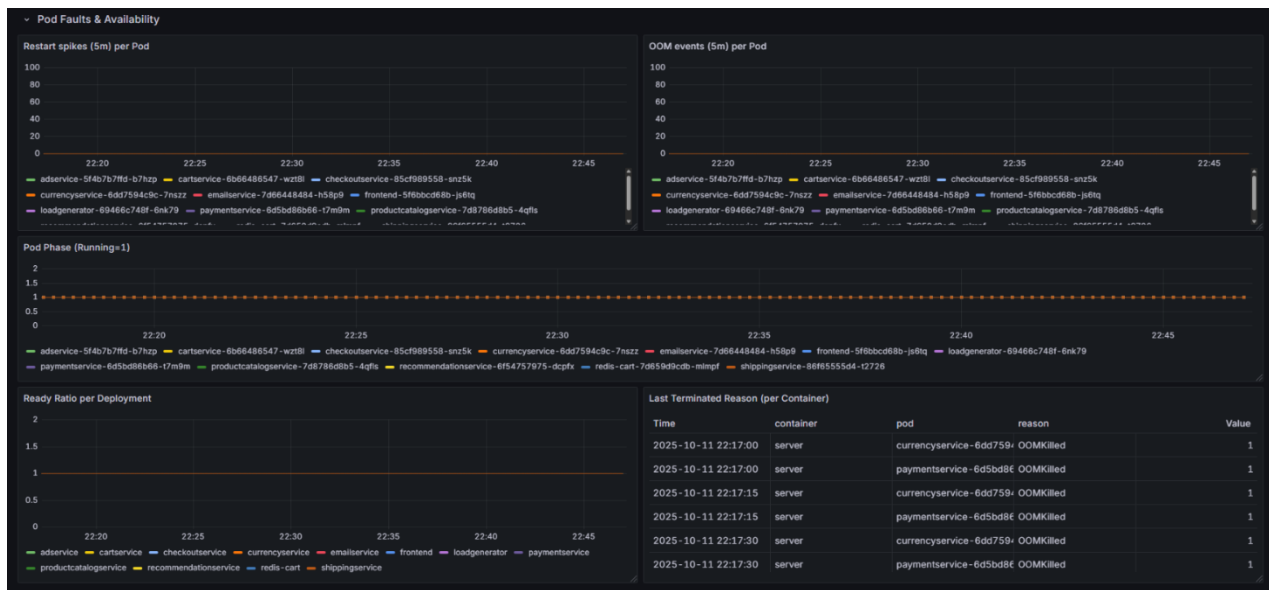
Εικόνα 17. Οι γενικές μετρικές σε σταθερή κατάσταση.

Όπως βλέπουμε στην Εικόνα 17 σε σταθερή κατάσταση λειτουργίας δεν έχουμε pods μη έτοιμα, κανένα τερματισμό pod λόγω εξάντλησης μνήμης, είτε καμία επανεκκίνηση pod είτε η επανεκκίνηση αφορά μία από τις μικροϋπηρεσίες paymentservice και currencyservice και μηδενική ή ελάχιστη αστοχία σε προσπάθειες σύνδεσης στην ιστοσελίδα της εφαρμογής. Οι δύο μικροϋπηρεσίες είναι αποδεκτό να επανεκκινούνται. Έχει γίνει αντιληπτό ότι έχουν πρόβλημα διαφυγής μνήμης, πιθανώς, λόγω κάποιου προβλήματος στον κώδικα τους.

3.2.2. Μετρικές σε επίπεδο Pods

Εδώ παρακολουθούμε την συμπεριφορά των pods στα αντικείμενα ανάπτυξης (επανεκκινήσεις, ετοιμότητα, λόγοι σφαλμάτων).

- **Restart Spikes (5m) per Pod:** Με αυτή την χρονοσειρά βλέπουμε τον ρυθμό επανεκκινήσεων σε ένα παράθυρο 5 λεπτών. Χρησιμοποιείται για να εντοπίζονται pods που τερματίζονται.
- **OOM events (5m) per Pod:** Εδώ παρακολουθούνται οι τερματισμοί των pods από το σύστημα λόγω εξάντλησης της διαθέσιμης μνήμης τους.
- **Pod Phase (Running=1):** Σε αυτή την χρονοσειρά παρακολουθούμε τα pods αν είναι σε κατάσταση εκτέλεσης. (Αν η τιμή είναι 1 τότε είναι σε κατάσταση εκτέλεσης, αλλιώς έχουν τιμή 0)
- **Ready Ratio per Deployment:** Δίνει το ποσοστό των διαθέσιμων αντιγράφων προς τα δηλωμένα αντίγραφα για κάθε αντικείμενο ανάπτυξης. Ελέγχεται με αυτόν τον τρόπο η διαθεσιμότητα ανά υπηρεσία. Αν είναι κάτω από 100 %, σημαίνει ότι δεν είναι όλα τα pods σε κατάσταση ετοιμότητας.
- **Last Terminated Reason (per Container):** Σε αυτόν τον πίνακα μπορούμε να δούμε τον τελευταίο λόγο για τον οποίο τερματίστηκε ένας περιέκτης.



Εικόνα 18. Οι μετρικές σε επίπεδο Pods σε σταθερή κατάσταση.

Παρατηρώντας το κομμάτι του πίνακα της Εικόνας 18, γίνεται αντιληπτό ότι πρόκειται για μία αναλυτικότερη παρουσίαση των γενικών μετρικών. Οπότε, ομοίως, σε λειτουργία σταθερής κατάστασης, έχουμε μηδενικές επανεκκινήσεις και τερματισμούς λόγω εξάντλησης μνήμης, όλα τα pods σε κατάσταση εκτέλεσης, 100% αναλογία έτοιμων pods ανά αντικείμενο ανάπτυξης. Τέλος έχουμε ένα πίνακα με τον τελευταίο λόγο που τερματίστηκε κάθε περιέκτης. Αν το σύστημα δεν είχε ποτέ πρόβλημα ο τελευταίος πίνακας θα είναι κενός.

3.2.3. Μετρικές Δικτύου

Εδώ παρακολουθούμε μετρικές που έχουν να κάνουν με την καθυστέρηση, την ταχύτητα και την διαθεσιμότητα του συστήματος από την σκοπιά των χρηστών.

- **Namespace Network Throughput:** Ο συνολικός ρυθμός εισερχόμενων/εξερχόμενων bytes για τον χώρο ονομάτων που μας ενδιαφέρει (σε αυτή την περίπτωση ο online-boutique). Μειώσεις υποδηλώνουν πτώση δραστηριότητας ή προβλήματα δικτύου.
- **Network I/O (per Pod):** Εδώ έχουμε χρονοσειρές των εισερχόμενων και εξερχόμενων bytes ανά pod. Χρησιμοποιείται για να εντοπιστούν pods με υπερβολική ή μηδενική δικτυακή δραστηριότητα.
- **Availability (1m avg):** Η μέση διαθεσιμότητα ενός λεπτού, την τελευταία ώρα. Η απεικόνιση είναι σε ποσοστιαία μορφή, με τιμές κοντά στο 1 να είναι οι επιθυμητές. Αντιπροσωπεύει τη διαθεσιμότητα της εφαρμογής όπως τη βλέπει ένας εξωτερικός χρήστης.
- **Latency p95/p99 (5m window):** Η συγκεκριμένη μετρική επιστρέφει τον χρόνο κατά τον οποίο το 95% ή 99% των αιτημάτων έχουν ολοκληρωθεί. Είναι μία χαρακτηριστική μετρική που βοηθάει να καταλάβουμε την ταχύτητα εξυπηρέτησης των χρηστών. Όσο αυξάνεται, τόσο καθυστερεί η εφαρμογή.

- **Blackbox HTTP status (current):** Σε αυτόν τον πίνακα βλέπουμε τον κωδικό κατάστασης του τελευταίου probe του blackbox στην ιστοσελίδα της εφαρμογής. Σε συνδυασμό με τη διαθεσιμότητα έχουμε μία πλήρη εικόνα.



Εικόνα 19. Δικτυακές μετρικές όταν το σύστημα βρίσκεται σε σταθερή κατάσταση (α).



Εικόνα 20. Δικτυακές μετρικές όταν το σύστημα βρίσκεται σε σταθερή κατάσταση (β).

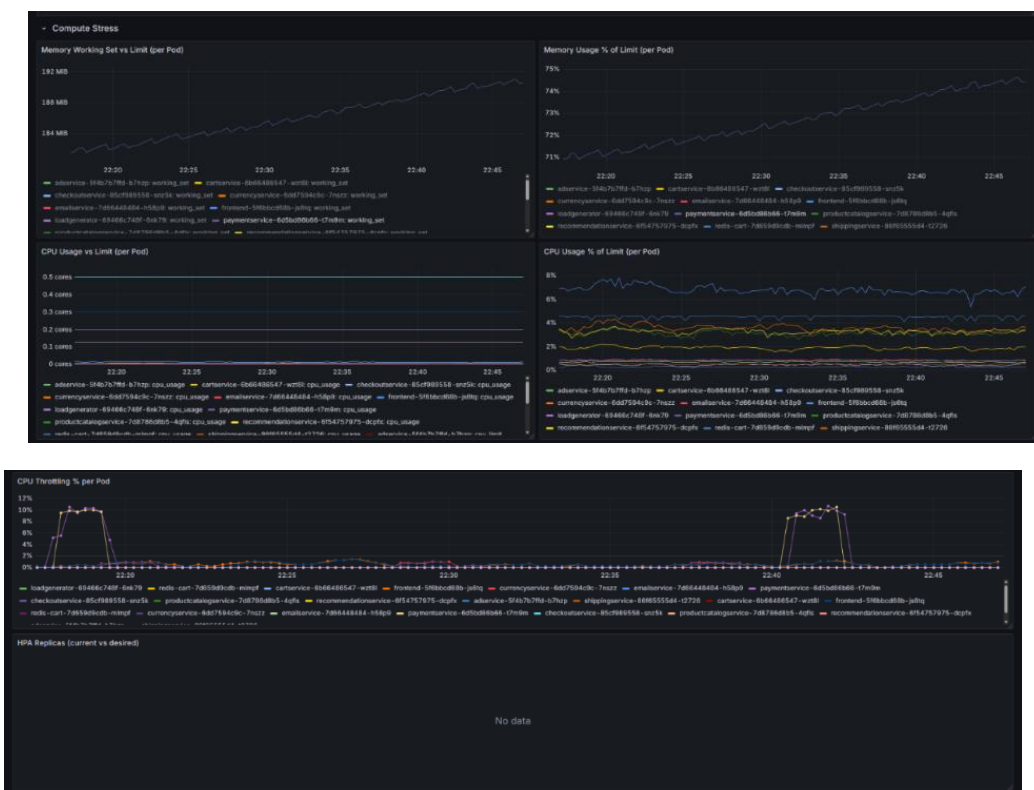
Στην παραπάνω εικόνα παρατηρούμε ότι η διαμεταγωγή (Network Throughput) έχει τιμές ανάμεσα στα 20kB/s με 60kB/s. Η εισερχόμενη και εξερχόμενη ταχύτητα δεδομένων για την ιστοσελίδα της εφαρμογής είναι μεταξύ 30kB/s και 40kB/s. Η καθυστέρηση αιτημάτων είναι περίπου στα 17ms με μερικές κορυφώσεις μέχρι τα 30ms για 95% των πακέτων και τα 40% για το 99% των πακέτων.

3.2.4. Μετρικές κατανάλωσης πόρων σε επίπεδο αντικειμένων ανάπτυξης

Σε αυτό το μέρος παρακολουθείται η κατανάλωση των πόρων του συστήματος (επεξεργαστική ισχύ, μνήμη, δίκτυο) σε επίπεδο αντικειμένων ανάπτυξης.

- **Memory Working Set vs Limit (per Deployment):** Σε αυτόν τον πίνακα γίνεται σύγκριση σε πραγματικό χρόνο της χρήσης της μνήμης σε σχέση με το δηλωμένο όριο ανά αντικείμενο ανάπτυξης.

- **Memory Usage % of Limit (per Deployment):** Ίδια πληροφορία σε ποσοστό %. Δείχνει πιο καθαρά πόσο κοντά είμαστε στο όριο μνήμης.
- **CPU Usage vs Limit (per Deployment):** Ομοίως με την μνήμη, εδώ παρακολουθείται ο ρυθμός χρήσης επεξεργαστικής ισχύος (σε πυρήνες) σε σχέση με το ορισμένο όριο ανά αντικείμενο ανάπτυξης.
- **CPU Usage % of Limit (per Deployment):** Εμφανίζει τη χρήση επεξεργαστικής ισχύος σε ποσοστό του ορίου. Αν πλησιάζει 100 %, το pod ίσως χρειάζεται περισσότερους πόρους.
- **CPU Throttling % (per Deployment):** Εδώ παρακολουθούμε το ποσοστό περιόδων όπου η επεξεργαστική ισχύς κόβεται από τον ΕΔΧ (Εντελώς Δίκαιο Χρονοπρογραμματιστή, CFS). Δηλαδή, όταν μία διεργασία απαιτήσει περισσότερους πυρήνες από αυτούς που της έχουν οριστεί, κόβεται. Υψηλό ποσοστό σημαίνει ότι η επεξεργαστική ισχύς είναι υπερφορτωμένη ή το όριο πολύ χαμηλό.
- **HPA Replicas (current vs desired):** Σε αυτή την χρονοσειρά συγκρίνουμε τα τρέχοντα αντίγραφα με τα επιθυμητά ανά αντικείμενο οριζόντιας κλιμάκωσης των pods. Αν αυξάνεται η επιθυμητή τιμή, σημαίνει ότι έγινε αυτόματη κλιμάκωση.



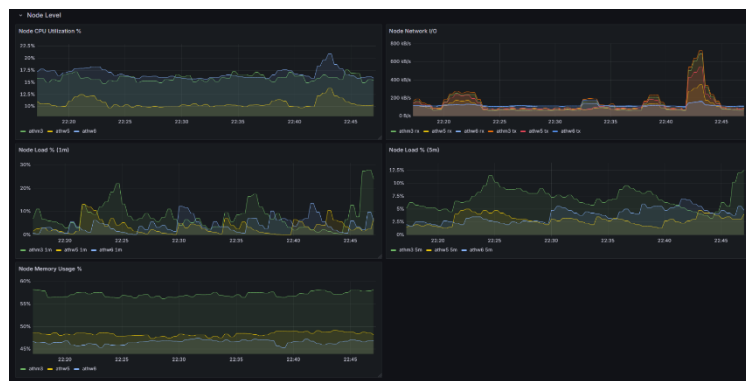
Εικόνα 21. Μετρικές κατανάλωσης πόρων σε επίπεδο αντικειμένων ανάπτυξης και pods.

Σε αυτό το μέρος του πίνακα εποπτείας ελέγχουμε αν κάποια μικροϋπηρεσία φτάνει την κατανάλωση των αντίστοιχων πόρων στο 100% της διαθεσιμότητας τους συνεχόμενα ή με περιοδική συχνότητα. Ακόμα παρακολουθείται ο πίνακας που εντοπίζει τον περιορισμό της επεξεργαστικής ισχύος, ώστε οι τιμές τους να είναι χαμηλές και οι κορυφώσεις σπάνιες και μόνο όταν είναι απαραίτητο. Επίσης στο τέλος της εικόνας

παρατηρούμε τα αντίγραφα στο αντικείμενο οριζόντιας κλιμάκωσης (δεν έχει δεδομένα εδώ, καθώς είναι απενεργοποιημένο).

3.2.5. Μετρικές κατανάλωσης πόρων σε επίπεδο εικονικών μηχανών

- **Node CPU Utilization %:** Η χρήση της επεξεργαστικής ισχύος ανά κόμβο. Δείχνει πόσο απασχολημένος είναι κάθε κόμβος.
- **Node Memory Usage %:** Δείχνει πόση μνήμη χρησιμοποιείται από τον κόμβο συνολικά. Όταν πλησιάζει το 100 %, ο κόμβος κινδυνεύει να αρχίσει να απορρίπτει pods.
- **Node Load Average (1m/5m):** Αντιπροσωπεύει το φορτίο του συστήματος σε διάστημα 1 και 5 λεπτών. Τιμές κοντά ή πάνω από 100 % δείχνουν κορεσμό.
- **Node Network I/O:** Πόσα δεδομένα λαμβάνει (rx) και στέλνει (tx) κάθε κόμβος. Βοηθά στον εντοπισμό υπερφόρτωσης του δικτύου.



Εικόνα 22. Μετρικές κατανάλωσης πόρων σε επίπεδο κόμβων σε σταθερή κατάσταση λειτουργίας.

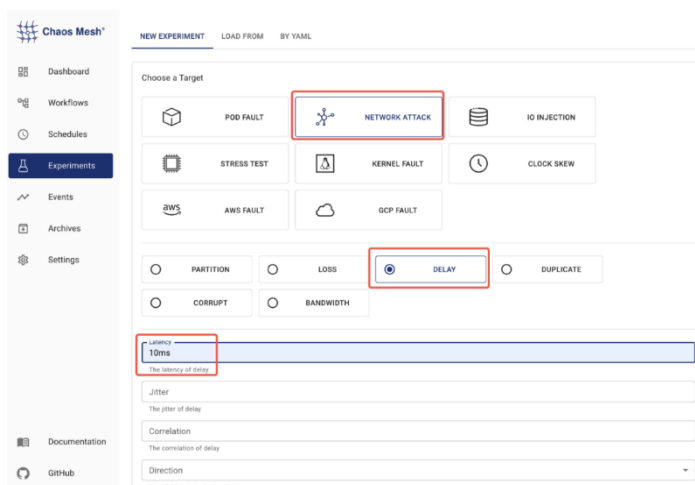
Εδώ παρακολουθούμε την γενική κατανάλωση των πόρων ανά εικονικό μηχάνημα της συστάδας. Καθώς οι μετρικές σε αυτό κομμάτι δίνουν μία γενική εικόνα ολόκληρου του συστήματος, το φυσιολογικό στην σταθερή κατάσταση είναι η απουσία ακραίων τιμών στις χρονοσειρές.

4. Πειράματα Μηχανικής Χάους

Τα πειράματα στα πλαίσια της εργασίας εκτελούνται μέσω του εργαλείου Chaos Mesh. Έχουν επιλεγεί 3 κύριες κατηγορίες πειραμάτων (Χάος σε επίπεδο pod, δικτύου και πόρων) με γνώμονα την συνολική αξιολόγηση της αξιοπιστίας της συστάδας σε διαφορετικά επίπεδα και είδη αστοχιών. Κάθε κατηγορία στοχεύει σε ξεχωριστό τμήμα υποδομής και καλύπτει διαφορετικές διαστάσεις της ανθεκτικότητας ενός συστήματος. Τα πειράματα τα οποία θα εκτελεστούν δοκιμαστικά είναι τα παρακάτω:

- 4.2.1. Αποτυχία των pods της ιστοσελίδας για 10 λεπτά
- 4.3.1. Περιορισμός του εύρους ζώνης στα 25 Kbps για 20 λεπτά
- 4.3.2. Περιορισμός του εύρους ζώνης στα 12 Kbps για 20 λεπτά
- 4.3.3. Περιορισμός του εύρους ζώνης στα 1 Kbps για 20 λεπτά
- 4.3.4. Προσθήκη 80ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά
- 4.3.5. Προσθήκη 150ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά
- 4.3.6. Προσθήκη 300ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά
- 4.4.1. Ελαφριά καταπόνηση πόρων επεξεργαστή και μνήμης
- 4.4.2. Μεσαία καταπόνηση πόρων επεξεργαστή και μνήμης
- 4.4.3. Βαριά καταπόνηση πόρων επεξεργαστή και μνήμης
- 4.5. Προσομοίωση αύξησης αριθμού χρηστών
- 4.6.1. Ολοκληρωμένο ήπιο σενάριο πειράματος μηχανικής χάους
- 4.6.2. Ολοκληρωμένο επιθετικό σενάριο πειράματος μηχανικής χάους

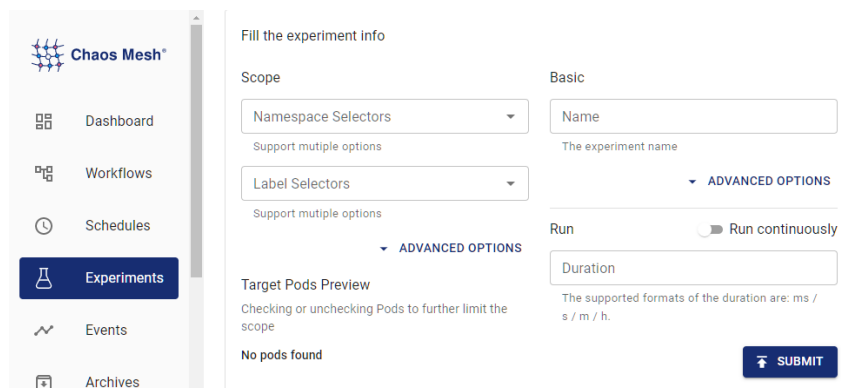
4.1. Το περιβάλλον δημιουργίας πειραμάτων



Εικόνα 23. Η σελίδα δημιουργίας πειράματος στο Chaos Mesh.

Στον πίνακα ελέγχου του εργαλείου, επιλέγοντας την καρτέλα των πειραμάτων και την δημιουργία νέου πειράματος, ο διαχειριστής βλέπει το περιβάλλον που φαίνεται στην Εικόνα 23. Στο πάνω μέρος φαίνονται

οι γενικές κατηγορίες πειραμάτων που υποστηρίζει το εργαλείο. Αφού επιλεγεί η επιθυμητή κατηγορία εμφανίζονται οι υποκατηγορίες και πιο κάτω οι παράμετροι κάθε συγκεκριμένου πειράματος. Μόλις συμπληρωθούν όλα τα απαραίτητα πεδία και ο χρήστης πατήσει το κουμπί της υποβολής, το πείραμα οριστικοποιείται. Για να εκτελεστεί πρέπει να συμπληρωθούν τα απαραίτητα πεδία αναγνώρισης και εμβέλειας του πειράματος που φαίνονται στην παρακάτω εικόνα:



Εικόνα 24. Επιλογή εμβέλειας, ονόματος και διάρκεια πειράματος.

Αφού συμπληρωθούν και τα στοιχεία του πειράματος, το λογισμικό τις μετατρέπει σε μορφή YAML η οποία είναι κατανοητή από το λογισμικό και στην συνέχεια εκτελείται.

4.2. Πειράματα σε επίπεδο Pod

Τα πειράματα στο επίπεδο των pods (Pod Chaos) προσομοιώνουν καταστάσεις όπου pods ή περιέκτες τερματίζονται απροσδόκητα ή καθίστανται μη διαθέσιμα. Αντιπροσωπεύουν τις πιο συχνές και ρεαλιστικές αστοχίες σε ένα περιβάλλον Kubernetes — όπως σφάλματα εφαρμογών, επανεκκινήσεις κόμβων, αποτυχίες ανιχνεύσεων (probes) ή ανθρώπινα λάθη (π.χ. λάθος διαγραφή pod).

Μέσω αυτών των πειραμάτων αξιολογείται η ικανότητα της συστάδας να αυτοθεραπεύεται (self-healing) μέσω μηχανισμών όπως τα Σύνολα αντιγράφων, Αντικείμενα ανάπτυξης και ανιχνεύσεις ετοιμότητας και ζωτικότητας.

Πραγματικά Σενάρια

Αυτά τα πειράματα προσομοιώνουν καταστάσεις όπου pods τερματίζονται αιφνίδια ή γίνονται προσωρινά μη διαθέσιμα. Μερικά από αυτά τα σενάρια είναι τα παρακάτω:

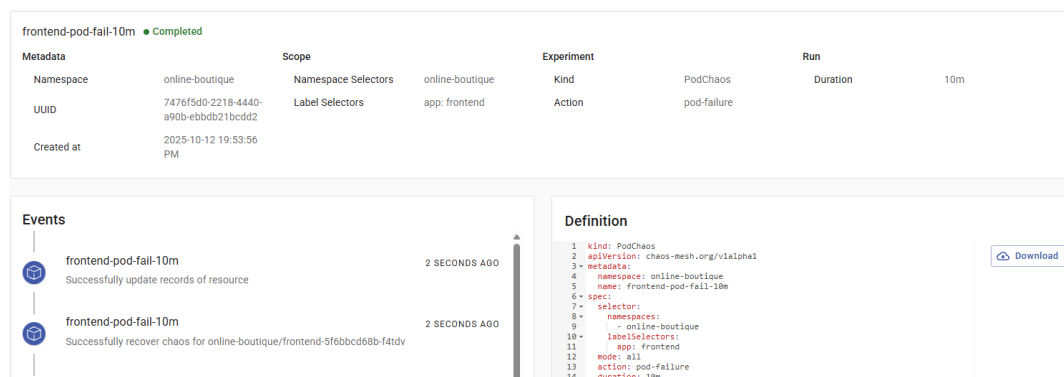
1. Πτώση ή επανεκκίνηση κόμβου (διακοπή ρεύματος, επανεκκίνηση του μηχανήματος).
2. Διακοπή σύνδεσης κόμβου από την συστάδα (απομόνωση από το δίκτυο, απώλεια αγωγού).

3. Απομάκρυνση κόμβου για συντήρηση.
4. Λανθασμένη ρύθμιση (misconfiguration) που οδηγεί σε καταστροφικό βρόχο.
5. Απομάκρυνση pods από έναν κόμβο σε άλλον λόγω αύξησης πίεσης στον πρώτο.
6. Αναβάθμιση εφαρμογής με προβληματική εικόνα (image). Στην ουσία κλείνουν τα λειτουργικά και τα νέα δεν μπορούν να φτάσουν σε κατάσταση ετοιμότητας.
7. Ανθρώπινο λάθος.

4.2.1. Αποτυχία των pods της ιστοσελίδας για 10 λεπτά

Περιγραφή πειράματος

Το συγκεκριμένο πείραμα έχει σκοπό να προσομοιώσει αστοχίες ή τερματισμό του pod της ιστοσελίδας για 10 λεπτά. Με αυτό τον τρόπο αξιολογεί ο διαχειριστής του συστήματος την ικανότητα αυτοϊασης του. Αντιπροσωπεύει προβλήματα όπως σφάλματα της εφαρμογής, επανεκκίνηση κόμβου ή κόμβων, αλλά και ανθρώπινα λάθη.



Εικόνα 25. Χαρακτηριστικά και ορισμός του πειράματος αποτυχίας του pod της ιστοσελίδας.

Οι μετρικές που παρακολουθούνται στα πλαίσια αυτού του πειράματος δίνονται παρακάτω με τις ανάλογες παρατηρήσεις:



Εικόνα 26. Οι επανεκκινήσεις του pod της ιστοσελίδας και η αναλογία έτοιμων pod ανά αντικείμενο ανάπτυξης.



Πίνακας 6. Επιπτώσεις ανά μετρική για σφάλμα του pod της ιστοσελίδας.

Μετρική	Κατάσταση Λειτουργίας	Peak κατά την διάρκεια του πειράματος
Διαθεσιμότητα	100%	0% (-100%)
Ρυθμός αποτυχίας σύνδεσης	0%	100% (+100%)
Καθυστέρηση (p95/p99)	16.05/16.12	~0/~0 (-100%/-100%)
Αναλογία Έτοιμων Pods (Ιστοσελίδα)	100%	0% (-100%)
Επανεκκινήσεις	0	8.50 (+8.50)
Διαμεταγωγή (kB/s)	43.33/43.38	1.57/1.2 (-96.38%)

4.3. Πειράματα σε επίπεδο Δικτύου

Τα πειράματα αυτής της κατηγορίας εστιάζουν στη συνδεσιμότητα μεταξύ μικροϋπηρεσιών (microservices) ή μεταξύ κόμβων της συστάδας. Προσομοιώνουν πραγματικά περιστατικά, όπως καθυστερήσεις δικτύου (latency), απώλειες πακέτων, διακοπές σύνδεσης (network partition), περιορισμούς του εύρους ζώνης ή ακόμη και επιθέσεις καταναμεμημένης άρνησης υπηρεσίας (DDoS). Η κατηγορία αυτή είναι κρίσιμη, διότι σε καταναμεμημένα συστήματα η πλειονότητα των αποτυχιών προκύπτει όχι από πλήρη κατάρρευση εφαρμογής, αλλά από υποβαθμισμένες ή ασταθείς επικοινωνίες.

Πραγματικά Σενάρια

Τα πειράματα τέτοιου τύπου προσομοιώνουν βλάβες και διαταραχές στα δίκτυα επικοινωνίας μεταξύ pods, κόμβων ή εξωτερικών υπηρεσιών. Οι καταστάσεις που μπορεί να οδηγήσουν σε αυτά είναι:

1. Ατύχημα σε αγωγό οπτικής ίνας ή βλάβη δικτύου (βλάβη στην οπτική ίνα, αποτυχία του δρομολογητή).
2. Εμφάνιση συμφόρησης (congestion) ή κακή δρομολόγηση των πακέτων.
3. Πραγματοποίηση επίθεσης καταναμεμημένης άρνησης υπηρεσιών ή αυξημένη κίνηση (εισερχόμενη/εξερχόμενη).
4. Διακοπή σύνδεσης με τρίτες εφαρμογές (ΔΠΕ, Βάσεις Δεδομένων, εξωτερικές μικροϋπηρεσίες).
5. Προβλήματα συστήματος ονοματοδοσίας ή τείχος ασφαλείας.

4.3.1. Περιορισμός του εύρους ζώνης στα 25 Kbps για 20 λεπτά

Περιγραφή πειράματος

Σε αυτό το πείραμα περιορίζεται το εύρος ζώνης του δικτύου σε μικρό βαθμό (25 Kbps) για 20 λεπτά μεταξύ όλων των μικροϋπηρεσιών. Πρόκειται για μία προσομοίωση ήπιας συμφόρησης του συστήματος, αφήνοντας διαθέσιμο το εύρος των 25 Kbps.

bandwidth-25kbps-20m • Completed

Metadata		Scope	Experiment		Run		
Namespace	online-boutique	Namespace Selectors	online-boutique	Kind	NetworkChaos	Duration	20m
UUID	766931eb-711b-467d-912b-39d85db1d423			Action	bandwidth		
Created at	2025-10-16 18:00:36 PM			Direction	to		

Events	Definition
bandwidth-25kbps-20m Successfully update records of resource 2 DAYS AGO	<pre>1 kind: NetworkChaos 2 apiVersion: chaos-mesh.org/v1alpha1 3 metadata: 4 namespace: online-boutique 5 name: bandwidth-25kbps-20m 6 spec: 7 selector: 8 namespaces: 9 - online-boutique 10 mode: all 11 action: bandwidth 12 duration: 20m 13 bandwidth: 14 rate: 25kbps 15 limit: 8192 16 buffer: 2048 17 minburst: 1500 18 direction: to 19 target: 20 selector: 21 namespaces: 22 - online-boutique 23 mode: all</pre> Download
bandwidth-25kbps-20m Successfully recover chaos for online-boutique/paymentservice-6d5bd86b66-t7m9m 2 DAYS AGO	
bandwidth-25kbps-20m Successfully recover chaos for online-boutique/paymentservice-6d5bd86b66-t7m9m 2 DAYS AGO	

Εικόνα 29. Ορισμός του πειράματος περιορισμού του εύρους ζώνης στα 25 Kbps.

Όπως φαίνεται από τον ορισμό σε μορφή YAML, στον πάνω μέρος επιλέγεται η κατηγορία για τα πειράματα δικτύου (NetworkChaos), στη συνέχεια βλέπουμε τον χώρο ονομάτων που θα εκτελεστεί ο οποίος είναι ο online-boutique και στοχεύει όλα τα αντικείμενα στον χώρο. Το όνομα του πειράματος είναι «bandwidth-25kbps-20m», η υποκατηγορία που ανήκει είναι το εύρος ζώνης (bandwidth) περιορισμένο στα 25kbps. Οι ιδιότητες ορίου, προσωρινής μνήμης (buffer) και η ρύθμιση της ροής (minburst) έχουν οριστεί στις τιμές 8192, 2048 και 1500 αντίστοιχα. Το όριο αναφέρεται στην ουρά αναμονής των πακέτων όταν το εύρος ζώνης είναι πλήρες, η προσωρινή μνήμη είναι ο μέγιστος όγκος που μπορεί να σταλεί με την μέγιστη ταχύτητα του δικτύου. Η ρύθμιση ροής περιορίζει τον ρυθμό μετάδοσης για να γίνεται με ακρίβεια, εξομαλύνει την ροή των πακέτων.

Στα πλαίσια αυτού του πειράματος χρειάζεται να παρακολουθήσουμε μόνο 3 μετρικές:

- Διαμεταγωγή του χώρου ονομάτων
- Ρυθμός εισερχόμενων και εξερχόμενων πακέτων ανά μικροϋπηρεσία
- Καθυστέρηση πακέτων (95% / 99%)



Εικόνα 30. Μετρικές για εύρος ζώνης 25 Kbps.

Παρατηρείται μία ήπια σταθερή μείωση της διαμεταγωγής και του ρυθμού εισερχόμενων και εξερχόμενων πακέτων. Ταυτόχρονα παρατηρώντας τις χρονοσειρές των καθυστερήσεων βλέπουμε αύξηση της καθυστέρησης με την βάση να αυξάνεται σε μία τιμή κοντά στα 40 ms με κορυφώσεις μέχρι τα 90-95 ms. Με μεγαλύτερη ακρίβεια οι τιμές δίνονται στον παρακάτω πίνακα:

Πίνακας 7. Ακριβείς μεταβολές των μετρικών λόγω του περιορισμού στα 25 Kbps.

	Κατάσταση Λειτουργίας			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Εισερχόμενη Διαμεταγωγή (kB/s)	45.54	23.6	56.7	37.56 (-17.52%)	16.9	50
Εξερχόμενη Διαμεταγωγή (kB/s)	42.39	7.67	59.4	35.62 (-15.97%)	4.65	51
Εισερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	10.99	9.68	11.9	9.52 (-13.38%)	8.29	11
Εξερχόμενα δεδομένα από την ιστοσελίδα	32.21	29	35.1	26.69	23	32.1

(kB/s)				(-17.14%)		
Καθυστέρηση p95 (ms)	16.63	13.6	20	51.77 (+211.3%)	15	92.7
Καθυστέρηση p99 (ms)	16.72	13.6	20.1	52.53 (+214.17%)	15.1	95.2
Διαθεσιμότητα (%)	100%	100%	100%	100% (0%)	100%	100%

Βλέποντας και τις ακριβείς μετρήσεις του Πίνακα 7 αξίζει να σχολιάσουμε πιο συγκεκριμένα ότι η καθυστέρηση σημείωσε τριπλασιασμό. Αυτό οφείλεται στον χρόνο που παραμένει το κάθε πακέτο στην ουρά, όταν δεν υπάρχει αρκετό εύρος ζώνης για να σταλεί αμέσως.

4.3.2. Περιορισμός του εύρους ζώνης στα 12 Kbps για 20 λεπτά

Περιγραφή πειράματος

Σε αυτό το πείραμα περιορίζεται το εύρος ζώνης του δικτύου σε μέτριο βαθμό (12 Kbps) για 20 λεπτά μεταξύ όλων των μικροϋπηρεσιών. Πρόκειται για μία προσομοίωση μέτριας συμφόρησης του συστήματος, αφήνοντας διαθέσιμο το εύρος των 12 Kbps.

The screenshot displays the Chaos Mesh interface for an experiment named 'bandwidth-12kbps-20m'. The top section shows the experiment's metadata, including its namespace ('online-boutique'), UUID, and creation time. Below this, the 'Events' section lists four successful events: 'Successfully update records of resource', 'Successfully recover chaos for online-boutique/cartservice-6b6486547-wzt8l', 'Successfully recover chaos for online-boutique/cartservice-6b6486547-wzt8l', and 'Successfully update records of resource'. The 'Definition' section on the right shows the YAML configuration for the experiment, which sets the network bandwidth to 12kbps for 20 minutes across all namespaces in the 'online-boutique' namespace.

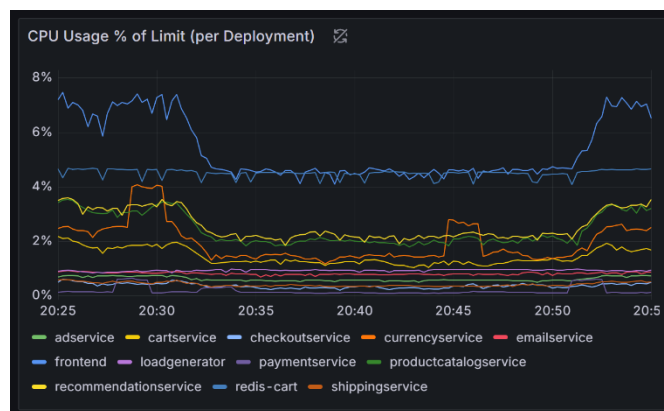
Εικόνα 31. Ορισμός του πειράματος περιορισμού του εύρους ζώνης στα 12 Kbps.

Ο ορισμός είναι παρόμοιος με αυτόν του προηγούμενου πειράματος, αλλά το bandwidth περιορίζεται στα 12 Kbps και όχι στα 25. Ακόμη, οι μετρικές που παρακολουθούμε είναι οι ίδιες με την προσθήκη της χρήσης επεξεργαστικής ισχύος.



Εικόνα 32. Μετρικές Δικτύου για περιορισμό του εύρους ζώνης στα 12 Kbps.

Όπως είναι αναμενόμενο, με τον περιορισμό του εύρους ζώνης, η διαμεταγωγή και ο ρυθμός μεταφοράς δεδομένων στο δίκτυο μειώνονται αρκετά. Η καθυστέρηση αυξάνεται με χρόνο βάσης περίπου στα 75ms με κορυφώσεις που φτάνουν μέχρι και τα 320ms. Αυτό είναι λογικό αφού λόγω περιορισμού των δεδομένων που μεταφέρονται χρειάζεται περισσότερο χρόνο για να ολοκληρωθεί ένα αίτημα.



Εικόνα 33. Ποσοστό χρήσης επεξεργαστικής ισχύος ανά αντικείμενο ανάπτυξης (12 Kbps).

Στην Εικόνα 33 παρατηρείται μείωση στην χρήση της επεξεργαστικής ισχύος από σχεδόν όλες τις υπηρεσίες. Αυτό συμβαίνει πιθανώς λόγω της μείωσης του ρυθμού μεταφοράς δεδομένων στο δίκτυο του συστήματος. Οι μικροϋπηρεσίες δέχονται λιγότερα αιτήματα, οπότε πολλές φορές ολοκληρώνονται πριν

έρθει κάποιο νέο αίτημα και μπαίνουν σε κατάσταση αδράνειας. Είναι σημαντικό να μην ταυτίζεται η χαμηλή χρήση επεξεργαστικής ισχύος με την ομαλή λειτουργία.

Πίνακας 8. Ακριβείς μεταβολές των μετρικών λόγω του περιορισμού στα 12 Kbps.

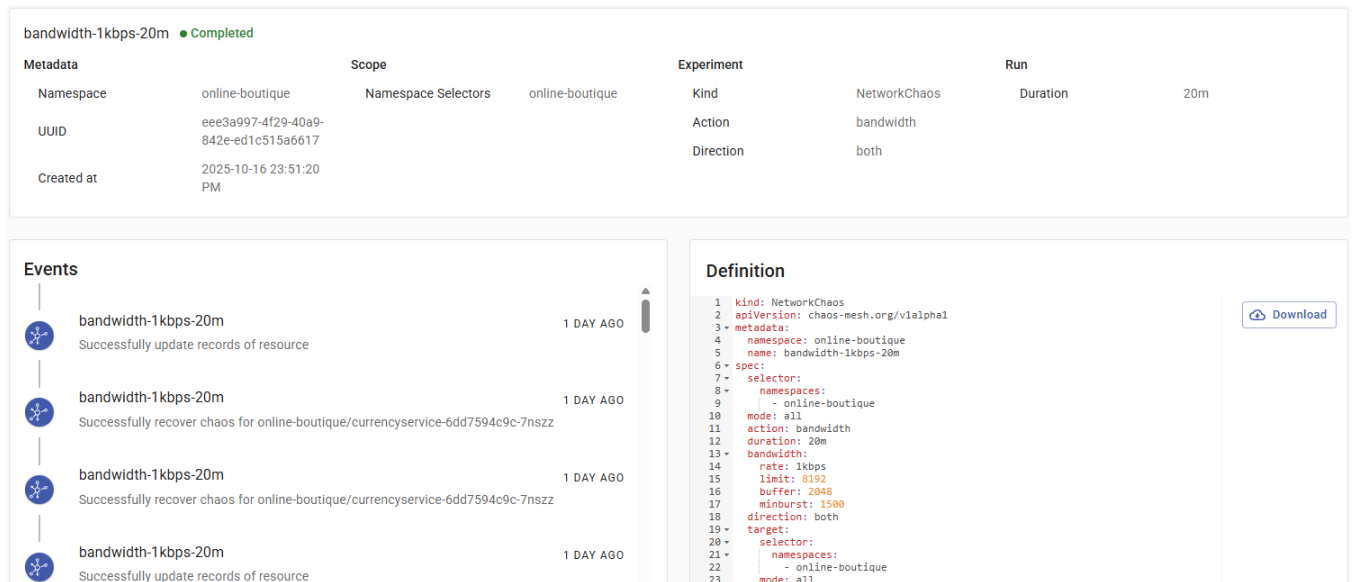
	Κατάσταση Λειτουργίας			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Εισερχόμενη Διαμεταγωγή (kB/s)	47.72	34.8	66.1	23.3 (-51.17%)	8.66	29.6
Εξερχόμενη Διαμεταγωγή (kB/s)	44.59	17.7	60.2	22.96 (-48.51%)	5.11	31.1
Εισερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	11.31	9.98	12.1	6.22 (-45%)	5.28	10.5
Εξερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	32.93	29.4	35.2	17.04 (-48.25%)	14.9	29
Καθυστέρηση p95 (ms)	16.42	14.4	17.9	128.80 (+684.41%)	56.7	325
Καθυστέρηση p99 (ms)	16.49	14.4	18	131.33 (+696.42%)	57.1	331
Διαθεσιμότητα (%)	100%	100%	100%	100% (0%)	100%	100%
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από την ιστοσελίδα (%)	7%	5.85%	7.47%	5% (-28.57%)	4.09%	6.87%

Μία παρατήρηση που αξίζει να αναφερθεί είναι ότι παρά τις εμφανείς επιπτώσεις του πειράματος, το σύστημα παραμένει σταθερά διαθέσιμο για τους χρήστες. Η καθυστέρηση είχε σημαντική αύξηση της τάξης των 700%. Αυτό σημαίνει ότι σύστημα λειτουργεί στα όρια της υπερχείλισης της ενδιάμεσης μνήμης. Οι χρήστες, πλέον, αντιλαμβάνονται την καθυστέρηση.

4.3.3. Περιορισμός του εύρους ζώνης στα 1 Kbps για 20 λεπτά

Περιγραφή πειράματος

Σε αυτό το πείραμα γίνεται επιθετικός περιορισμός του εύρους ζώνης μέχρι τα 1 Kbps, για μεγάλη χρονική διάρκεια. Σε αυτό το σενάριο προσομοιώνονται ακραίες καταστάσεις του δικτύου όπως είναι κάποιες αυστηρές ρυθμίσεις και πολιτικές στο τείχος προστασίας, λανθασμένη τμηματοποίηση των πακέτων δεδομένων (fragmentation), βλάβη στο φυσικό μέσο μετάδοσης (καλώδιο, οπτική ίνα).



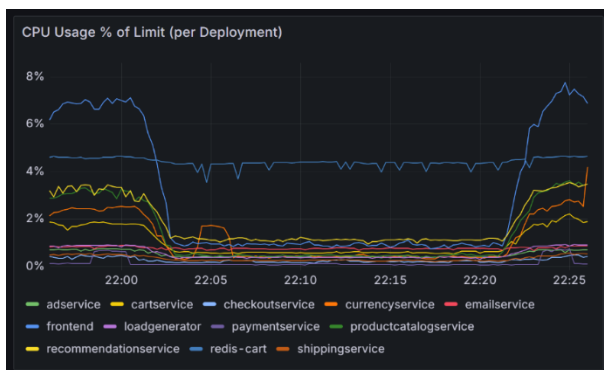
Εικόνα 34. Ορισμός του πειράματος περιορισμού του εύρους ζώνης στα 1 Kbps.

Το συγκεκριμένο πείραμα επιλέχθηκε για να παρατηρηθεί το σύστημα σε περιόδους ακραίας υποβάθμισης του δικτύου.



Εικόνα 35. Μετρικές Δικτύου για περιορισμό του εύρους ζώνης στα 1 Kbps.

Από τις δύο πρώτες μετρικές του δικτύου είναι φανερό ότι ο ρυθμός μετάδοσης πακέτων σχεδόν μηδενίζεται. Στην συνέχεια βλέπουμε ότι η διαθεσιμότητα μειώνεται παροδικά σε δύο διαστήματα, το οποίο σχετίζεται άμεσα με την καθυστέρηση, αφού στα χρονικά διαστήματα που μειώνεται η διαθεσιμότητα ταυτόχρονα η καθυστέρηση φτάνει τα 5 δευτερόλεπτα. Η συσχέτιση αυτή υπάρχει γιατί τα 5 δευτερόλεπτα είναι το χρονικό όριο λήξης για την αναμονή ολοκλήρωσης κάθε αιτήματος.



Εικόνα 36. Ποσοστό χρήσης επεξεργαστικής ισχύος ανά αντικείμενο ανάπτυξης (1 Kbps).

Πίνακας 9. Ακριβείς μεταβολές των μετρικών λόγω του περιορισμού στα 1 Kbps.

	Κατάσταση Λειτουργίας			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Εισερχόμενη Διαμεταγωγή (kB/s)	43.41	9.4	60.6	4.99 (-88.51%)	3.19	6.82
Εξερχόμενη Διαμεταγωγή (kB/s)	42.39	6.61	62.1	4.85 (-88.56%)	2.88	6.77
Εισερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	11.02	9.75	11.5	1.18 (-89.29%)	0.94	3.66
Εξερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	32.5	29	34.1	2.32 (-92.86%)	1.96	9.51
Καθυστέρηση p95 (ms)	14.84	13.4	15.6	2356.78 (+15781%)	822	5000
Καθυστέρηση p99 (ms)	14.87	13.4	15.6	2404.6 (+16071%)	822	5000
Διαθεσιμότητα (%)	100%	100%	100%	92.77% (-7.23%)	0%	100%
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από την ιστοσελίδα (%)	6.84%	6.16%	7.1%	0.92% (-86.55%)	0.74%	1.21%

Συνδυάζοντας τις πληροφορίες από τις εικόνες 35 και 36 με αυτές του Πίνακα 9 βλέπουμε ότι ο οριακός μηδενισμός (μείωση κατά 90%) της διαμεταγωγής και των εισερχόμενων και εξερχόμενων δεδομένων, οδήγησε σε σχεδόν πλήρη αδρανοποίηση των μικροϋπηρεσιών. Αυτό συμβαίνει γιατί τα αιτήματα που αργούν τόσο πολύ να φτάσουν, οδηγώντας τες σε μία μόνιμη κατάσταση αναμονής.

Τέλος αξίζει να σημειωθεί ότι σε πειράματα στα οποία εφαρμόζεται περαιτέρω μείωση του εύρους ζώνης, παρατηρείται πλήρης κατάρρευση του συστήματος και μηδενική διαθεσιμότητα του για τους χρήστες.

4.3.4. Προσθήκη 80ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά

Περιγραφή πειράματος

Σε αυτό το πείραμα εισάγεται μία επιπλέον καθυστέρηση των 80ms (με διακύμανση 20ms) στις δικτυακές συνδέσεις μεταξύ των μικροϋπηρεσιών του συστήματος. Μία τέτοια κατάσταση θα μπορούσε να παρουσιαστεί σε περίπτωση μέτριας συμφόρησης στο σύστημα, σε περιπτώσεις που το δίκτυο του συστήματος περιορίζεται από τον πυρήνα (kernel), όταν η δρομολόγηση των πακέτων είναι ασταθής ή όταν υπάρχουν ζημιές στο φυσικό μέσο μετάδοσης. Στόχος είναι ο έλεγχος της μακροπρόθεσμης σταθερότητας όταν εμφανιστεί αύξηση στην καθυστέρηση των πακέτων.

The screenshot displays the Chaos Mesh interface for an experiment named 'delay-80ms-20m'. The top section shows the experiment's metadata, including its namespace ('online-boutique'), UUID, and creation time. Below this, the 'Events' section lists three successful events: updating resource records, recovering chaos for 'productcatalogservice', and recovering chaos for 'productcatalogservice'. The 'Definition' section on the right shows the YAML configuration for the experiment, which is a NetworkChaos of kind 'delay' with a duration of 20m, a latency of 80ms, and a jitter of 20ms. A 'Download' button is visible next to the definition.

Metadata	Scope	Experiment	Run
Namespace: online-boutique	Namespace Selectors: online-boutique	Kind: NetworkChaos	Duration: 20m
UUID: 6279fb3b-47cc-4044-973b-f6b04812dd14		Action: delay	
Created at: 2025-10-13 20:51:47 PM		Direction: both	

```

1 kind: NetworkChaos
2 apiVersion: chaos-mesh.org/v1alpha1
3 metadata:
4   namespace: online-boutique
5   name: delay-80ms-20m
6 spec:
7   selector:
8     namespaces:
9       - online-boutique
10  mode: all
11  action: delay
12  duration: 20m
13  delay:
14    latency: 80ms
15    correlation: '25'
16    jitter: 20ms
17  direction: both
18  target:
19    selector:
20      namespaces:
21        - online-boutique
22  mode: all
  
```

Εικόνα 37. Ορισμός του πειράματος αύξησης καθυστέρησης κατά 80ms.

Στον ορισμό του πειράματος, τα πεδία που αξίζουν προσοχή είναι η καθυστέρηση (latency) στην οποία ορίζουμε την επιθυμητή τιμή 80ms και η συσχέτιση η οποία ορίζεται στο 25%, δηλαδή ότι υπάρχει 25% πιθανότητα δύο συνεχόμενα πακέτα να ανήκουν στο ίδιο αίτημα. Τέλος, σημαντικό είναι το πεδίο της διακύμανσης το οποίο ορίζεται στα 20ms. Με άλλα λόγια σε κάθε πακέτο η καθυστέρηση που προστίθεται είναι μία τυχαία τιμή στο διάστημα 60 με 100ms.



Εικόνα 38. Μετρικές δικτύου για αύξηση της καθυστέρησης κατά 80ms.

Παρατηρούμε πως εισάγοντας μία μικρή καθυστέρηση μειώνεται ελαφρώς ο ρυθμός μετάδοσης δεδομένων από και προς κάθε υπηρεσία και στον χώρο ονομάτων γενικότερα. Παρόλα αυτά η καθυστέρηση των αιτημάτων προς την ιστοσελίδα αυξάνεται κατά πολύ περισσότερο από τα 80ms, χωρίς όμως να μειώσει καθόλου την διαθεσιμότητά της.

Πίνακας 10. Αναλυτικές μετρήσεις των μετρικών δικτύου για αύξηση της καθυστέρησης κατά 80ms.

	Κατάσταση Λειτουργίας			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Εισερχόμενη Διαμεταγωγή (kB/s)	43.12	19.4	59.2	33.39 (-22.56%)	9.61	48.4
Εξερχόμενη Διαμεταγωγής (kB/s)	40.02	10.2	58.4	34.61 (-13.52%)	7.06	47.5
Εισερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	10.67	10.1	11.4	8.73 (-18.18%)	7.99	9.51
Εξερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	31.11	29.1	33.1	25.60 (-17.71%)	23	28.4
Καθυστέρηση p95 (ms)	16.32	14.3	17.7	1291.64 (+7 814%)	1160	1490

Καθυστέρηση p99 (ms)	16.4	14.4	17.8	1293.56 (+7 788%)	1160	1490
Διαθεσιμότητα (%)	100%	100%	100%	100% (-0%)	100%	100%

Βλέποντας και τον Πίνακα 10 βλέπουμε ότι παρόλο που η μείωση του ρυθμού μετάδοσης των δεδομένων μειώθηκε κατά 15-20% η αύξηση της καθυστέρησης είναι κατά πολύ μεγαλύτερη. Αυτό συμβαίνει γιατί ένα αίτημα στην ιστοσελίδα του συστήματος απαιτεί στην συνέχεια κλήσεις (παράλληλες και σειριακές) προς άλλες υπηρεσίες όπως αυτή των διαφημίσεων, του καλαθιού, των προτάσεων, κλπ. Έτσι ο συνολικός χρόνος ολοκλήρωσης ενός αιτήματος προς την ιστοσελίδα αθροίζοντας όλες τις επιμέρους κλήσεις φτάνει σχεδόν τα 1.3 δευτερόλεπτα.

4.3.5. Προσθήκη 150ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά

Περιγραφή πειράματος

Σε αυτό το σενάριο η καθυστέρηση αυξάνεται κατά 150ms, αυτή η αύξηση μπορεί να αναπαραστήσει σοβαρά προβλήματα δρομολόγησης ή επικοινωνία μεταξύ πολύ απομακρυσμένων κόμβων. Στόχος ο εντοπισμός πιθανών λήξεων χρονικών ορίων ή σφάλματα συγχρονισμού.

delay-150ms-20m Completed

Metadata		Scope	Experiment		Run	
Namespace	online-boutique	Namespace Selectors	online-boutique	Kind	NetworkChaos	Duration
UUID	08133edf-6ebd-4c04-bbb6-7efab18d9353			Action	delay	20m
Created at	2025-10-13 22:10:25 PM			Direction	both	

Events

delay-150ms-20m

Successfully update records of resource

7 MINUTES AGO

delay-150ms-20m

Successfully recover chaos for online-boutique/productcatalogservice-7d8786d8b5-4qfls

7 MINUTES AGO

delay-150ms-20m

Successfully recover chaos for online-boutique/productcatalogservice-7d8786d8b5-4qfls

7 MINUTES AGO

Definition

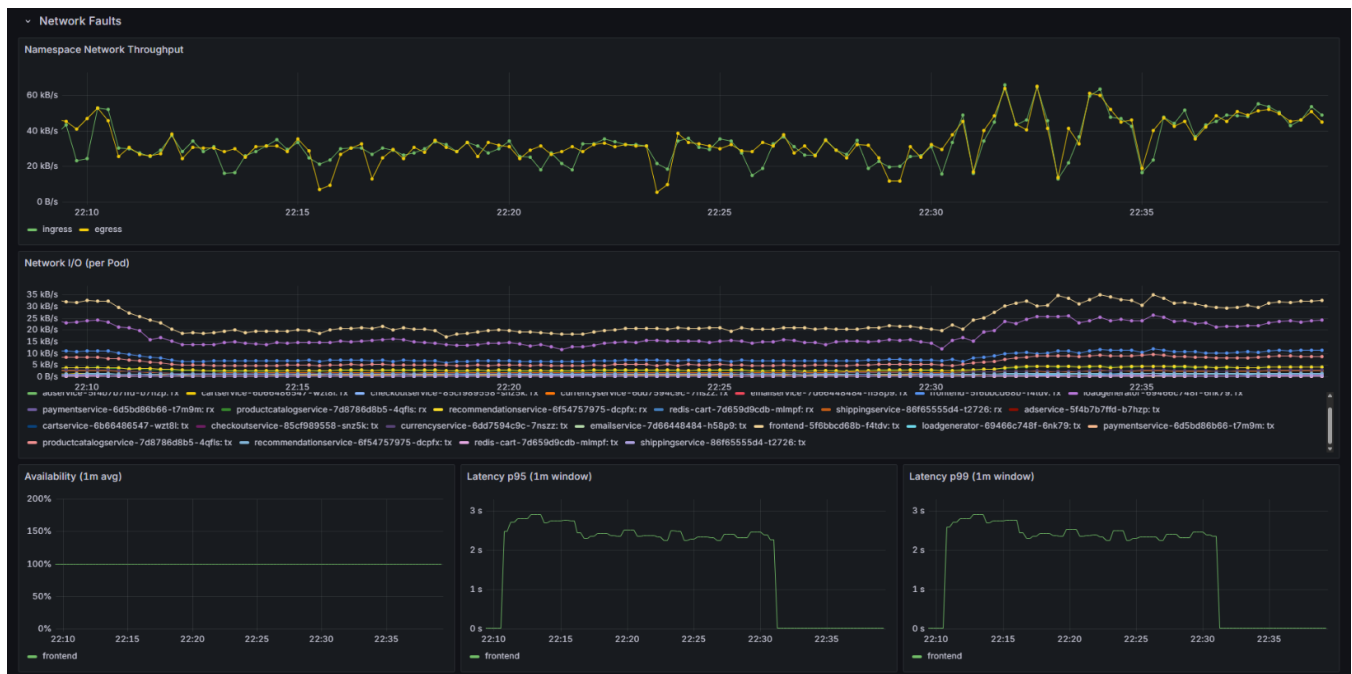
```

1 kind: NetworkChaos
2 apiVersion: chaos-mesh.org/v1alpha1
3 metadata:
4   namespace: online-boutique
5   name: delay-150ms-20m
6 spec:
7   selector:
8     namespaces:
9       - online-boutique
10  mode: all
11  action: delay
12  duration: 20m
13  delay:
14    latency: 150ms
15    correlation: '25'
16    jitter: 50ms
17  direction: both
18  target:
19    selector:
20      namespaces:
21        - online-boutique
22  mode: all

```

Download

Εικόνα 39. Ορισμός του πειράματος αύξησης καθυστέρησης κατά 150ms.



Εικόνα 40. Μετρικές δικτύου για αύξηση της καθυστέρησης κατά 150ms.

Παρατηρείται αισθητή μείωση στην ταχύτητα μετάδοσης δεδομένων από και προς τον χώρο ονομάτων και για κάθε μικροϋπηρεσία ξεχωριστά. Η καθυστέρηση έχει και πάλι μία πολύ μεγαλύτερη αύξηση από την μείωση του ρυθμού κάνοντας πιο εμφανή την εκθετική σχέση μεταξύ των μετρικών. Παρά την μεγάλη αύξηση το σύστημα παραμένει πλήρως διαθέσιμο, προσφέροντας όμως πολύ δυσάρεστη εμπειρία χρήσης στους πελάτες.

Πίνακας 11. Αναλυτικές μετρήσεις των μετρικών δικτύου για αύξηση της καθυστέρησης κατά 150ms.

	Κατάσταση Λειτουργίας			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Εισερχόμενη Διαμεταγωγή (kB/s)	42.74	23.3	62.8	27.14 (-36.50%)	11.5	39.8
Εξερχόμενη Διαμεταγωγή (kB/s)	45.81	37.2	59.4	26.53 (-42.09%)	9.05	38.5
Εισερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	10.8	9.84	11.5	6.96 (-35.56%)	5.56	9.12
Εξερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	31.43	28.3	33.4	19.93 (-36.59%)	15.8	26
Καθυστέρηση p95 (ms)	17.57	15.6	28.2	2471.08 (+13964%)	2250	2910
Καθυστέρηση p99 (ms)	17.68	15.7	28.8	2475.14 (+13900%)	2250	2930

Διαθεσιμότητα (%)	100%	100%	100%	100% (-0%)	100%	100%
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από την ιστοσελίδα (%)	6.79%	5.98%	7.23%	6.15% (-9.43%)	5.29%	6.8%

Ο χρόνος καθυστέρησης των αιτημάτων έχει γίνει σχεδόν 2.5 δευτερόλεπτα με μέγιστες τιμές τα 2.9 δευτερόλεπτα. Αυτό θα οδηγούσε πολλούς χρήστες σε εγκατάλειψη της προσπάθειας σύνδεσης στο σύστημα σε πραγματικές συνθήκες λειτουργίας. Ακόμη στον Πίνακα 11 βλέπουμε ότι η μείωση του ρυθμού μετάδοσης των δεδομένων έχει οδηγήσει σε ελαφριά μείωση χρήσης της επεξεργαστικής ισχύος. Δηλαδή, το σύστημα ολοκληρώνει αιτήματα πριν προλάβουν να καταφθάσουν νέα, οδηγώντας το σε κατάσταση αδράνειας για μικρά χρονικά διαστήματα.

4.3.6. Προσθήκη 300ms καθυστέρησης μεταξύ των μικροϋπηρεσιών για 20 λεπτά

Περιγραφή πειράματος

Το πείραμα αυτό αποτελεί το πιο απαιτητικό σενάριο προσθήκης καθυστέρησης, καθώς επιβάλλει πολύ υψηλή τιμή σε αυτή ($300\text{ms} \pm 100\text{ms}$) για 20 λεπτά μεταξύ όλων των μικροϋπηρεσιών. Προσομοιώνει σοβαρή και παρατεταμένη αποσταθεροποίηση του δικτύου, όπως προβλήματα σε φυσικό μέσο ή μη βέλτιστη δρομολόγηση σε πολλαπλές δρομολογήσεις. Στόχος είναι να μελετηθεί η συμπεριφορά του συστήματος υπό συνθήκες συνεχούς υψηλής καθυστέρησης. Εάν δηλαδή διατηρείται στοιχειώδης διαθεσιμότητα, εάν ενεργοποιούνται βρόχοι επανάληψης αποστολής των αιτημάτων, καθώς και ο χρόνος αποκατάστασης μετά το πέρας του πειράματος.

delay-300ms-20m

Completed

Metadata		Scope	Experiment		Run	
Namespace	online-boutique	Namespace Selectors	online-boutique	Kind	NetworkChaos	Duration
UUID	a023302e-8c1d-4125-928d-67ceea535673			Action	delay	20m
Created at	2025-10-13 23:35:32 PM			Direction	both	

Events

delay-300ms-20m

Successfully update records of resource

20 HOURS AGO

delay-300ms-20m

Successfully recover chaos for online-boutique/adservice-5f4b7b7ffd-b7hzp

20 HOURS AGO

delay-300ms-20m

Successfully recover chaos for online-boutique/adservice-5f4b7b7ffd-b7hzp

20 HOURS AGO

delay-300ms-20m

20 HOURS AGO

Definition

```

1 kind: NetworkChaos
2 apiVersion: chaos-mesh.org/v1alpha1
3 metadata:
4   namespace: online-boutique
5   name: delay-300ms-20m
6 spec:
7   selector:
8     namespaces:
9       - online-boutique
10  mode: all
11  action: delay
12  duration: 20m
13  delay:
14    latency: 300ms
15    correlation: '25'
16    jitter: 50ms
17  direction: both
18  target:
19    selector:
20      namespaces:
21        - online-boutique
22    mode: all

```

Download

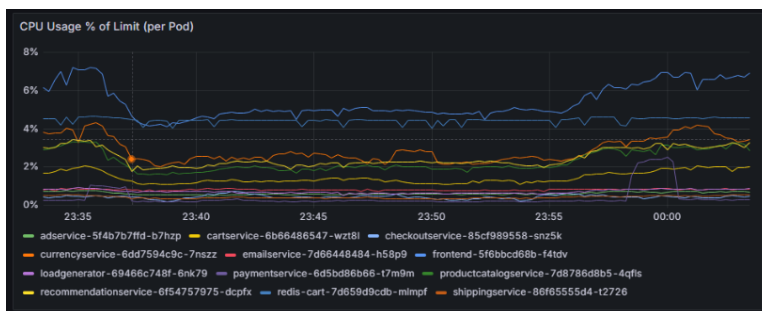
Εικόνα 41. Ορισμός του πειράματος αύξησης καθυστέρησης κατά 300ms.

Λαμβάνοντας υπόψη τα αποτελέσματα των προηγούμενων πειραμάτων, από αυτό το πείραμα αναμένουμε μείωση της διαθεσιμότητας του συστήματος λόγω υπέρβασης των χρονικών ορίων σε κάποια από τα αιτήματα.



Εικόνα 42. Μετρικές δικτύου για αύξηση της καθυστέρησης κατά 300ms.

Όπως αναφέρθηκε στην υπόθεση που έγινε κατά την υποβολή του πειράματος παρατηρούμε, μετά την μεγάλη μείωση του ρυθμού μετάδοσης των δεδομένων, ότι η διαθεσιμότητα του συστήματος μειώνεται και μηδενίζεται στο πρώτο διάστημα εκτέλεσης του πειράματος. Στην συνέχεια το σύστημα επανέρχεται και είναι πλήρως διαθέσιμο μέχρι και το τέλος. Αυτό σχετίζεται άμεσα με τις επόμενες δύο χρονοσειρές των καθυστερήσεων. Σε αυτές φαίνεται πως ο χρόνος ολοκλήρωσης ενός αιτήματος ξεπερνάει τα 5 δευτερόλεπτα, στο οποίο χρονικό σημείο τελειώνει ο μέγιστος χρόνος αναμονής και ακυρώνεται το αίτημα.



Εικόνα 43. Ποσοστό χρήσης επεξεργαστή για αύξηση της καθυστέρησης κατά 300ms.

Λόγω της μεγάλης αύξησης στην καθυστέρηση της μετάδοσης των πακέτων, οι μικροϋπηρεσίες ολοκληρώνουν τις διεργασίες που εκτελούσαν και αδρανοποιούνται μέχρι να καταφτάσει το επόμενο αίτημα. Με το πέρας του πειράματος βλέπουμε γρήγορη επαναφορά του συστήματος στην κανονική, χωρίς προβλήματα, λειτουργία του.

Πίνακας 12. Αναλυτικές μετρήσεις του δικτύου για αύξηση της καθυστέρησης κατά 300ms.

	Κατάσταση Λειτουργίας			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Εισερχόμενη Διαμεταγωγή (kB/s)	40.61	19.5	51.5	22.43 (-44.77%)	9.83	30.9
Εξερχόμενη Διαμεταγωγή (kB/s)	42.47	11.4	55.5	21.65 (-49.02%)	4.61	30.8
Εισερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	10.61	10	11.3	5.26 (-50.42%)	4.43	5.79
Εξερχόμενα δεδομένα στην ιστοσελίδα (kB/s)	30.96	29.7	32.7	15.43 (-50.16%)	12.3	17.6
Καθυστέρηση p95 (ms)	15.89	14.2	18.3	4465.07 (+28000%)	4190	5000
Καθυστέρηση p99 (ms)	15.97	14.2	18.4	4469.58 (+27887%)	4190	5000
Διαθεσιμότητα (%)	100%	100%	100%	83.10% (-16.90%)	0%	100%
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από την ιστοσελίδα (%)	6.59%	6.13%	6.93%	4.80% (-27.16%)	3.89%	5.41%

Παρατηρώντας τις τιμές της καθυστέρησης γίνεται αντιληπτό ότι τα περισσότερα αιτήματα ολοκληρώνονται οριακά πριν την λήξη του χρονικού περιθωρίου. Ενώ η διαθεσιμότητα μειώνεται αισθητά, η τιμή όμως 83.1% είναι παραπλανητική γιατί η διαθεσιμότητα είναι 0% (με μία παροδική ανάκαμψη) στην αρχή του πειράματος και στην συνέχεια επανέρχεται.

4.4. Πειράματα σε επίπεδο πόρων

Τα πειράματα σε επίπεδο πόρων στοχεύουν στην παρατήρηση της συμπεριφοράς της συστάδας σε συνθήκες υψηλού φόρτου ή περιορισμένων πόρων. Μέσω τεχνητής επιβάρυνσης της επεξεργαστικής ισχύος, της μνήμης ή του εύρους ζώνης ελέγχεται πως ανταποκρίνονται οι μηχανισμοί αυτόματης κλιμάκωσης (οριζόντιας/κατακόρυφης), η κατανομή των πόρων και οι πολιτικές των ορίων και των αιτημάτων των μικροϋπηρεσιών. Αυτά τα σενάρια αντιπροσωπεύουν αιφνίδιες αυξήσεις των χρηστών, αύξηση των εργασιών υψηλής επεξεργασίας και κακή διαχείριση μνήμης από τις εφαρμογές.

Πραγματικά Σενάρια

Πιθανά σενάρια που μπορεί να οδηγήσουν σε καταστάσεις που θα προσομοιωθούν παρακάτω είναι:

1. Ξαφνική αύξηση των χρηστών ή του φορτίου.
2. Επίθεση τύπου καταναμημένης άρνησης υπηρεσιών.
3. Προβληματικές διεργασίες ή αλγόριθμοι με υψηλό υπολογιστικό κόστος.
4. Διαρροές μνήμης ή πόροι που δεν αποδεδμεύονται ποτέ από κάποια υπηρεσία.
5. Κακή πολιτική πόρων σε κάποιον κόμβο.

4.4.1. Μέτρια καταπόνηση πόρων επεξεργαστή και μνήμης

Περιγραφή πειράματος

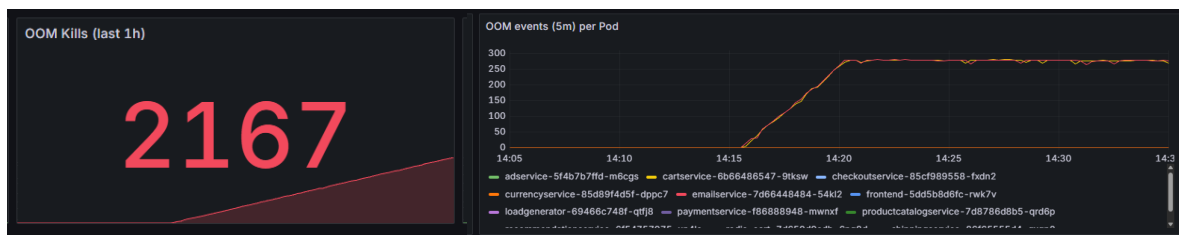
Σε αυτό το πείραμα εφαρμόζεται πίεση στους πόρους του συστήματος για 20 λεπτά. Στόχος είναι η αξιολόγηση της απόκρισης του συστήματος σε συνθήκες μέτριου φορτίου.

The screenshot displays the Chaos Mesh interface for an experiment named 'full-stress-medium-20m', which is marked as 'Completed'. The interface is divided into several sections:

- Metadata:** Shows details for the 'online-boutique' namespace, including a UUID and creation time (2025-11-15 14:15:28 PM).
- Scope:** Indicates the 'Namespace Selectors' are set to 'online-boutique'.
- Experiment:** Details the 'Kind' as 'StressChaos', 'CPU' stressors with 'workers: 2' and 'size: 96MIB', and 'Memory' stressors with 'workers: 1' and 'size: 96MIB'.
- Run:** Shows the 'Duration' as '20m'.
- Events:** A list of three events, all occurring '34 MINUTES AGO', showing successful updates and recoveries of chaos for various resources.
- Definition:** A code block showing the YAML definition of the experiment, including namespace, name, spec, selector, and stressors.
- Download:** A button to download the definition.

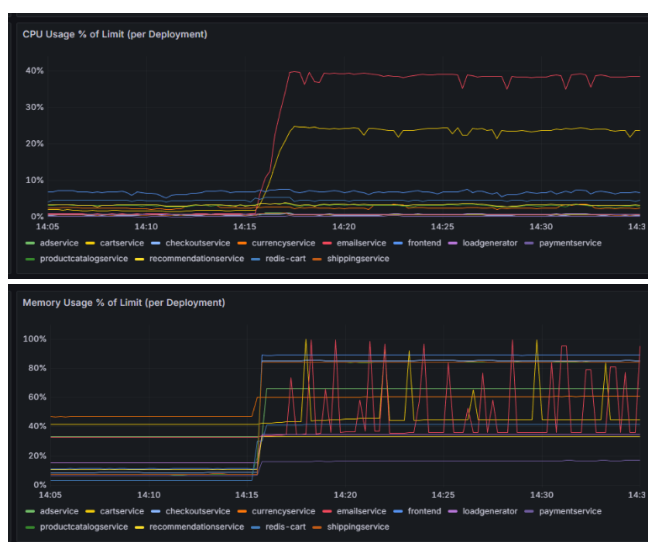
Εικόνα 44. Ορισμός του πειράματος μέτριας καταπόνησης των πόρων.

Από τον ορισμό βλέπουμε ότι το πείραμα έχει εμβέλεια ολόκληρο τον χώρο ονομάτων online-boutique και εφαρμόζεται σε όλα τα pods. Οι μηχανισμοί πίεσης που εφαρμόζονται είναι μνήμης και επεξεργαστικής ισχύος. Για την μνήμη δημιουργείται μία διεργασία που εφαρμόζεται σε κάθε μικροϋπηρεσία η οποία καταναλώνει μέχρι και 96 MB, αν το επιτρέπουν τα όρια του αντικειμένου ανάπτυξης. Οι διεργασίες που δημιουργούνται για να καταναλώνουν επεξεργαστική ισχύ είναι 2 και καταναλώνουν μέχρι και το 80% της ισχύος 2 εικονικών πυρήνων.

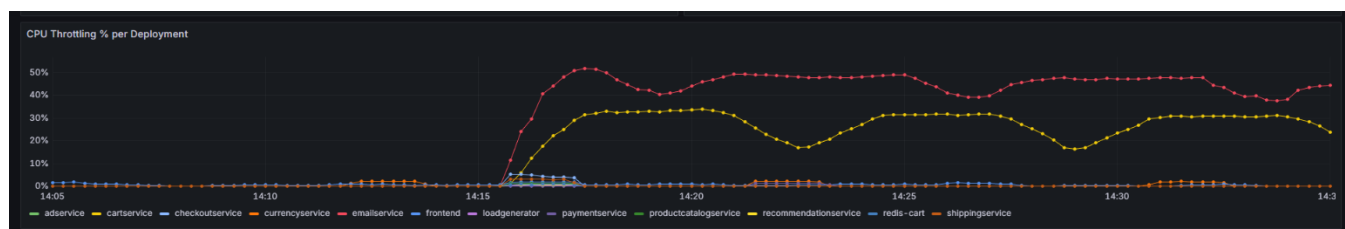


Εικόνα 45. Μετρικές γεγονότων εξάντλησης μνήμης.

Στο αριστερό διάγραμμα βλέπουμε ότι κατά την διάρκεια του πειράματος υπήρξαν περισσότερα από 2000 γεγονότα εξάντλησης μνήμης, ενώ στο δεξί διάγραμμα παρατηρείται ότι αυτά τα γεγονότα αφορούν μόνο δύο μικροϋπηρεσίες, του καλαθιού αγορών και του ηλεκτρονικού ταχυδρομείου.

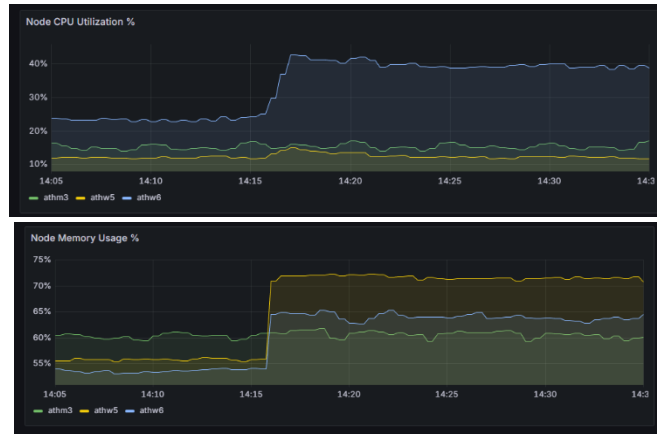


Εικόνα 46. Μετρικές χρήσης πόρων από το σύστημα κατά την μέτρια κατανάλωση πόρων (α).



Εικόνα 47. Μετρικές χρήσης πόρων από το σύστημα κατά την μέτρια κατανάλωση πόρων (β).

Στα παραπάνω διαγράμματα φαίνεται η αύξηση κατανάλωση μνήμης από όλες τις υπηρεσίες του συστήματος, με το καλάθι και το ηλεκτρονικό ταχυδρομείο να εμφανίζουν συχνές κορυφώσεις οι οποίες είναι και ο πιθανός λόγος που συμβαίνουν τα οι τερματισμοί των αντίστοιχων pods λόγω εξάντλησης μνήμης. Επιπλέον, στο δεξιά διάγραμμα της κατανάλωσης επεξεργαστικής ισχύος παρατηρείται σημαντική αύξηση κατανάλωσης από τις ίδιες δύο μικροϋπηρεσίες, ενώ στο τελευταίο σχήμα βλέπουμε την συνεχή προσπάθεια από τον πυρήνα να περιορίσει την λειτουργία τους.



Εικόνα 48. Κατανάλωση πόρων ανά εικονική μηχανή.

Η αύξηση κατανάλωσης φαίνεται και σε επίπεδο συστάδας, αφού ο κόμβος εκτέλεσης 6 παρουσιάζει σημαντική αύξηση χρήσης των διαθέσιμων πόρων από 23.3% σε 39.9%. Ταυτόχρονα και οι δύο εικονικές μηχανές (κόμβοι εκτέλεσης 5 και 6) καταναλώνουν περισσότερη μνήμη κατά την διάρκεια του πειράματος.

Πίνακας 13. Αναλυτική μεταβολή κατανάλωσης πόρων κατά την μέτρια καταπόνηση τους.

Μετρική	Αρχική Κατάσταση			Διάρκεια Πειράματος		
	Μέσος Όρος	Ελάχιστο	Μέγιστο	Μέσος Όρος	Ελάχιστο	Μέγιστο
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από το καλάθι (%)	1.76%	1.41%	2.02%	23.75% (+1249.4%)	20.7%	24.8%
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από το ηλ. Ταχυδρομείο (%)	0.85%	0.81%	0.9%	38.40% (+4417.6%)	33.9%	39.7%
Ποσοστό κατανάλωσης της Διαθέσιμης μνήμης από το καλάθι (%)	41.42%	41.4%	41.5%	48.40% (+16.9%)	43.2%	99.6%
Ποσοστό κατανάλωσης της Διαθέσιμης μνήμης από το ηλ. Ταχυδρομείο (%)	32.61%	32.6%	32.7%	50.42% (+54.6%)	34.5%	99.6%
Περιορισμός επεξεργαστικής ισχύος του καλαθιού (%)	0%	0%	0%	28.02% (-)	16.5%	33.8%
Περιορισμός επεξεργαστικής ισχύος του ηλ. Ταχυδρομείου (%)	0%	0%	0%	45.58% (-)	37.7%	51.7%
Αξιοποίηση Επεξεργαστικής ισχύος από τον κόμβο εκτέλεσης 5 (%)	12.07%	11.7%	12.5%	12.52% (+3.7%)	11.7%	15%
Αξιοποίηση Επεξεργαστικής ισχύος από τον κόμβο εκτέλεσης 6 (%)	23.34%	22.7%	24.3%	39.86% (+70.8%)	38.3%	42.7%

Χρήση Μνήμης από τον κόμβο εκτέλεσης 5 (%)	55.79%	55.4%	56.2%	71.60% (+28.3%)	70.7%	72.2%
Χρήση Μνήμης από τον κόμβο εκτέλεσης 6 (%)	53.60%	53.1%	54.1%	63.97% (+19.3%)	62.6%	65.3%

4.4.2. Βαριά καταπόνηση πόρων επεξεργαστή και μνήμης

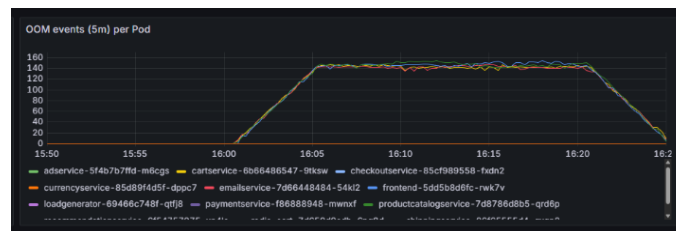
Περιγραφή πειράματος

Σε αυτό το σενάριο ωθούμε το σύστημα σε κατανάλωση των πόρων στα όρια σταθερότητας του κόμβου, με στόχο την παρακολούθηση όλων των γεγονότων εξάντλησης μνήμης ή μη διαθεσιμότητας του συστήματος.

The screenshot displays the Chaos Mesh interface for an experiment named 'full-stress-aggressive-20m'. The top section shows the experiment's metadata, including its namespace ('online-boutique'), UUID, and creation time. The 'Scope' section lists namespace selectors and the target namespace. The 'Experiment' section details the kind ('StressChaos'), CPU, and Memory specifications. The 'Run' section shows the duration ('20m'). Below this, the 'Events' section lists three successful events: updating resource records, recovering chaos for 'emailservice-7d66448484-54k12/server', and recovering chaos for 'checkoutservice-85cf989558-'. The 'Definition' section shows the YAML configuration for the experiment, including the kind, apiVersion, namespace, name, spec, selector, namespaces, mode, stressors, memory, workers, cpu, load, and duration.

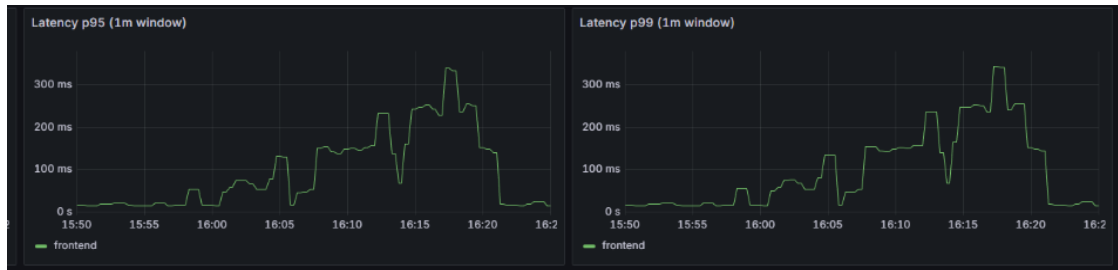
Εικόνα 49. Ορισμός του πειράματος μεγάλης καταπόνησης των πόρων.

Στον ορισμό αυξήθηκαν οι πόροι που καταναλώνονται μέσω του πειράματος. Δηλαδή οι διεργασίες κατανάλωσης μνήμης έγιναν 2 και η κάθε μία μπορεί να καταναλώσει μέχρι 192 MB, αν το σύστημα δεν παρέμβει, ενώ η κατανάλωση επεξεργαστικής ισχύος θα γίνει μέσω 4 διεργασιών η οποίες θα καταναλώνουν την επεξεργαστική ισχύ ενός ολόκληρου πυρήνα.



Εικόνα 50. Μετρικές επανεκκινήσεων και τερματισμών λόγω εξάντλησης μνήμης.

Στα παραπάνω διαγράμματα βλέπουμε και πάλι μεγάλο αριθμό γεγονότων εξάντλησης μνήμης (περισσότερα από 2300).



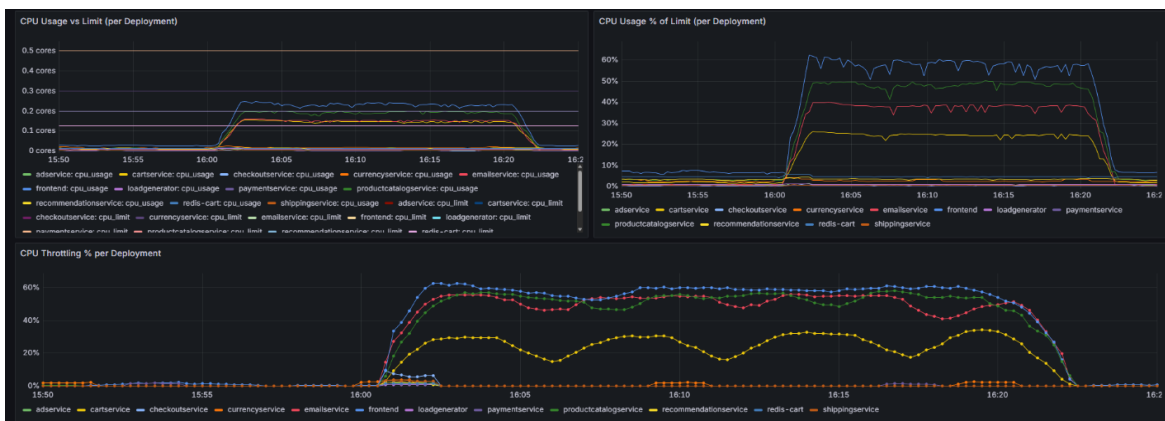
Εικόνα 51. Καθυστερήσεις p95 και p99.

Η μεγάλη κατανάλωση πόρων και δεν επιτρέπει στο σύστημα να διεκπεραιώνει πολλές από τις πραγματικές διεργασίες ταυτόχρονα, οπότε η καθυστέρηση των αιτημάτων αυξάνεται σημαντικά.



Εικόνα 52. Ποσοστό κατανάλωσης διαθέσιμης μνήμης ανά μικροϋπηρεσία.

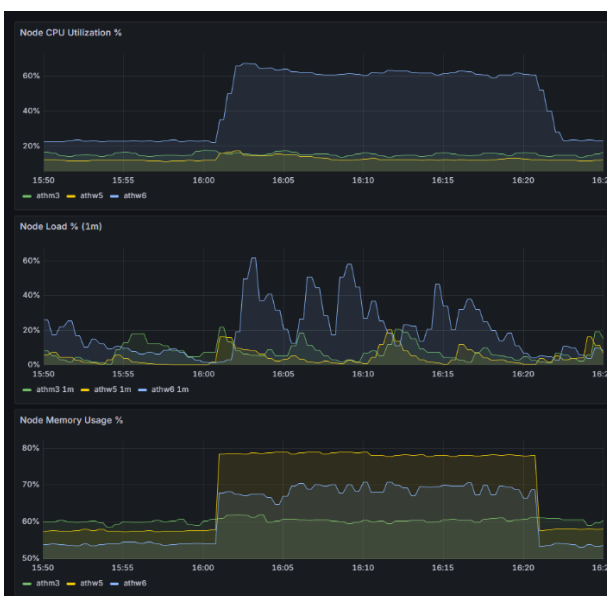
Παρόμοια με το προηγούμενο πείραμα παρατηρούμε μία αύξηση στην κατανάλωση της διαθέσιμης μνήμης από όλες τις μικροϋπηρεσίες. Στην περίπτωση όμως της μεγάλης κατανάλωσης πόρων οι κορυφώσεις που παρατηρούνταν σε δύο υπηρεσίες, τώρα παρατηρούνται σε 4 (καλάθι αγορών, ιστοσελίδα, κατάλογος προϊόντων και ηλεκτρονικό ταχυδρομείο).



Εικόνα 53. Ποσοστό κατανάλωσης υπολογιστικής ισχύος και περιορισμός διεργασιών.

Με ανάλογο τρόπο βλέπουμε την αύξηση κατανάλωσης υπολογιστικών πόρων ξανά από το καλάθι αγορών και το ηλεκτρονικό ταχυδρομείο και σε αυτά, πλέον προστίθενται ο κατάλογος προϊόντων και η ιστοσελίδα. Αυτό ήταν αναμενόμενο μετά την παρατήρηση που έγινε και στο διάγραμμα κατανάλωσης μνήμης της Εικόνας 53.

Σε αυτό το διάγραμμα βλέπουμε ότι η κατανάλωση των πόρων αυξάνεται ακόμα περισσότερα για τον κόμβο εκτέλεσης 6, ενώ, παρά την μεγάλη αύξηση στην πίεση των πόρων, ο κόμβος εκτέλεσης 5 παραμένει ανεπηρέαστος στην κατανάλωση υπολογιστικής ισχύος. Αντίθετα, η κατανάλωση μνήμης φαίνεται να αυξάνεται ακόμα περισσότερο, σε σχέση με το προηγούμενο πείραμα, και στους δύο κόμβους εκτέλεσης.



Εικόνα 54. Συνολική κατανάλωση πόρων του συστήματος ανά κόμβο.

Πίνακας 14. Αναλυτική μεταβολή κατανάλωσης πόρων κατά την μεγάλη καταπόνηση τους.

Μετρική	Αρχική Κατάσταση			Διάρκεια Πειράματος		
	Μέσος Όρος	Ελάχιστο	Μέγιστο	Μέσος Όρος	Ελάχιστο	Μέγιστο
Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από την ιστοσελίδα (%)	6.66%	5.83%	7.6%	57.65% (+765.6%)	50.7%	62.3%

Ποσοστό Χρήσης της Διαθέσιμης Ισχύος από το ηλ. ταχυδρομείο (%)	0.81%	0.66%	0.86%	37.92% (+4581.5%)	33.9%	39.9%
Ποσοστό κατανάλωσης της Διαθέσιμης μνήμης από την ιστοσελίδα (%)	12.46%	12.3%	12.7%	71.26% (+471.9%)	12.1%	99.7%
Ποσοστό κατανάλωσης της Διαθέσιμης μνήμης από το ηλ. ταχυδρομείο (%)	32.61%	32.6%	32.8%	51.07% (+56.6%)	34.7%	99.7%
Περιορισμός επεξεργαστικής ισχύος της ιστοσελίδας (%)	0.74%	0%	2.15%	58.61% (+7820.3%)	52.6%	62.7%
Περιορισμός επεξεργαστικής ισχύος του ηλ. Ταχυδρομείου (%)	0%	0%	0%	51.72% (-)	41.1%	56%
Αξιοποίηση Επεξεργαστικής ισχύος από τον κόμβο εκτέλεσης 5 (%)	12%	11.6%	12.5%	13.31% (+10.9%)	12.1%	17.1%
Αξιοποίηση Επεξεργαστικής ισχύος από τον κόμβο εκτέλεσης 6 (%)	23.13%	22.6%	23.8%	62.23% (+169%)	58.9%	67.1%
Χρήση Μνήμης από τον κόμβο εκτέλεσης 5 (%)	57.55%	57.3%	58%	78.32% (+36.1%)	77.6%	79%
Χρήση Μνήμης από τον κόμβο εκτέλεσης 6 (%)	53.95%	53.5%	54.5%	68.77% (+27.5%)	64.6%	70.8%

Ο παραπάνω πίνακας είναι μια ποσοτική επιβεβαίωση των ποιοτικών παρατηρήσεων που έγιναν με βάση τα διαγράμματα των εικόνων.

4.5. Προσομοίωση αύξησης αριθμού χρηστών για 20 λεπτά

Το συγκεκριμένο πείραμα δεν πρόκειται για ένα σενάριο του εργαλείου μηχανικής χάους Chaos Mesh, αλλά μία αύξηση στην εικονική κίνηση που εφαρμόζεται ούτως ή άλλως στην εφαρμογή του ηλεκτρονικού καταστήματος (online-boutique) που εκτελούμε στα πλαίσια της διπλωματικής. Με αυτόν τον τρόπο θα παρακολουθήσουμε την απόκριση του συστήματος αλλά και του περιβάλλοντος Kubernetes όταν ο αριθμός των χρηστών αυξάνεται ξαφνικά σε μεγάλο βαθμό.

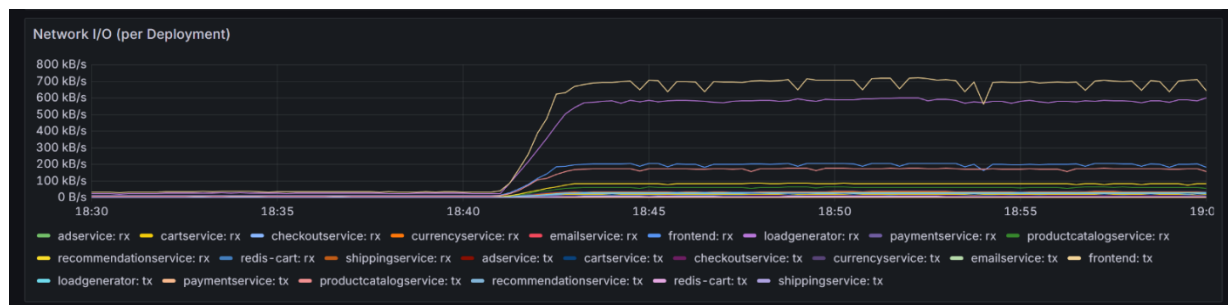
```
kubectl -n online-boutique set env deploy/loadgenerator \
  USERS=50 \
  RATE=5

kubectl -n online-boutique scale deploy loadgenerator --replicas=5

kubectl -n online-boutique rollout restart deploy/loadgenerator
```

Εικόνα 55. Εντολές για την αύξηση των εικονικών χρηστών.

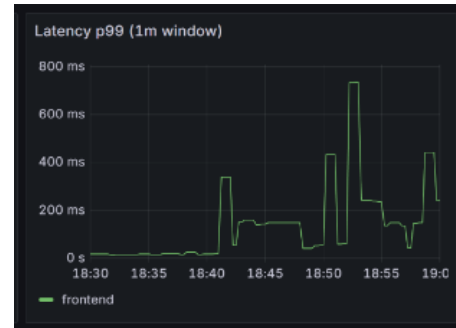
Για να προσομοιάσουμε την αύξηση αυτή, εκτελέσαμε στην γραμμή εντολών τις παραπάνω εντολές της Εικόνας 55. Αρχικά αυξήσαμε τον αριθμό εικονικών χρηστών σε 50 από 10 και τον ρυθμό αποστολής αιτημάτων σε 5 από 1 που ήταν αρχικά. Στην συνέχεια έγινε κλιμάκωση της υπηρεσίας προσομοίωσης χρηστών από 1 αντίγραφο σε 5, αυξάνοντας την εικονική κίνηση του συστήματος κατά 125 φορές συνολικά.



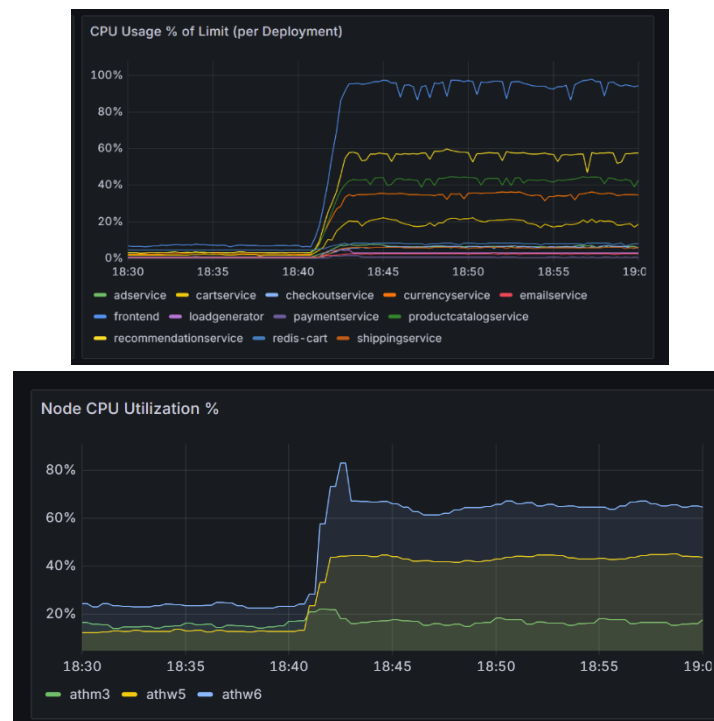
Εικόνα 56. Ρυθμός μετάδοσης δεδομένων ανά υπηρεσία.

Όπως ήταν αναμενόμενο ο ρυθμός μετάδοσης εισερχόμενων και εξερχόμενων δεδομένων ανά υπηρεσία αυξήθηκε με ανάλογο τρόπο. Για παράδειγμα η υπηρεσία της ιστοσελίδας που είχε εξερχόμενο ρυθμό δεδομένων περίπου 35kB/s, αύξησε τον ρυθμό αυτό στα 700kB/s. Με άλλα λόγια ο ρυθμός αυξήθηκε κατά 20 φορές. Ενώ ο εξερχόμενος ρυθμός του καταλόγου των προϊόντων από τα 5kB/s αυξήθηκε σχεδόν 40 φορές. Σε παρακάτω διάγραμμα θα γίνει κατανοητό γιατί δεν παρατηρήθηκε αύξηση στον ρυθμό σε αντίστοιχη κλίμακα με την αύξηση που ορίστηκε για τους χρήστες.

Εδώ (Εικόνα 57) μπορεί να γίνει αντιληπτός ο πρώτος λόγος που η αύξηση περιορίστηκε σε τιμές μεταξύ 40 και 20. Με την αύξηση των χρηστών το σύστημα άρχισε να δέχεται πολύ περισσότερα αιτήματα, τα οποία πλέον δεν μπορεί προωθήσει ταυτόχρονα στις αντίστοιχες μικροϋπηρεσίες. Κάποια πακέτα δεδομένων έμπαιναν στην ουρά αυξάνοντας τον χρόνο διεκπεραίωσης αιτημάτων και μειώνοντας τον μέγιστο δυνατό ρυθμό μεταφοράς δεδομένων.

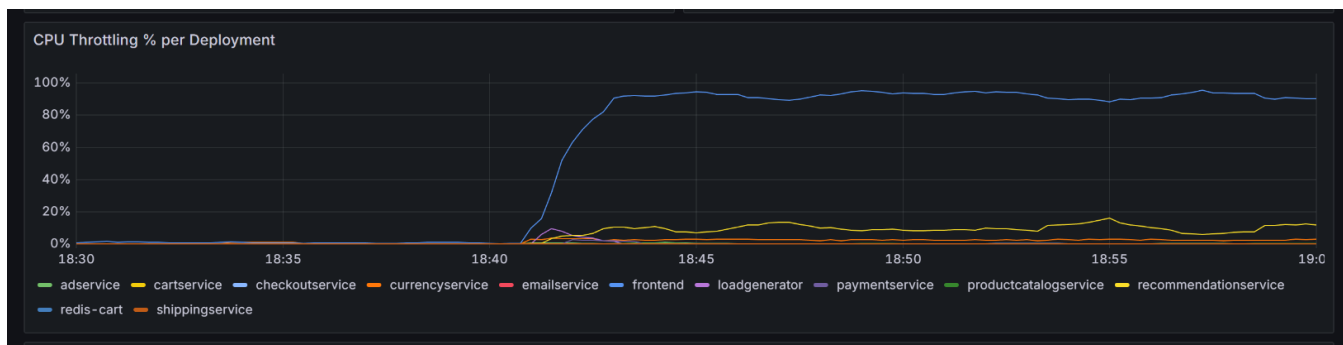


Εικόνα 57. Καθυστέρηση p99 των αιτημάτων.



Εικόνα 58. Κατανάλωση διαθέσιμης υπολογιστικής ισχύος σε επίπεδο μικροϋπηρεσιών (αριστερά) και κόμβων (δεξιά).

Όπως ήταν αναμενόμενο με την αύξηση των χρηστών και επομένως των αιτημάτων το σύστημα αύξησε την κατανάλωση διαθέσιμης της υπολογιστικής ισχύος, εκτελώντας περισσότερες διεργασίες παράλληλα.



Εικόνα 59. Περιορισμός χρήσης υπολογιστικής χρήσης ανά υπηρεσία.

Αυτή η ξαφνική αύξηση στην απαίτηση επεξεργαστικής ισχύος από το σύστημα οδήγησε στην αύξηση του περιορισμού χρήσης πόρων από τον πυρήνα για αρκετές μικροϋπηρεσίες σε μικρότερο βαθμό, αλλά σε μεγάλο βαθμό στην μικροϋπηρεσία της ιστοσελίδας η οποία ξαφνικά είχε πάρα πολλά αιτήματα και συνεπώς διεργασίες οι οποίες ξεπερνούσαν το ορισμένα όρια κατανάλωσης πόρων.

4.6. Βελτιώσεις Συστήματος

Εκτελώντας τα παραπάνω πειράματα, των τριών κατηγοριών που παρουσιάστηκαν (χάος σε επίπεδο pods, δικτύου και πόρων), ανέδειξαν αρκετές ευπάθειες του συστήματος. Κάποιες ευπάθειες ήταν μεγάλης σημασίας, όπως μη διαθεσιμότητα του συστήματος στους χρήστες παροδικά, και κάποιες άλλες μικρότερης σημασίας, όπως η αύξηση της καθυστέρησης διεκπεραίωσης αιτημάτων. Σε αυτή την ενότητα θα παρουσιαστούν κάποιες παρεμβάσεις που θα μπορούσαν να εφαρμοστούν στο σύστημα με στόχο την βελτίωση του μειώνοντας ή εξαλείφοντας τις αδυναμίες αυτές. Οι βελτιώσεις αυτές χωρίζονται, επίσης σε τρεις κατηγορίες: στρατηγικές κατανομής, διαχείριση των πόρων και διαχείριση της κυκλοφορίας των δεδομένων.

4.6.1. Στρατηγική κατανομής και υψηλής διαθεσιμότητας

1. Πολλαπλά αντίγραφα

Σε πειράματα που προκαλούσαν τερματισμό ή μη διαθεσιμότητα κάποιου pod, οδηγούσαν και μη διαθεσιμότητα μίας ολόκληρης υπηρεσίας ή και της εφαρμογής, καθώς όλες οι υπηρεσίες στην αρχική κατάσταση του πειράματος εκτελούνται με ένα μοναδικό αντίγραφο ανά αντικείμενο ανάπτυξης. Αυτό κάνει αρκετά ασταθές ολόκληρο το σύστημα και μειώνει σημαντικά την αξιοπιστία του. Λύση σε αυτό το πρόβλημα είναι η αύξηση των ελάχιστων αντιγράφων ανά μικροϋπηρεσία σε 2 εξαλείφοντας τα μοναδικά σημεία αποτυχίας (Single Point of Failure) ολόκληρου του συστήματος. Οι εντολές που μπορούν να εφαρμόσουν αυτό το μέτρο είναι οι παρακάτω:


```
NS=online-boutique
```

```
SRV="adservice cartservice checkoutservice currencyservice emailservice  
frontend paymentservice productcatalogservice recommendationservice  
shippingservice"
```

```
for s in $SRV; do kubectl -n $NS scale deploy/$s --replicas=2; done
```

```
kubectl -n $NS
```

2. Σωστή κατανομή των pods (Anti-Affinity)

Η απλή αύξηση των αντιγράφων δεν αποτελεί από μόνη της επαρκές μέτρο για την αύξηση της ανθεκτικότητας του συστήματος, στην περίπτωση που τα pods μίας υπηρεσίας εκτελούνται στον ίδιο κόμβο. Σε περίπτωση βλάβης του κόμβου, όπως είναι η διακοπή ρεύματος, η υπηρεσία θα είναι και πάλι μη διαθέσιμη. Ορίζοντας έναν κανόνα αποφυγής ορισμού των pods της ίδιας υπηρεσίας να εκτελούνται σε κοινό κόμβο, είναι δυνατό να αποφευχθεί ένα τέτοιο συμβάν. Αυτό μπορεί να εφαρμοστεί για ένα αντικείμενο ανάπτυξης στον παρόν σύστημα (π.χ. το καλάθι αγορών) ως εξής:

```
# 04-spread-cartservice.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: cartservice
```

```
  namespace: online-boutique
```

```
spec:
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: cartservice
```

```
    spec:
```

```
      affinity:
```

```
        podAntiAffinity:
```

```
          requiredDuringSchedulingIgnoredDuringExecution:
```

```
            - labelSelector:
```

```
              matchLabels: app: cartservice
```

```
              topologyKey: topology.kubernetes.io/hostname
```

```
        topologySpreadConstraints:
```

```
          - maxSkew: 1
```

```
            topologyKey: topology.kubernetes.io/hostname
```

```
            whenUnsatisfiable: DoNotSchedule
```

```
            labelSelector:
```

```
              matchLabels: app: cartservice
```

Και στην συνέχεια εκτελώντας τις εντολές:

```
kubectl apply -f 04-spread-cartservice.yaml
```

```
kubectl -n online-boutique get pods -o wide -l app=cartservice
```

3. Πολιτική προϋπολογισμού διακοπής (Pod Disruption Budget)

Κατά την διάρκεια αναβάθμισης ή συντήρησης κάποιων κόμβων ή μικροϋπηρεσιών του συστήματος γίνεται τερματισμός των αντίστοιχων pods. Έτσι η μικροϋπηρεσία ενδέχεται να καταρρεύσει συνολικά. Για την αποφυγή αυτού του φαινομένου, είναι δυνατή η εφαρμογή κάποιων κανόνων που ονομάζονται πολιτικές προϋπολογισμού διακοπής, μέσω των οποίων ορίζεται είτε ο ελάχιστος αριθμός διαθέσιμων pods ανά deployment ή μέγιστος αριθμός μη διαθέσιμων. Με αυτό τον τρόπο το Kubernetes δεν επιτρέπει σε ενέργειες συντήρησης να εφαρμοστούν εάν παραβιάζεται το όριο αυτό. Για παράδειγμα, εάν ο διαχειριστής ήθελε να εφαρμόσει αυτή την πολιτική για το καλάθι αγορών θα έγραφε τον ορισμό της:

```
# 03-pdb-cartservice.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pdb-cartservice
  namespace: online-boutique
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: cartservice
```

και θα την εφαρμόζε με τις παρακάτω εντολές:

```
kubectl apply -f 03-pdb-cartservice.yaml
```

```
kubectl -n online-boutique get pdb
```

4. Ομαλός τερματισμός

Πολλές φορές ένα pod μπορεί να βρίσκεται κατά την διάρκεια επεξεργασίας ενός αιτήματος όταν το στέλνεται εντολή τερματισμού από τον χρονοπρογραμματιστή. Για την αποφυγή τερματισμού ενός pod πριν την ολοκλήρωση του αιτήματος, αρκεί ο ορισμός ενός κανόνα στον ορισμό του αντικειμένου ανάπτυξης:

```
# 05-graceful-frontend.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: online-boutique
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 30
      containers:
      - name: frontend
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "sleep 10"]
```

και η εκτέλεση των αντίστοιχων εντολών για την εφαρμογή των αλλαγών:

```
kubectl apply -f 05-graceful-frontend.yaml
```

```
kubectl -n online-boutique rollout status deploy/frontend
```

4.6.2. Διαχείριση των πόρων και δυναμική κλιμάκωση

1. Αιτήματα και όρια

Ορισμός κατάλληλων ορίων και αιτημάτων. Όταν δεν δίνονται μικρές οι τιμές σε αυτά τα χαρακτηριστικά των αντικειμένων ανάπτυξης μπορεί αρκετοί κόμβοι ή μικροϋπηρεσίες να μετατραπούν σε «θορυβώδεις γείτονες», ο οποίος καταναλώνει μεγάλο μέρος ή όλους τους κοινούς πόρους επηρεάζοντας τις υπόλοιπες. Αντίθετα, εάν οι τιμές είναι μεγάλες, μπορεί οι μικροϋπηρεσίες να περιοριστούν πολύ νωρίς, πριν υπάρξει πραγματική ανάγκη, αυξάνοντας το latency και κάνοντας χειρότερη την εμπειρία των χρηστών. Ο ορισμός τέτοιων τιμών γίνεται ως εξής:

2. Αυτόματη κλιμάκωση

Πολλές φορές το φορτίο ενός συστήματος δεν είναι σταθερό, αλλά παρουσιάζει σημαντικές διακυμάνσεις κατά την διάρκεια μίας μέρας ή εβδομάδας. Μέσω της αυτόματης κλιμάκωσης το Kubernetes διασφαλίζει την διαθεσιμότητα του συστήματος όταν το φορτίο αυξάνεται σημαντικά. Αυτό μπορεί να επιτευχθεί με δύο διαφορετικούς τρόπους:

- Την οριζόντια αυτόματη κλιμάκωση (Horizontal Pod Autoscaler, HPA), η οποία αποτελεί τον πιο απλό μηχανισμό και στην ουσία είναι ένα αντικείμενο το οποίο αυξάνει ή μειώνει δυναμικά τον αριθμό των

αντιγράφων ενός pod, λαμβάνοντας υπόψη κυρίως την χρήση των πόρων. Όταν ο αριθμός των απαιτούμενων διεργασιών αυξηθεί ο μηχανισμός οριζόντιας αυτόματης κλιμάκωσης δημιουργεί νέα pods για να γίνει διαμοιρασμός του φορτίου. Για παράδειγμα για την αντικείμενο ανάπτυξης της ιστοσελίδας η εφαρμογή γίνεται μέσω της παρακάτω εντολής:

```
kubectl autoscale deployment frontend --cpu-percent=50 --min=1 --max=10
```

- Την κατακόρυφη αυτόματης κλιμάκωση (Vertical Pod Autoscaler, VPA), η οποία δεν επηρεάζει τον αριθμό των αντιγράφων αλλά προσαρμόζει τους διαθέσιμους πόρους που επιτρέπεται να δεσμεύσουν τα ήδη υπάρχοντα pods. Αυτός ο μηχανισμός είναι χρήσιμος για υπηρεσίες που δεν έχουν την δυνατότητα κλιμάκωσης, αλλά επειδή δεν είναι ιδιαίτερα ασφαλής συνήθως δεν χρησιμοποιείται άμεσα. Όταν αυτός ο μηχανισμός εντοπίσει την ανάγκη προσαρμογής των διαθέσιμων πόρων, στέλνει ειδοποίηση στον διαχειριστή του συστήματος προτείνοντας τις καταλληλότερες τιμές. Εδώ ο ορισμός του μηχανισμού γίνεται μόνο με δηλωτικό τρόπο μέσω ενός αρχείου YAML:

```
# vpa-frontend.yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: frontend-vpa
  namespace: online-boutique
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: frontend
  updatePolicy:
    updateMode: "Off"
```

Και στην συνέχεια:

```
kubectl apply -f vpa-frontend.yaml
```

4.6.3. Ανθεκτικότητα δικτύου και μηχανισμοί ελέγχου συστήματος

1. Χρονικά όρια και επαναπροσπάθειες

Για την ολοκλήρωση ενός αιτήματος ενός χρήστη από το σύστημα συνήθως χρειάζεται επικοινωνία μεταξύ αρκετών μικροϋπηρεσιών, αυτός είναι ο κύριος λόγος της καθυστέρησης του συστήματος. Το μέγιστο χρονικό όριο ορίζει το μέγιστο χρονικό διάστημα κατά την διάρκεια του οποίου μία μικροϋπηρεσία περιμένει απάντηση σε ένα αίτημα της προς μία άλλη. Αν αυτό το χρονικό όριο οριστεί πολύ μεγάλο, το αίτημα θα μείνει σε κατάσταση αναμονής δεσμεύοντας υπολογιστικούς πόρους. Αυτό όταν γίνεται για μεγάλο αριθμό αιτημάτων μπορεί να οδηγήσει σε εξάντληση των πόρων και μη διαθεσιμότητα μίας μικροϋπηρεσίας.

Από την άλλη ένα αίτημα μπορεί να ακυρωθεί λόγω σφάλματος στο δίκτυο του συστήματος. Πολλά από αυτά τα σφάλματα είναι παροδικά, ακόμη και στιγμιαία. Ο μηχανισμός επαναπροσπαθειών επιτρέπει σε μία μικροϋπηρεσία να ξαναστέλλει αυτόματα το ίδιο αίτημα. Για την αποφυγή της υπερφόρτωσης του συστήματος ο χρόνος επαναποστολής αυξάνεται εκθετικά μετά από κάθε αποτυχία. Αυτό ονομάζεται εκθετική οπισθοχώρηση. Οι μηχανισμοί αυτοί μπορούν να οριστούν μέσω ενός αρχείου YAML:

```
# timeout-retries-productcatalog.yaml

apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: product-service-route
spec:
  hosts:
  - productcatalogservice
  http:
  - route:
    - destination:
        host: productcatalogservice
      timeout: 2s
      retries:
        attempts: 3
        perTryTimeout: 1s
        retryOn: 5xx,connect-failure,refused-stream
```

Παρατήρηση: Η συγκεκριμένη βελτίωση απαιτεί την εγκατάσταση ενός λογισμικού όπως το Istio.

2. Ανιχνευτές υγείας συστήματος

Το Kubernetes παρέχει στους διαχειριστές μηχανισμούς για την παρακολούθηση της κατάστασης των εφαρμογών και των μικροϋπηρεσιών, γνωστούς ως ανιχνευτές (probes). Η σωστή ρύθμιση των ανιχνευτών είναι ζωτικής σημασίας για την αυτοϊαση του συστήματος. Οι ανιχνευτές που θα βελτιώναν την απόκριση για τα πειράματα που αναφέρθηκαν στα προηγούμενα κεφάλαια είναι τρεις:

- Ανιχνευτής ζωτικότητας (liveness Probe): Ελέγχει αν μία μικροϋπηρεσία εκτελείται κανονικά. Εάν ο ανιχνευτής αποτύχει, το kubelet τερματίζει τον περιέκτη και τον επανεκκινεί, σε μία προσπάθεια να επαναφέρει την διαθεσιμότητα της υπηρεσίας.
- Ανιχνευτής ετοιμότητας (readiness Probe): Ελέγχει αν η μικροϋπηρεσία δέχεται δικτυακή κίνηση, καθώς μπορεί να εκτελείται αλλά να μην είναι έτοιμη. Αν αποτύχει ο έλεγχος, το Kubernetes σταματάει να δρομολογεί αιτήματα προς το συγκεκριμένο pods μέχρι την επαναφορά του.
- Ανιχνευτής εκκίνησης (startup probe): Ο συγκεκριμένος ανιχνευτής είναι αναγκαίος για τις εφαρμογές που έχουν μεγάλο χρόνο εκκίνησης, αποτρέποντας τον λανθασμένο τερματισμό του.

frontend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  template:
    spec:
      containers:
      - name: server
        image: frontend:v1

        livenessProbe:
          httpGet:
            path: /healthz
            port: 8080
          initialDelaySeconds: 3
          periodSeconds: 3

        readinessProbe:
          httpGet:
            path: /readiness
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5

        startupProbe:
          httpGet:
            path: /healthz
            port: 8080
          failureThreshold: 30
          periodSeconds: 10
```

3. Διακόπτης κυκλώματος

Ο διακόπτης κυκλώματος (circuit breaker) είναι ένας μηχανισμός που αποτρέπει τις επανειλημμένες προσπάθειες εκτέλεσης αιτημάτων σε μία υπηρεσία η οποία έχει κάποιο σφάλμα ή έχει κάποιο σημαντικό πρόβλημα απόδοσης. Όταν ο αριθμός των αποτυχημένων αιτημάτων ξεπεράσει ένα όριο ο διακόπτει

τίθεται σε ανοιχτή κατάσταση και όλα τα αιτήματα από εκείνη την χρονική στιγμή και μετά απορρίπτονται άμεσα (fail fast). Έτσι αποφεύγονται άσκοπες αναμονές και επιταχύνει την ανάκαμψη του συστήματος.

```
# productcatalog-cb.yaml

apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: product-service-cb
spec:
  host: productcatalogservice
  trafficPolicy:
    connectionPool:
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutive5xxErrors: 2
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
```

5. Εφαρμογή μεθοδολογίας μηχανικής χάους

Σε αυτό το κεφάλαιο θα εφαρμοστεί η μεθοδολογία μηχανικής χάους σε δύο ρεαλιστικά και πολυδιάστατα σενάρια. Θα οριστούν ροές εργασιών (workflows) οι οποίες εισάγουν πολλαπλές διαταραχές είτε σειριακά, είτε παράλληλα μεταξύ τους. Η δημιουργία και η εφαρμογή των ροών εργασίας είναι αρκετά πολύπλοκη στο εγγενές γραφικό περιβάλλον του Chaos Mesh, οπότε επιλέχθηκε η εφαρμογή των ροών μέσω της γραμμής εντολών.

5.1. Ήπια υποβάθμιση της υπηρεσίας ολοκλήρωσης αγορών

Περιγραφή πειράματος

Ουσιαστικά σε αυτό το πείραμα προσομοιώνεται υποβάθμιση του συστήματος, στοχεύοντας κυρίως την μικροϋπηρεσία ολοκλήρωσης αγορών, σε περίοδο αιχμής. Η εκτέλεση γίνεται σε δύο φάσεις για την αξιολόγηση του συστήματος σε δύο διαφορετικούς τύπους πίεσης:

1. Στην πρώτη φάση προσομοιώνεται αποτυχία σε δικτυακό επίπεδο και σε επίπεδο pods. Αυτό γίνεται μέσω προσθήκης μικρής καθυστέρησης, αύξηση του ρυθμού απώλειας πακέτων στην μικροϋπηρεσία της ιστοσελίδας.
2. Στην δεύτερη φάση του πειράματος εφαρμόζεται πίεση στον υπολογιστικούς πόρους στις μικροϋπηρεσίες της ιστοσελίδας και του ολοκλήρωσης αγορών.

Σταθερή Κατάσταση

- Διαθεσιμότητα > 99%
- Καθυστέρηση p99 στην μικροϋπηρεσία της ιστοσελίδας < 20ms (μέση τιμή)
- Κατάσταση όλων των pod: «Εκτελούνται»

Υπόθεση

Σε συνθήκες ήπιας φόρτισης, με παρατηρήσιμη καθυστέρηση στην ιστοσελίδα, το σύστημα θα πρέπει να παραμένει πλήρως διαθέσιμο και να ανακάμπτει άμεσα. Ενώ όταν μία δευτερεύουσα υπηρεσία δεν είναι διαθέσιμη, δεν θα έπρεπε να επηρεάζονται οι βασικές και σημαντικότερες μικροϋπηρεσίες του συστήματος.

Συγκεκριμένα:

1. Αν προστεθεί καθυστέρηση δικτύου και φορτίο στο CPU, ο χρόνος απόκρισης αναμένεται να αυξηθεί παροδικά, χωρίς όμως να επιστραφούν σφάλματα HTTP 500 σε χρήστες. Αυτή η αύξηση θα πρέπει να μετριαστεί σύντομα από το σύστημα.
2. Στην περίπτωση που η μικροϋπηρεσία των διαφημίσεων αποτύχει να εκτελεστεί, η σελίδα θα πρέπει να φορτώσει επιτυχώς.

Αρχείο ορισμού ροής εργασιών

```
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: easy-chaos-test
  namespace: chaos-mesh
spec:
  entry: main
  templates:
    - name: main
      templateType: Serial
      deadline: 35m
      children:
        - warmup
        - network-failure-phase
        - recovery-intermediate
        - cpu-stress-phase
        - recovery-final

    # 1. Warmup
    - name: warmup
      templateType: Suspend
      deadline: 5m

    # -----
    # ΦΑΣΗ A: Network & Pod Failure
    # -----
    - name: network-failure-phase
      templateType: Parallel
      deadline: 7m
      children:
        - mild-network-delay-frontend
        - mild-network-loss-frontend
        - mild-pod-failure-adservice

    # Chaos Definitions Φάση A
    - name: mild-network-delay-frontend
      templateType: NetworkChaos
      deadline: 7m
      networkChaos:
        action: delay
        mode: all
        direction: to
        selector:
          namespaces:
            - online-boutique
          labelSelectors:
            app: frontend
        delay:
          latency: "150ms"
          correlation: "25"
          jitter: "50ms"

    - name: mild-network-loss-frontend
      templateType: NetworkChaos
      deadline: 7m
      networkChaos:
        action: loss
        mode: all
        direction: to
        selector:
          namespaces:
            - online-boutique
          labelSelectors:
            app: frontend
        loss:
          loss: "1"
          correlation: "0"

    - name: mild-pod-failure-adservice
      templateType: PodChaos
      deadline: 7m
      podChaos:
        action: pod-failure
        mode: one
        selector:
          namespaces:
            - online-boutique
          labelSelectors:
            app: adservice

    # 2. Ενδιάμεση Ανάκαμψη
    - name: recovery-intermediate
      templateType: Suspend
      deadline: 5m

    # -----
    # ΦΑΣΗ B: CPU Stress
    # -----
    - name: cpu-stress-phase
      templateType: Parallel
      deadline: 7m
      children:
        - mild-cpu-stress-frontend
        - mild-cpu-stress-checkout

    # Chaos Definitions για Φάση B
    - name: mild-cpu-stress-frontend
      templateType: StressChaos
      deadline: 7m
      stressChaos:
        mode: all
        selector:
          namespaces:
            - online-boutique
          labelSelectors:
            app: frontend
        stressors:
          cpu:
            workers: 2
            load: 70

    - name: mild-cpu-stress-checkout
      templateType: StressChaos
      deadline: 7m
      stressChaos:
        mode: all
        selector:
          namespaces:
            - online-boutique
          labelSelectors:
            app: checkoutservice
        stressors:
          cpu:
            workers: 2
            load: 70

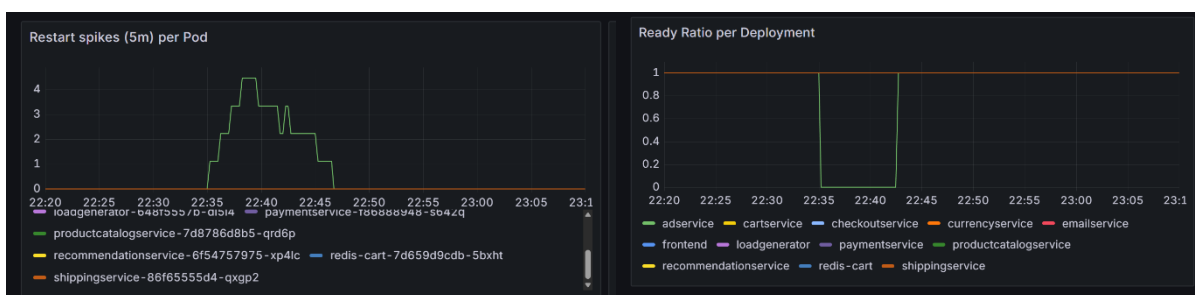
    # 3. Τελική Ανάκαμψη
    - name: recovery-final
      templateType: Suspend
      deadline: 5m
```

Εκτέλεση πειράματος

Η ροή του πειράματος έχει ως εξής:

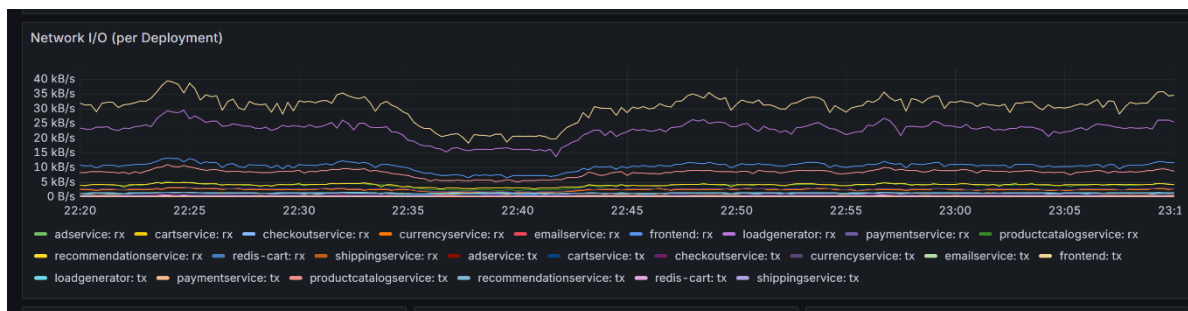
1. **Προετοιμασίας (5 λεπτά):** Κανονική λειτουργία. Σε αυτή τη φάση γίνεται μέτρηση των τιμών βάσης για κάθε διάγραμμα που αφορά το πείραμα.
2. **Διαταραχή (7 λεπτά):** Ταυτόχρονη εφαρμογή όλων των παραπάνω σφαλμάτων (Πίεση υπολογιστικών πόρων, Δικτυακή καθυστέρηση, Αποτυχία pod).
3. **Ανάκαμψη (5 λεπτά):** Τερματισμός πειράματος και επαναφορά συστήματος στην σταθερή κατάσταση.

Επαλήθευση Αποτελεσμάτων



Εικόνα 60. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.

Το πείραμα εισάγει ένα σταθερό σφάλμα στο pod της υπηρεσίας διαφημίσεων. Αυτό δεν καταφέρνει να επανέλθει μέχρι να τελειώσει η διάρκεια του συγκεκριμένου σφάλματος. Για αυτόν το λόγο στα δεξιά, υπάρχει αυτή η πτώση στην αναλογία ετοιμότητας της υπηρεσίας.



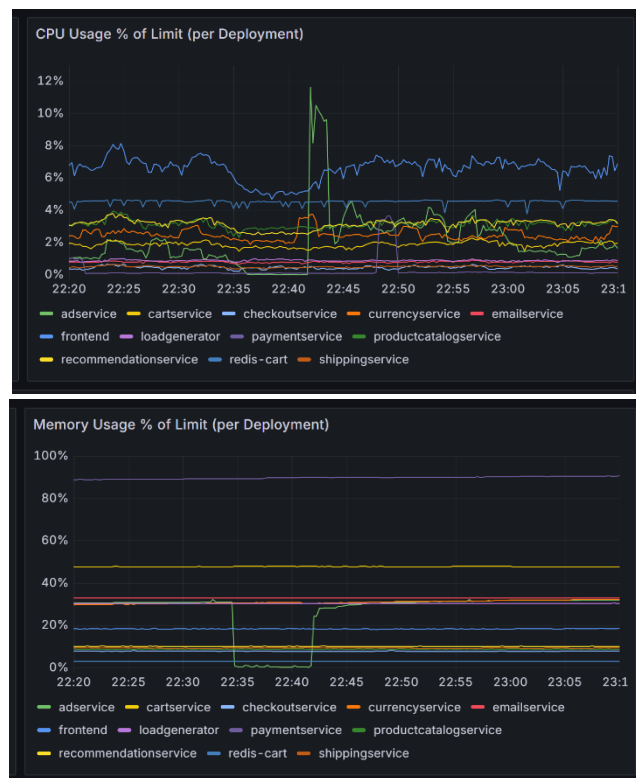
Εικόνα 61. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.

Στην Εικόνα 61 είναι παρατηρήσιμη η μείωση της μετάδοσης για όλες της μικροϋπηρεσίες και ειδικά για αυτές που έχουν υψηλότερο, όπως ο εξερχόμενος της ιστοσελίδας.



Εικόνα 62. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.

Στην Εικόνα 62 είναι φανερό πως το σύστημα σε όλη την διάρκεια του πειράματος είναι πλήρως διαθέσιμο, όμως η διάρκεια ολοκλήρωσης των αιτημάτων των χρηστών προς την ιστοσελίδα του συστήματος είναι πολύ μεγάλη. Ακόμα και όταν μετριάζεται η καθυστέρηση είναι αρκετή για να κάνει πολύ δύσκολη την εμπειρία των χρηστών.



Εικόνα 63. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.

Στην Εικόνα 63 παρατηρείται μείωση στην χρήση των υπολογιστικών πόρων κατά την διάρκεια της ροής εργασιών. Αυτό είναι στα επιτρεπτά όρια και αναμενόμενο, καθώς η μείωση ροής πακέτων λόγω της καθυστέρησης και της απώλειας πακέτων μειώνει και τον ρυθμό διεκπεραίωσης διεργασιών. Η κορύφωση που φαίνεται στο αριστερό διάγραμμα αφορά την διαδικασία επανεκκίνησης της υπηρεσίας διαφημίσεων. Το ίδιο ισχύει και για τον μηδενισμό στην χρήση μνήμης στο δεξί διάγραμμα.

Πίνακας 15. Αναλυτικές μεταβολές των μετρικών κατά το πρώτο σενάριο μηχανικής χάους.

	Σταθερή Κατάσταση			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Αύξηση επανεκκινήσεων υπηρεσίας διαφημίσεων	0	0	0	2.61	1.11	4.44
Αναλογία ετοιμότητας pods υπηρεσίας διαφημίσεων	100%	100%	100%	0% (-100%)	0%	0%
Εισερχόμενη Διαμεταγωγή	45.36 kB/s	15.7 kB/s	64.5 kB/s	31.03 kB/s (-31.59%)	12.1 kB/s	52.2 kB/s
Εξερχόμενη Διαμεταγωγή	44.09 kB/s	12.6 kB/s	66.4 kB/s	31.83 kB/s (-27.81%)	6.84 kB/s	56.1 kB/s
Εισερχόμενος ρυθμός δεδομένων ιστοσελίδας	11.15 kB/s	9.55 kB/s	13.3 kB/s	7.57 kB/s (-32.11%)	6.5 kB/s	9.05 kB/s
Εξερχόμενος ρυθμός δεδομένων ιστοσελίδας	32.85 kB/s	28.2 kB/s	39.5 kB/s	21.47 kB/s (-34.64%)	18.3 kB/s	25.6 kB/s
Καθυστέρηση p99	16.93 ms	14.8 ms	19.4 ms	3151.54 ms (+18515.12%)	2350 ms	4740 ms
Ποσοστό χρήσης διαθέσιμης υπολογιστικής ισχύος της ιστοσελίδας (%)	6.98%	6.14%	8.13%	5.24% (-24.93%)	4.67%	6.03%
Ποσοστό χρήσης διαθέσιμης μνήμης της υπηρεσίας διαφημίσεων (%)	30.86%	30.7%	32.2%	0.48% (-98.44%)	0.20%	1.24%

Τα αποτελέσματα του πειράματος, σύμφωνα και με τον Πίνακα 15, δείχνουν σημαντική υποβάθμιση του συστήματος στην φάση της δικτυακής πίεσης και των pods. Κατά την δεύτερη φάση, της πίεσης των πόρων, δεν παρατηρείται ιδιαίτερη υποβάθμιση του συστήματος, ούτε πίεση του.

5.2. Επιθετική υποβάθμιση της υπηρεσίας ολοκλήρωσης αγορών

Περιγραφή πειράματος

Σε αυτό το σενάριο προσομοιώνεται, σε πρώτη φάση, μεγάλη υποβάθμιση του συστήματος μέσω πρόκλησης καταπονήσεων σε δικτυακό επίπεδο και στην διαθεσιμότητα απαραίτητων υπηρεσιών και, σε δεύτερη φάση, εξάντληση μεγάλου μέρους των διαθέσιμων πόρων. Πιο αναλυτικά οι δύο φάσεις που ορίζονται είναι:

1. Χάος σε επίπεδο δικτύου και pods: Αύξηση καθυστέρησης κατά 1 δευτερόλεπτο και του ποσοστού απώλειας πακέτων σε 7% για την μικροϋπηρεσία της ιστοσελίδας. Ταυτόχρονα τερματίζεται στιγμιαία το pod του καλαθιού αγορών.
2. Καταπόνηση πόρων: Εξάντληση των υπολογιστικών πόρων και της μνήμης στην μικροϋπηρεσία ολοκλήρωσης αγορών και υπολογιστικής μνήμης της ιστοσελίδας.

Σταθερή Κατάσταση

Ομοίως με το προηγούμενο πείραμα:

- Διαθεσιμότητα > 99%
- Καθυστέρηση p99 στην μικροϋπηρεσία της ιστοσελίδας < 20ms (μέση τιμή)
- Κατάσταση όλων των pod: «Εκτελούνται»

Υπόθεση

Κατά την πρώτη φάση αναμένεται να καταστήσει το σύστημα μη διαθέσιμο για τους χρήστες λόγω της προσθήκης μεγάλης καθυστέρησης (1 δευτερόλεπτο) και της αυξημένης απώλειας πακέτων, που θα προκαλέσουν λήξεις χρονικών ορίων. Στην δεύτερη φάση, η εξάντληση των πόρων αναμένεται να οδηγήσει σε έντονο περιορισμό των υπολογιστικών πόρων

Αρχείο ορισμού ροής εργασιών

```
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: ob-aggressive-checkout
  namespace: chaos-mesh
spec:
  entry: main
  templates:
    - name: main
      templateType: Serial
      deadline: 60m
      children:
        - warmup
        - aggressive-network-failure-phase
        - recovery-intermediate
        - aggressive-resource-stress-phase
        - recovery-final

# 1. Warmup
- name: warmup
  templateType: Suspend
  deadline: 10m

# -----
# ΦΑΣΗ A: Network Chaos & Pod Kill
# -----
- name: aggressive-network-failure-phase
  templateType: Parallel
  deadline: 10m
  children:
    - aggressive-network-delay-frontend
    - aggressive-network-loss-frontend
    - aggressive-pod-kill-cart

- name: aggressive-network-delay-frontend
  templateType: NetworkChaos
  deadline: 10m
  networkChaos:
    action: delay
    mode: all
    direction: to
    selector:
      namespaces:
        - online-boutique
      labelSelectors:
        app: frontend
    delay:
      latency: "1s"
      correlation: "50"
      jitter: "300ms"

- name: aggressive-network-loss-frontend
  templateType: NetworkChaos
  deadline: 10m
  networkChaos:
    action: loss
    mode: all
    direction: to
    selector:
      namespaces:
        - online-boutique
      labelSelectors:
        app: frontend
    loss:
      loss: "7"
      correlation: "25"

- name: aggressive-pod-kill-cart
  templateType: PodChaos
  deadline: 3m

podChaos:
  action: pod-kill
  mode: one
  selector:
    namespaces:
      - online-boutique
    labelSelectors:
      app: cartservice

# 2. Ενδιάμεση Ανάκαμψη
- name: recovery-intermediate
  templateType: Suspend
  deadline: 10m

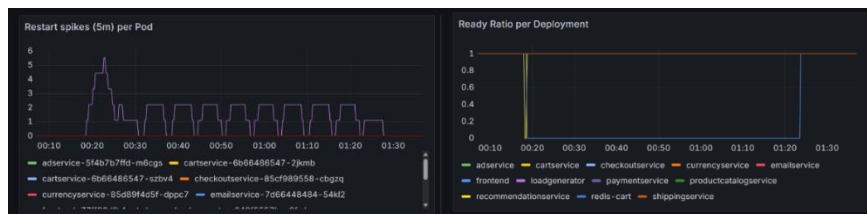
# -----
# ΦΑΣΗ B: CPU & Memory Stress
# -----
- name: aggressive-resource-stress-phase
  templateType: Parallel
  deadline: 10m
  children:
    - aggressive-cpu-mem-checkout
    - aggressive-cpu-stress-frontend

- name: aggressive-cpu-mem-checkout
  templateType: StressChaos
  deadline: 10m
  stressChaos:
    mode: all
    selector:
      namespaces:
        - online-boutique
      labelSelectors:
        app: checkoutservice
    stressors:
      cpu:
        workers: 2
        load: 100
      memory:
        workers: 1
        size: "90MB"

- name: aggressive-cpu-stress-frontend
  templateType: StressChaos
  deadline: 10m
  stressChaos:
    mode: all
    selector:
      namespaces:
        - online-boutique
      labelSelectors:
        app: frontend
    stressors:
      cpu:
        workers: 2
        load: 100

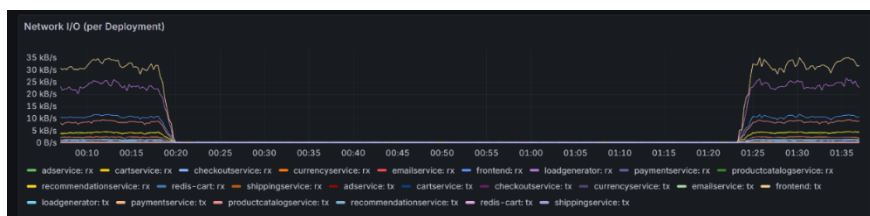
# 3. Τελική Ανάκαμψη
- name: recovery-final
  templateType: Suspend
  deadline: 10m
```

Επαλήθευση Αποτελεσμάτων



Εικόνα 64. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.

Στο αριστερό διάγραμμα της Εικόνας 64 παρατηρείται αύξηση των επανεκκινήσεων στο pod της υπηρεσίας της ιστοσελίδας για όλη την διάρκεια της ροής εργασιών, ενώ στο δεξί διάγραμμα είναι φανερός ο τερματισμός του pod του καλαθιού, το οποίο επανέρχεται άμεσα. Αντίθετα, αυτό της ιστοσελίδας καθίσταται ανέτοιμο και παραμένει έτσι.



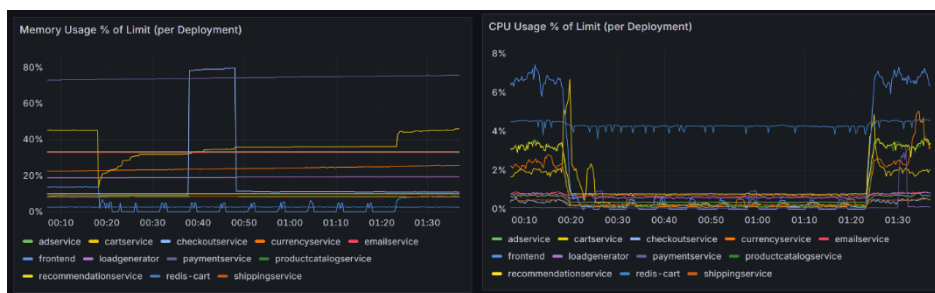
Εικόνα 65. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.

Στην Εικόνα 66 ο ρυθμός μετάδοσης πακέτων όλων των υπηρεσιών φαίνεται να έχει σχεδόν μηδενιστεί, δηλαδή η υπηρεσία φαίνεται να είναι μη διαθέσιμη προς όλους τους χρήστες. Στην Εικόνα 66 αυτό επιβεβαιώνεται από το διάγραμμα διαθεσιμότητας (αριστερά), όπου η διαθεσιμότητα είναι στο 0% σταθερά για όλη την διάρκεια της ροής εργασιών. Αυτό πιθανώς οφείλεται στην αύξηση καθυστέρησης στην αρχή του σεναρίου.



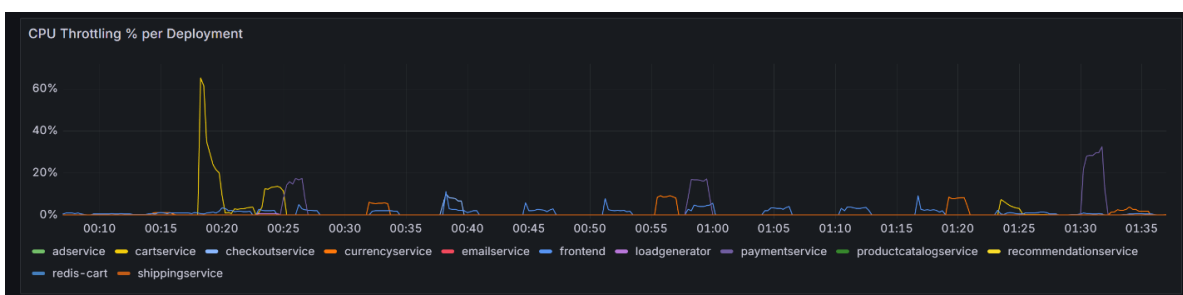
Εικόνα 66. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.

Από τα διαγράμματα καθυστέρησης επιβεβαιώνουμε ότι στιγμιαία η τιμή της καθυστέρησης έχει μία μεγάλη κορύφωση στα 5 δευτερόλεπτα, αλλά στην συνέχεια μειώνεται. Η μείωση όμως δεν είναι σε τιμές σταθερής κατάστασης, αλλά μηδενισμός της καθυστέρησης όπως παρουσιάζεται στον Πίνακα 15. Αυτό σημαίνει πως το σύστημα από την πλευρά των χρηστών μοιάζει να είναι εντελώς κλειστό.



Εικόνα 67. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.

Από το διάγραμμα χρήσης της μνήμης (αριστερά) γίνεται αντιληπτό ότι η απώλεια διαθεσιμότητας ισχύει μόνο για την μικροϋπηρεσία της ιστοσελίδας, της οποίας η κατανάλωση της διαθέσιμης μνήμης μηδενίζεται, ενώ όλων των υπολοίπων παραμένει σε σταθερές τιμές. Εξαιρέσεις στην προηγούμενη παρατήρηση αποτελούν οι υπηρεσίες του καλαθιού, η οποία τερματίζεται στιγμιαία στην αρχή του πειράματος και η μικροϋπηρεσία ολοκλήρωσης αγορών στην οποία εφαρμόζεται καταπόνηση των πόρων, εξού και η αύξηση κατανάλωσης της διαθέσιμης μνήμης. Στο δεξί διάγραμμα παρατηρούμε μείωση κατανάλωσης της διαθέσιμης υπολογιστικής ισχύος για όλες τις μικροϋπηρεσίες, οι οποίες είναι προσβάσιμες στους χρήστες έμμεσα και μόνο μέσω της ιστοσελίδας.



Εικόνα 68. Διάγραμμα περιορισμού υπολογιστικής ισχύος ανά μικροϋπηρεσία.

Στο διάγραμμα περιορισμού της υπολογιστικής ισχύος, βλέπουμε περιορισμό για την υπηρεσία του καλαθιού κατά την διάρκεια επανεκκίνησης του και αύξηση στον αριθμό των περιορισμών σε αρκετές μικροϋπηρεσίες κατά την διάρκεια της καταπόνησης των πόρων.

Πίνακας 16. Αναλυτικές μεταβολές των μετρικών κατά το δεύτερο σενάριο μηχανικής χάους.

	Σταθερή Κατάσταση			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Αύξηση των επανεκκινήσεων της ιστοσελίδας	0	0	0	1.67	0	5.56
Αναλογία έτοιμων pods του καλαθιού αγορών (%)	100%	100%	100%	99% (-1%)	0%	100%
Αναλογία έτοιμων pods του ιστοσελίδας (%)	100%	100%	100%	0% (-100%)	0%	0%
Εισερχόμενη διαμεταγωγή	41.94 kB/s	18.6 kB/s	62.6 kB/s	2.57 kB/s (-93.87%)	1.35 kB/s	3.63 kB/s
Εξερχόμενη διαμεταγωγή	43.61 kB/s	13.7 kB/s	57.6 kB/s	6.17 kB/s (-85.85%)	0.99 kB/s	2.89 kB/s
Εισερχόμενος ρυθμός δεδομένων ιστοσελίδας (kB/s)	10.88 kB/s	9.92 kB/s	11.7 kB/s	0.06 kB/s (-99.45%)	0 kB/s	0.29 kB/s
Εξερχόμενος ρυθμός δεδομένων ιστοσελίδας (kB/s)	31.98 kB/s	28.4 kB/s	34.9 kB/s	0.06 kB/s (-99.81%)	0 kB/s	0.77 kB/s
Καθυστέρηση p99	15.99 ms	13.5 ms	18.8 ms	0.75 ms * (-95.31%)	0.458 ms *	2.7 ms *
Περιορισμός επεξεργαστικής ισχύος της ιστοσελίδας (%)	0.52 %	0 %	1.04 %	1.14 % (+119.23%)	0 %	11.1 %
Χρήση διαθέσιμης υπολογιστικής ισχύος από την ιστοσελίδα	6.7 %	5.83 %	7.45 %	0.14 % (-97.91%)	0 %	0.78 %
Χρήση διαθέσιμης μνήμης από την ιστοσελίδα	14 %	13.7 %	14.2 %	0.98 % (-93.00%)	0.21 %	6.61 %
Ποσοστό αξιοποίησης της υπολογιστικής ισχύος του	15.21 %	13.1 %	16.9 %	13.15 % (-13.54%)	12.2 %	14.2 %

κόμβου εκτέλεσης 5 (%)						
Ποσοστό αξιοποίησης της υπολογιστικής ισχύος του κόμβου εκτέλεσης 6 (%)	23.95 %	21 %	29.1 %	22.17 % (-7.43%)	20.3 %	27.3 %

Στον Πίνακα 16 στην μετρική της καθυστέρησης p99 αφαιρέθηκαν 4 μετρήσεις (~1 λεπτό) στις οποίες η τιμή της καθυστέρησης ήταν 4950ms. Η τιμή 0.75ms είναι υπερβολικά μικρή και δεν αποτελεί ένδειξη ολοκλήρωσης αιτήματος, αλλά ένδειξη ότι το η υπηρεσία είναι μη προσβάσιμη. Η μείωση στην αναλογία ετοιμότητας της υπηρεσίας καλαθιού αγορών κατά 1% υποδεικνύει την γρήγορη επαναφορά της μικροϋπηρεσίας από τον ξαφνικό τερματισμό της.

5.3. Εφαρμογή Βελτιώσεων Ανθεκτικότητας

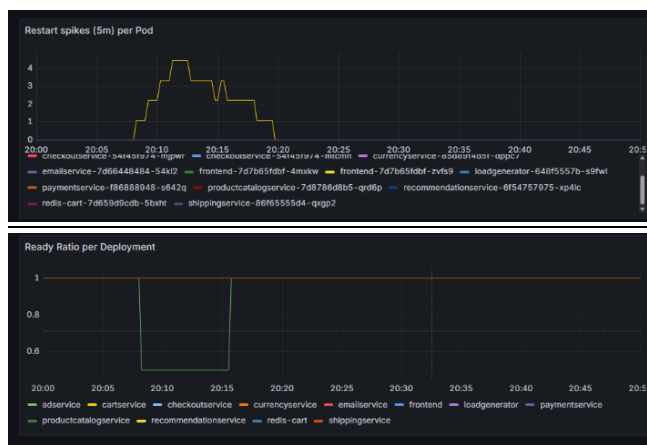
Βάσει των μετρήσεων των προηγούμενων σεναρίων, και ιδιαίτερα του πιο επιθετικού, παρατηρούνται φαινόμενα αστοχίας μεμονωμένων pods, καθυστερήσεις στην κυκλοφορία πακέτων στο δίκτυο και πίεση πόρων του συστήματος. Εφαρμόστηκαν βελτιώσεις συγκεκριμένα στις μικροϋπηρεσίες που παρατηρήθηκαν τα σημαντικότερα προβλήματα (ιστοσελίδας, ολοκλήρωσης αγορών, καλαθιού αγορών και διαφημίσεων). Οι βελτιώσεις που εφαρμόστηκαν έχουν ως στόχο την επίτευξη υψηλής διαθεσιμότητας του συστήματος και είναι:

- Αύξηση του αριθμού των αντιγράφων σε 2.
- Ορισμός αυτόματης οριζόντιας κλιμάκωσης με μέγιστο αριθμό αντιγράφων τα 5 ή 6, ανάλογα την μικροϋπηρεσία, σε περίπτωση αύξησης κατανάλωσης των πόρων.
- Ορισμός πολιτικής προϋπολογισμού διακοπής, η οποία θα εξασφαλίζει ότι κατά την διάρκεια οποιασδήποτε εργασίας συντήρησης ή ανακατανομής των pods του συστήματος, θα υπάρχει τουλάχιστον 1 ενεργό pod ανά μικροϋπηρεσία.
- Ορισμός πολιτικής σωστής κατανομής των pods, με σκοπό την αποφυγή ανάθεσης pods μίας μικροϋπηρεσίας στον ίδιο κόμβο από τον scheduler.
- Μείωση των τιμών requests ώστε να δεσμεύουν μόνο τους απαραίτητους πόρους για την εκκίνηση ενός pod σε κάποιον κόμβο εκτέλεσης.
- Αύξηση των ορίων χρήσης πόρων ώστε να αποφεύγεται ο περιορισμός τους από τους πυρήνες των κόμβων εκτέλεσης και τα γεγονότα εξάντλησης μνήμης σε περιπτώσεις που υπάρχει ακόμα διαθέσιμη μνήμη στο σύστημα.
- Μείωση του χρονικού ορίου των ανιχνευτών ζωτικότητας και ετοιμότητας για τον γρηγορότερο εντοπισμό προβληματικών pods, ώστε να απομονώνονται.

Αφού έγινε η εφαρμογή όλων των παραπάνω μέτρων βελτίωσης, το επόμενο βήμα σύμφωνα με την μεθοδολογία της μηχανικής χάους είναι η επανεκτέλεση των πειραμάτων.

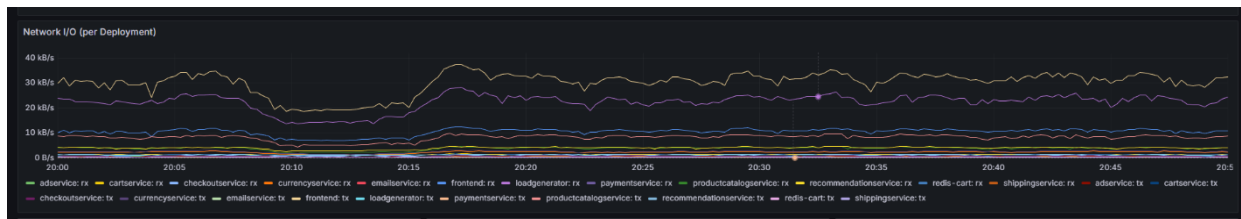
5.4. Επανεκτέλεση ήπιου ολοκληρωμένου σεναρίου

Έχοντας ορίσει την σταθερή κατάσταση κατά την πρώτη εκτέλεση και έχοντας τα αποτελέσματα της σε αυτό το κεφάλαιο αρκεί η παρατήρηση των αποτελεσμάτων της επανεκτέλεσης της ροής εργασιών.



Εικόνα 69. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.

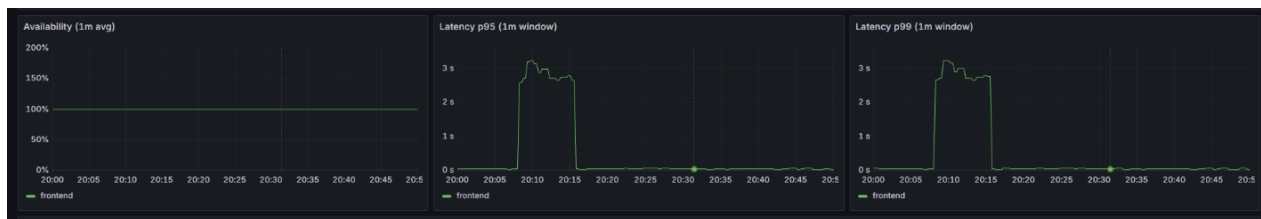
Όπως ήταν αναμενόμενο, ο αριθμός επανεκκινήσεων στα pod της μικροϋπηρεσίας παραμένει ίδιος με αυτόν της πρώτης εκτέλεσης αφού η υπηρεσία κάνει συνεχόμενη εφαρμογή σφάλματος στο pod για το χρονικό διάστημα που ορίστηκε. Η αύξηση του αριθμού των αντιγράφων που εφαρμόστηκε διατήρησε την υπηρεσία ενεργή με την αναλογία ετοιμότητας να μειώνεται στο 50% αντί για το 0%.



Εικόνα 70. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.

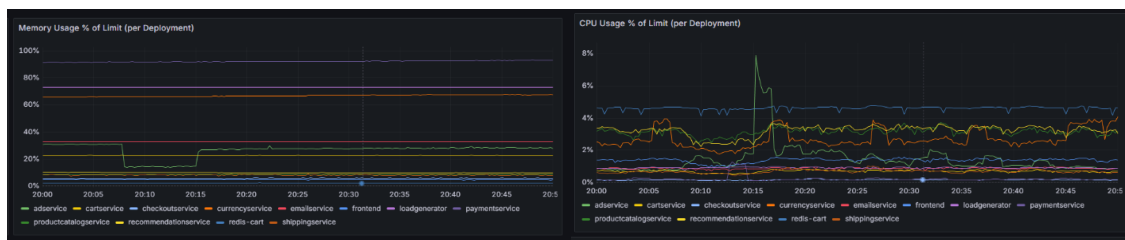
Όσον αφορά τον ρυθμό μετάδοσης, δεν παρατηρείται κάποια βελτίωση στην ταχύτητα μετάδοσης κατά την διάρκεια της ροής εργασιών. Αυτό δεν είναι ιδιαίτερα προβληματικό, καθώς η μείωση του ρυθμού είναι

αναμενόμενη σε περίπτωση προβλημάτων του συστήματος. Πραγματικό πρόβλημα θα υπήρχε εάν αυτή η μείωση ήταν τέτοια που το σύστημα θα ήταν πρακτικά αδύνατο να χρησιμοποιηθεί.



Εικόνα 71. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.

Σύμφωνα με την Εικόνα 71 η διαθεσιμότητα της ιστοσελίδας παραμένει στο 100%, ενώ όσον αφορά την αύξηση της καθυστέρησης, μετά την εφαρμογή των βελτιώσεων, η κορύφωση στα 4.7 δευτερόλεπτα μετριάστηκε πλήρως διατηρώντας την καθυστέρηση στα 3 δευτερόλεπτα περίπου.



Εικόνα 72. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.

Στην χρήση των διαθέσιμων υπολογιστικών πόρων δεν παρατηρούμε κάποια διαφορά, καθώς στην πρώτη εκτέλεση δεν υπάρχει κάποια υπηρεσία, η οποία να παρουσιάζει πρόβλημα το οποίο να αναμένεται να βελτιωθεί κατά την επανεκτέλεση της ροής εργασιών.

Πίνακας 17. Αναλυτικές μεταβολές των μετρικών κατά την επανεκτέλεση του ήπιου σεναρίου μηχανικής χάους.

	Σταθερή Κατάσταση			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Αύξηση επανεκκινήσεων υπηρεσίας διαφημίσεων	0	0	0	2.61	1.11	4.44
Αναλογία ετοιμότητας	1	1	1	0.5 (-50%)	0.5	0.5

prods υπηρεσίας διαφημίσεων						
Εισερχόμενη Διαμεταγωγή	44.86 kB/s	35.9 kB/s	56.7 kB/s	28.38 kB/s (-36.74%)	17.6 kB/s	35.4 kB/s
Εξερχόμενη Διαμεταγωγή	41.34 kB/s	16.6 kB/s	53.3 kB/s	26.49 kB/s (-35.92%)	7.31 kB/s	37.6 kB/s
Εισερχόμενος ρυθμός δεδομένων ιστοσελίδας	10.58 kB/s	8.53 kB/s	11.8 kB/s	7.37 kB/s (-30.34%)	7.04 kB/s	8.11 kB/s
Εξερχόμενος ρυθμός δεδομένων ιστοσελίδας	30.38 kB/s	24.1 kB/s	34.2 kB/s	19.55 kB/s (-35.64%)	18.5 kB/s	21.7 kB/s
Καθυστέρηση p99	41.63 ms	34.2 ms	49.4 ms	2938.42 ms (+6958.42%)	2650 ms	3230 ms
Ποσοστό χρήσης διαθέσιμης υπολογιστικής ισχύος της ιστοσελίδας (%)	1.40%	1.26%	1.49%	1.07% (-23.57%)	0.96%	1.16%
Ποσοστό χρήσης διαθέσιμης μνήμης της υπηρεσίας διαφημίσεων (%)	30.93%	30.7%	31.1%	14.29% (-53.80%)	14%	14.7%

Μελετώντας τόσο τον Πίνακα 16, όσο και τον Πίνακα 17 που ακολουθεί, παρατηρούμε ότι οι τιμές που αφορούν τον ρυθμό μετάδοσης δεδομένων, ανά υπηρεσία και συνολικά, αλλάζουν ελαφρώς προς το χειρότερο. Αυτό δεν χρειάζεται να προβληματίσει τον διαχειριστή, αν δοθεί σημασία στο γεγονός ότι η καθυστέρηση ολοκλήρωσης αιτημάτων προς την υπηρεσία της ιστοσελίδας μειώνεται κατά 0.2 δευτερόλεπτα, σημαντικό ποσό για την συγκεκριμένη μετρική.

Σύγκριση αρχικής και τελευταίας εκτέλεσης

Πίνακας 18. Σύγκριση μετρικών κατά την διάρκεια της ήπιας ροής εργασιών πριν και μετά τις βελτιώσεις.

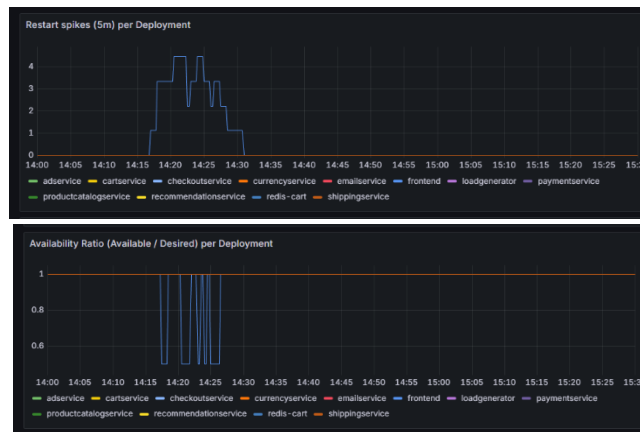
	Αρχικό Πείραμα			Νέο Πείραμα		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Αύξηση επανεκκινήσεων	2.61	1.11	4.44	2.61	1.11	4.44

υπηρεσίας διαφημίσεων						
Αναλογία ετοιμότητας pods υπηρεσίας διαφημίσεων	0% (-100%)	0%	0%	0.5 (-50%)	0.5	0.5
Εισερχόμενη Διαμεταγωγή	31.03 kB/s (-31.59%)	12.1 kB/s	52.2 kB/s	28.38 kB/s (-36.74%)	17.6 kB/s	35.4 kB/s
Εξερχόμενη Διαμεταγωγή	31.83 kB/s (-27.81%)	6.84 kB/s	56.1 kB/s	26.49 kB/s (-35.92%)	7.31 kB/s	37.6 kB/s
Εισερχόμενος ρυθμός δεδομένων ιστοσελίδας	7.57 kB/s (-32.11%)	6.5 kB/s	9.05 kB/s	7.37 kB/s (-30.34%)	7.04 kB/s	8.11 kB/s
Εξερχόμενος ρυθμός δεδομένων ιστοσελίδας	21.47 kB/s (-34.64%)	18.3 kB/s	25.6 kB/s	19.55 kB/s (-35.64%)	18.5 kB/s	21.7 kB/s
Καθυστέρηση p99	3151.54 ms (+18515.12%)	2350 ms	4740 ms	2938.42 ms (+6958.42%)	2650 ms	3230 ms
Ποσοστό χρήσης διαθέσιμης υπολογιστικής ισχύος της ιστοσελίδας (%)	5.24% (-24.93%)	4.67%	6.03%	1.07% (-23.57%)	0.96%	1.16%
Ποσοστό χρήσης διαθέσιμης μνήμης της υπηρεσίας διαφημίσεων (%)	0.48% (-98.44%)	0.20%	1.24%	14.29% (-53.80%)	14%	14.7%

Συγκρίνοντας τις επιπτώσεις στο σύστημα κατά τις δύο εκτελέσεις του ήπιου ολοκληρωμένου σεναρίου παρατηρούμε ότι σε απόλυτους αριθμούς η υποβάθμιση στον ρυθμό μετάδοσης δεδομένων είναι πολύ μικρή μετά την εφαρμογή των βελτιώσεων. Ποσοστιαία η υποβάθμιση φαίνεται μεγαλύτερη καθώς όταν δεν υπάρχει συμφόρηση, το γεγονός ότι πλέον υπάρχουν περισσότερα pods σημαίνει ότι να εξυπηρετηθούν παράλληλα περισσότεροι χρήστες. Αντίθετα μετά την εφαρμογή του συστήματος η κατανομή των διεργασιών είναι καλύτερη στα pods οπότε η συνολική κατανάλωση υπολογιστικών πόρων μειώνεται κατά την συμφόρηση από το αρχικό στο νέο σύστημα. Ακόμα παρατηρείται ότι η χρήση της διαθέσιμης μνήμης από την υπηρεσία διαφημίσεων δεν μηδενίζεται οπότε, όπως φαίνεται και από την τιμή της αναλογίας διαθεσιμότητας, η μικροϋπηρεσία παραμένει λειτουργική καθώς υπάρχει τουλάχιστον ένα pod για την εξυπηρέτηση των αιτημάτων.

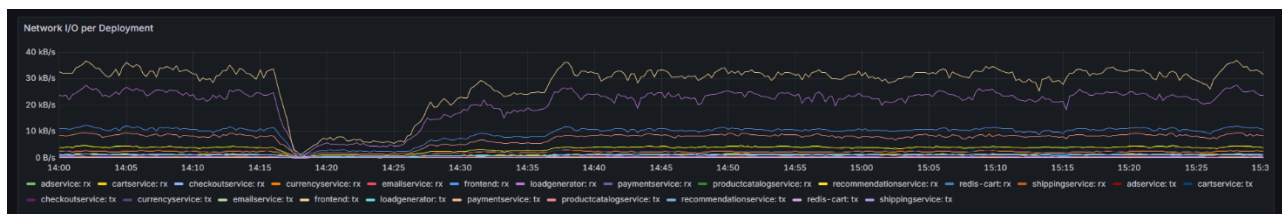
5.5. Επανεκτέλεση επιθετικού ολοκληρωμένου σεναρίου

Ομοίως με την επανεκτέλεση του ήπιου ολοκληρωμένου σεναρίου εδώ θα παρουσιαστούν απλά τα αποτελέσματα της επανεκτέλεσης μετά την εφαρμογή των μέτρων βελτίωσης της ανθεκτικότητας του συστήματος.



Εικόνα 73. Αύξηση επανεκκινήσεων και αναλογία ετοιμότητας ανά μικροϋπηρεσία.

Από το πρώτο κιάλας διάγραμμα της Εικόνας 73 εντοπίζεται σημαντική βελτίωση στο σύστημα καθώς η αύξηση των επανεκκινήσεων περιορίζεται μόνο στην αρχή του επιθετικού σεναρίου. Στην πρώτη εκτέλεση η αύξηση στον αριθμό των επανεκκινήσεων παρατηρείται σε ολόκληρη την διάρκεια του. Στο δεξί διάγραμμα της ίδιας εικόνας η αναλογία της διαθεσιμότητας της ιστοσελίδας δεν μηδενίζεται ποτέ, έχοντας τουλάχιστον τα μισά pods (εδώ 1 στα 2) σε λειτουργική κατάσταση. Ταυτόχρονα το pod που καταπονείται δεν παραμένει ανέτοιμο για όλη την διάρκεια στην αρχή επανεκκινείται και τερματίζεται λόγω σφάλματος που εφαρμόζεται μέσω του πειράματος, ενώ στην συνέχεια το πρόβλημα διορθώνεται και επανέρχεται σε πλήρη διαθεσιμότητα η μικροϋπηρεσία της ιστοσελίδας.



Εικόνα 74. Ρυθμός μετάδοσης πακέτων ανά υπηρεσία.

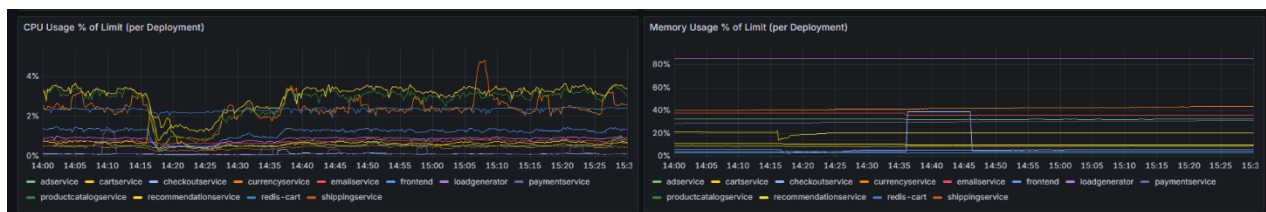
Στην Εικόνα 74 βλέπουμε ολική κατάρρευση του συστήματος, λόγω μηδενισμού του ρυθμού κυκλοφορίας δεδομένων από και προς όλες τις μικροϋπηρεσίες. Σε αντίθεση με την πρώτη εκτέλεση, αυτή η κατάρρευση

είναι στιγμιαία. Σταδιακά ο ρυθμός αυξάνεται, έστω και σε μικρό βαθμό στην αρχή και εν τέλει επανέρχεται πλήρως αρκετά γρήγορα.



Εικόνα 75. Διαθεσιμότητα συστήματος και καθυστέρηση ολοκλήρωσης των αιτημάτων.

Εδώ βλέπουμε ότι η ιστοσελίδα είναι μη διαθέσιμη (αριστερό διάγραμμα) όσο η καθυστέρηση είναι 5 δευτερόλεπτα (κεντρικό και δεξί διάγραμμα). Αυτό είναι αναμενόμενο καθώς η προσθήκης ενός ολόκληρου δευτερολέπτου καθιστά αδύνατη την ολοκλήρωση ενός αιτήματος σε μικρότερο χρόνο, αφού όταν ένα αίτημα φτάσει στα pods της ιστοσελίδας δημιουργεί επί μέρους αιτήματα προς τις υπόλοιπες μικροϋπηρεσίες, αθροίζοντας τον χρόνο εκπλήρωσης τον επί μέρους χρόνο όλο των αιτημάτων στα οποίο προστίθεται επίσης 1 δευτερόλεπτο, είναι αδύνατο να ολοκληρωθεί το αρχικό αίτημα σε 5 δευτερόλεπτα. Το σημαντικό που είναι φανερό σε αυτά τα τρία διαγράμματα είναι πως όταν λήγει το πείραμα προσθήκης καθυστέρησης, η διαθεσιμότητα και ο χρόνος ολοκλήρωσης αιτημάτων επανέρχονται στις τιμές σταθερής λειτουργίας.



Εικόνα 76. Χρήση των διαθέσιμων υπολογιστικών πόρων και μνήμης ανά υπηρεσία.

Από την Εικόνα 76 ο διαχειριστής μπορεί να εντοπίσει την έναρξη του πειράματος όταν διαταράσσονται οι χρονοσειρές της κατανάλωσης των πόρων, αλλά το σύστημα αυτοθεραπεύεται επαναφέροντας την κατανάλωση στα επίπεδα σταθερής λειτουργίας σε μικρό χρονικό διάστημα.

Πίνακας 19. Αναλυτικές μεταβολές των μετρικών κατά την επανεκτέλεση του επιθετικού σεναρίου μηχανικής χάους.

	Σταθερή Κατάσταση			Διάρκεια Πειράματος		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη

Αύξηση των επανεκκινήσεων της ιστοσελίδας	0	0	5.56	2.85	1.11	4.44
Αναλογία έτοιμων pods του ιστοσελίδας (%)	100%	100%	100%	72.22% (-27.18%)	50%	100%
Αναλογία έτοιμων pods του καλαθιού αγορών (%)	100%	100%	100%	100% (0%)	100%	100%
Εισερχόμενη διαμεταγωγή	44.29 kB/s	20.3 kB/s	67.3 kB/s	27.34 kB/s (-47.39%)	11.9 kB/s	44.6 kB/s
Εξερχόμενη διαμεταγωγή	44.06 kB/s	20.4 kB/s	63.9kB/s	26.77 kB/s (-47.92%)	10.8 kB/s	43.9 kB/s
Εισερχόμενος ρυθμός δεδομένων ιστοσελίδας (kB/s)	11.18 kB/s	9.61 kB/s	12.3 kB/s	5.73 kB/s (-48.61%)	2.28 kB/s	9.43 kB/s
Εξερχόμενος ρυθμός δεδομένων ιστοσελίδας (kB/s)	32.64 kB/s	27.8 kB/s	36.3 kB/s	17.02 kB/s (-48.31%)	5.56 kB/s	29.2 kB/s
Καθυστέρηση p99	16.93 ms	13.7 ms	19.9 ms	4993.91 ms (+29590.31%)	4950 ms	5000 ms
Περιορισμός επεξεργαστικής ισχύος της ιστοσελίδας (%)	0 %	0 %	0 %	0% (0%)	0%	0%
Χρήση διαθέσιμης υπολογιστικής ισχύος από την ιστοσελίδα	1.37 %	1.26 %	1.48 %	0.44 % (-67.41%)	0.24 %	0.54 %
Χρήση διαθέσιμης μνήμης από την ιστοσελίδα	5.46 %	5.4 %	5.54 %	3.40 % (-41.07%)	1.92 %	4.47 %
Ποσοστό αξιοποίησης της υπολογιστικής ισχύος του κόμβου εκτέλεσης 5 (%)	23.48 %	22.3 %	27.3 %	34.48 % (+46.85%)	26.6 %	63.5 %
Ποσοστό αξιοποίησης της υπολογιστικής ισχύος του	18.26 %	16.6 %	19.3 %	19.78 % (+8.32%)	16.7 %	24.4 %

κόμβου εκτέλεσης 6 (%)						
---------------------------	--	--	--	--	--	--

Σύγκριση αρχικής και τελευταίας εκτέλεσης

Πίνακας 20. Σύγκριση μετρικών κατά την διάρκεια της επιθετικής ροής εργασιών πριν και μετά τις βελτιώσεις.

	Αρχικό Πείραμα			Νέο Πείραμα		
	Μέση Τιμή	Ελάχιστη	Μέγιστη	Μέση Τιμή	Ελάχιστη	Μέγιστη
Αύξηση των επανεκκινήσεων της ιστοσελίδας	1.67	1.11	5.56	2.85	1.11	4.44
Αναλογία έτοιμων pods του ιστοσελίδας (%)	0% (-100%)	0%	0%	72.22% (-27.18%)	50%	100%
Εισερχόμενη διαμεταγωγή	2.57 kB/s (-93.87%)	1.35 kB/s	3.63 kB/s	27.34 kB/s (-47.39%)	11.9 kB/s	44.6 kB/s
Εξερχόμενη διαμεταγωγή	6.17 kB/s (-85.85%)	0.99 kB/s	2.89 kB/s	26.77 kB/s (-47.92%)	10.8 kB/s	43.9 kB/s
Εισερχόμενος ρυθμός δεδομένων ιστοσελίδας (kB/s)	0.06 kB/s (-99.45%)	0 kB/s	0.29 kB/s	5.73 kB/s (-48.61%)	2.28 kB/s	9.43 kB/s
Εξερχόμενος ρυθμός δεδομένων ιστοσελίδας (kB/s)	0.06 kB/s (-99.81%)	0 kB/s	0.77 kB/s	17.02 kB/s (-48.31%)	5.56 kB/s	29.2 kB/s
Καθυστέρηση p99	0.75 ms * (-95.31%)	0.458 ms *	2.7 ms *	4993.91 ms (+29590.31%)	4950 ms	5000 ms
Περιορισμός επεξεργαστικής ισχύος της ιστοσελίδας (%)	1.14 % (+119.23%)	0 %	11.1 %	0% (0%)	0%	0%
Χρήση διαθέσιμης υπολογιστικής ισχύος από την ιστοσελίδα	0.14 % (-97.91%)	0 %	0.78 %	0.44 % (-67.41%)	0.24 %	0.54 %
Χρήση διαθέσιμης μνήμης από την ιστοσελίδα	0.98 % (-93.00%)	0.21 %	6.61 %	3.40 % (-41.07%)	1.92 %	4.47 %
Ποσοστό αξιοποίησης της υπολογιστικής	13.15 % (-13.54%)	12.2 %	14.2 %	34.48 % (+46.85%)	26.6 %	63.5 %

ισχύος του κόμβου εκτέλεσης 5 (%)						
Ποσοστό αξιοποίησης της υπολογιστικής ισχύος του κόμβου εκτέλεσης 6 (%)	22.17 % (-7.43%)	20.3 %	27.3 %	19.78 % (+8.32%)	16.7 %	24.4 %

Σε αντίθεση με το ήπιο σενάριο στο επιθετικό η διαφορά μεταξύ της αρχικής και της νέας εκτέλεσης είναι πολύ μεγάλη. Το σύστημα υποβαθμίζεται σημαντικά αλλά παραμένει πλέον λειτουργικό, αν εξαιρεθούν τα χρονικά πλαίσια στα οποία οι καταπονήσεις δεν μπορούν να αντιμετωπιστούν (π.χ. προσθήκη 1 δευτερολέπτου καθυστέρησης). Η διαμεταγωγή μειώνεται μόνο μέχρι το μισό ενώ η μικροϋπηρεσία της ιστοσελίδας παραμένει διαθέσιμη κατά 72% με ελάχιστη διαθεσιμότητα στο 50%. Ταυτόχρονα το καλάθι αγορών παρέμεινε πλήρως λειτουργικό.

6. Συμπεράσματα

Στην παρούσα εργασία ολοκληρώθηκε ένας πλήρης κύκλος εφαρμογής της μεθοδολογίας της μηχανικής χάους σε ένα σύγχρονο περιβάλλον νέφους, βασισμένο στην πλατφόρμα διαχείρισης περιεκτών Kubernetes. Με την χρήση του εργαλείου Chaos Mesh και της εφαρμογής μικροϋπηρεσιών Online Boutique, εκτελέστηκαν πειράματα για την ανάδειξη ευπαθειών σε ένα τυπικό σύστημα χωρίς επιπρόσθετα μέτρα ανθεκτικότητας. Τα αποτελέσματα επιβεβαίωσαν ότι τα κατανεμημένα συστήματα έχουν σημαντικά ζητήματα αξιοπιστίας. Η ευελιξία που εισάγει η αρχιτεκτονική μικροϋπηρεσιών σε σχέση με την μονολιθική αρχιτεκτονική αντισταθμίζεται από την πολυπλοκότητα που προστίθεται στην διαχείριση των αποτυχιών σε αιτήματα.

Τα πειραματικά δεδομένα των κεφαλαίων 4 και 5 επιβεβαίωσαν την αρχική υπόθεση ότι η ανθεκτικότητα (resilience) δεν αποτελεί εγγενές χαρακτηριστικό της υποδομής ενός συστήματος, αλλά αποτέλεσμα του σχεδιασμού του συστήματος και της συνεχούς επαλήθευσης της ανθεκτικότητας μέσω δοκιμών. Η ανάλυση και τα συμπεράσματα αυτού του κεφαλαίου εστιάζουν στην συμπεριφορά του συστήματος υπό συνθήκες ήπιας και ακραίας πίεσης και βασίζονται στις μετρικές που συλλέχθηκαν όπως είναι η καθυστέρηση (latency), η διαμεταγωγή (throughput) και ο χρόνος ανάκαμψης του συστήματος μετά από κατάρρευση.

6.1. Αξιολόγηση Ευρημάτων

6.1.1. Εξάντληση πόρων

Τα πειράματα καταπόνησης πόρων (Ενότητα 4.4) επιβεβαίωσαν την κρισιμότητα της διαχείρισης των πόρων για την σταθερότητα του συστήματος. Κατά την διάρκεια της μέτριας καταπόνησης, καταγράφηκαν περισσότερα από 2000 γεγονότα εξάντλησης μνήμης (OOM events). Αυτή η τιμή υποδεικνύει πως αυτά τα γεγονότα δεν είναι στιγμιαίες αποτυχίες, αλλά συστημική αδυναμία της διαχείρισης του φορτίου όταν αυτό είναι μεγάλο.

Συγκεκριμένα, όπως φαίνεται στον Πίνακα 13, η υπηρεσία του καλαθιού παρουσίασε αύξηση στη χρήση της διαθέσιμης επεξεργαστικής ισχύος κατά 1242% (από 1.23% σε 23.75%), ενώ η υπηρεσία ηλεκτρονικού ταχυδρομείου αύξησε την κατανάλωση της ισχύος κατά 4417%. Το γεγονός ότι η χρήση της διαθέσιμης μνήμης αυξήθηκε από 53.6% σε 64% επιβεβαιώνει την ύπαρξη ενός θορυβώδους γειτονικού κόμβου, στον οποίο μία μικροϋπηρεσία λόγω δυσλειτουργίας είναι ικανή να μονοπωλήσει τους πόρους του, επηρεάζοντας την απόδοση των άλλων μικροϋπηρεσιών που συστεγάζονται στον ίδιο κόμβο.

Στο σενάριο της βαριάς καταπόνησης των πόρων (Πίνακας 14), η κατάσταση επιδεινώθηκε δραματικά. Η χρήση της υπολογιστικής ισχύος της υπηρεσίας της ιστοσελίδας αυξήθηκε κατά 765.6%, οδηγώντας τον πυρήνα του λειτουργικού συστήματος να περιορίσει την λειτουργία της (CPU throttle) κατά 58.61%. Αυτός ο περιορισμός γίνεται αντιληπτός και μέσω της αύξησης της καθυστέρησης των αιτημάτων, καθώς αρκετά πακέτα τίθενται σε κατάσταση αναμονής. Το συγκεκριμένο εύρημα απαντά στο **Ερώτημα 3** των ερευνητικών ερωτημάτων, αποδεικνύοντας την ανάγκη για σωστή παραμετροποίηση της των ορίων και των αιτημάτων ανά υπηρεσία.

6.1.2. Ανεπάρκεια αυτοϊάσης σε επίπεδο pods

Το πείραμα αποτυχίας του pod της ιστοσελίδας (Κεφάλαιο 4.2.1) παρείχε σημαντικά ευρήματα για την αξιολόγηση των μηχανισμών αυτοϊάσης της πλατφόρμας Kubernetes. Όπως φαίνεται και στον Πίνακα 6, η πρόκληση σφάλματος μηδένισε την διαθεσιμότητα της ιστοσελίδας και συνεπώς όλου του συστήματος. Παρά τις προσπάθειες του Kubernetes να επαναφέρει την μικροϋπηρεσία, όπως φαίνεται από την αύξηση των επανεκκινήσεων σε 8.5, αυτή παρέμεινε μη διαθέσιμη. Η καθυστέρηση στον εντοπισμό του σφάλματος και η καθυστέρηση στην έναρξη των προσπάθειών επανεκκίνησης οφείλονται στα μεγάλα προεπιλεγμένα όρια των ανιχνευτών ζωτικότητας και ετοιμότητας. Έτσι απαντάται και το **Ερώτημα 2**, αφού υπάρχει ανάγκη εξειδικευμένης παραμετροποίησης των ορίων των ανιχνευτών για να επιτευχθεί υψηλότερη διαθεσιμότητα του συστήματος.

6.1.3. Μη γραμμική φύση της δικτυακής συμπεριφοράς

Η συμπεριφορά του δικτύου της συστάδας υπό συνθήκες πίεσης αποτέλεσε ένα από τα σημαντικότερα ευρήματα των πειραμάτων, υποδεικνύοντας ότι η αρχιτεκτονική υπηρεσιών λειτουργεί ως εκθετικός ενισχυτής της καθυστέρησης. Τα πειράματα αύξησης της καθυστέρησης (Κεφάλαια 4.3.4 – 4.3.6) έδειξαν ότι όταν προστέθηκε μία μικρή εικονική καθυστέρηση 80ms μεταξύ των μικροϋπηρεσιών η καθυστέρηση δεν αυξήθηκε γραμμικά αλλά είχε εκτινάχθηκε στα 1291.94ms (Πίνακας 10). Αυτή η τιμή οφείλεται στην αλυσίδα των κλήσεων μεταξύ των μικροϋπηρεσιών: ένα αίτημα προς την ιστοσελίδα απαιτεί κλήσεις προς άλλες μικροϋπηρεσίες όπως είναι ο κατάλογος προϊόντων, το καλάθι αγορών, οι διαφημίσεις, όπου η καθυστέρηση αθροίζεται. Το φαινόμενο έφτασε το σύστημα στα όρια του κατά την διάρκεια του πειράματος των 300ms, όπου η τελική καθυστέρηση έφτασε στα 4469.58ms, ενώ πριν μετριάσουν τα προβλήματα πολλά αιτήματα ξεπερνούσα το χρονικό όριο (time out) μειώνοντας παροδικά την διαθεσιμότητα στο 0%, ορίζοντας τα όρια του συστήματος (**Ερώτημα 1**).

Παράλληλα, ο περιορισμός του εύρους ζώνης (Bandwidth) οδήγησε σε παρόμοια εκθετική υποβάθμιση του συστήματος. Στον περιορισμό του εύρους ζώνης στα 25 Kbps (Πίνακας 7), αύξηση της καθυστέρησης ήταν διαχειρίσιμη (+211%), ενώ στα 12 Kbps (Πίνακας 8) εντοπίστηκε το όριο του συστήματος με αύξηση κατά 684%. Στο 1 Kbps το σύστημα κατέρρευσε παροδικά, καθυστέρηση αυξήθηκε κατά 15781% (2.3 δευτερόλεπτα) και η διαμεταγωγή μειώθηκε κατά 88.51%. Αυτό υποδεικνύει κατάρρευση ακόμα και των πρωτοκόλλων TCP μετά από συνεχείς αποτυχημένες αναμεταδόσεις και εξάντλησης του παραθύρου συμφόρησης (congestion window).

6.2. Αξιολόγηση Βελτιώσεων

Στο κεφάλαιο 5, εφαρμόζονται νέα μέτρα και μηχανισμοί (αυτόματη οριζόντια κλιμάκωση, πολιτική κατανομής των pods, πολιτική προϋπολογισμού διακοπής, παραμετροποίηση ανιχνευτών), οι οποίοι οδήγησαν σε μετρήσιμη αύξηση της ανθεκτικότητας και της διαθεσιμότητας του συστήματος.

Στο ήπιο ολοκληρωμένο σενάριο, κατά το οποίο εφαρμόζεται αποτυχία στην μικροϋπηρεσία των διαφημίσεων και προστίθεται μέτρια καθυστέρηση, παρατηρείται άμεση βελτίωση της σταθερότητας συγκρίνοντας τις δύο εκτελέσεις (Πίνακας 18). Στην αρχική εκτέλεση η ετοιμότητα της μικροϋπηρεσίας των διαφημίσεων μηδενίζεται πλήρως (0%), μετά την εφαρμογή των βελτιωτικών αλλαγών επιτεύχθηκε η διατήρηση της ετοιμότητας στο 50%, καθώς υπήρχε πάντοτε ένα δεύτερο αντίγραφο το οποίο μπορούσε να ολοκληρώσει αιτήματα προς την μικροϋπηρεσία. Ταυτόχρονα παρατηρήθηκε μικρή βελτίωση στην καθυστέρηση ολοκλήρωσης αιτημάτων κατά 6.7% (από 3151ms σε 2938ms), βελτιώνοντας ελαφρώς την εμπειρία των χρηστών. Το σημαντικότερο εύρημα αφορά την διαχείριση των πόρων καθώς ο περιορισμός της χρήσης υπολογιστικής ισχύος μειώθηκε από 5.24% σε 1.07% (79.5% βελτίωση). Τέλος, η υπηρεσία δεν αδρανοποιείται πλέον, συνεχίζοντας να αξιοποιεί την διαθέσιμη μνήμη (14.29%).

Στο επιθετικό σενάριο, τα μέτρα βελτίωσης ήταν καθοριστικής σημασίας για την αποφυγή της κατάρρευσης του συστήματος. Η ιστοσελίδα όχι μόνο κατάφερε να διατηρήσει την λειτουργία της (50% ελάχιστη διαθεσιμότητα), αλλά στην συνέχεια η πλατφόρμα Kubernetes κατάφερε να επαναφέρει όλα τα pods σε λειτουργία. Η ροή των δεδομένων αποκαταστάθηκε στα 5.73 kB/s, σε σχέση με τον μηδενισμό της στην αρχική εκτέλεση (~0.06 kB/s). Αν και η καθυστέρηση έφτασε σε τιμές κοντά στο χρονικό όριο (4993ms), το σύστημα κατάφερε να μείνει λειτουργικό για να ολοκληρώνει αιτήματα, έστω και σε αυτό το μεγάλο χρονικό διάστημα.

6.3. Μελλοντικές Προοπτικές

Η εργασία ανέδειξε την Μηχανική Χάους όχι ως απλό εργαλείο δοκιμών, αλλά ως απαραίτητο μέρος της ανάπτυξης και της ενδυνάμωσης ενός συστήματος ακόμα και αφού γίνει διαθέσιμο για το ευρύ κοινό. Βασιζόμενοι στα αποτελέσματα των πειραμάτων και τις παρατηρήσεις που έγιναν, θα παρατεθούν μερικές προτάσεις για μελλοντική έρευνα στο αντικείμενο με χρήση σύγχρονων τάσεων και τεχνολογιών.

6.3.1. Ενσωμάτωση eBPF

Η τεχνολογία eBPF (extended Berkeley Packet Filter) αποτελεί έναν μηχανισμό επόμενης γενιάς για την παρατηρησιμότητα και την ασφάλεια. Προσφέρει σημαντικά πλεονεκτήματα σε σχέση με τις παραδοσιακές μεθόδους και εκτελείται σε επίπεδο πυρήνα (kernel), επιτρέποντας την παρακολούθηση ολόκληρου του κόμβου χωρίς την ανάγκη επιπλέον κώδικα, με ελάχιστη επιπρόσθετη επιβάρυνση (overhead). Σε μελλοντικές έρευνες μπορούν να αξιοποιηθούν εργαλεία αυτής της τεχνολογίας, όπως το Pixie¹⁸ ή το Cilium¹⁹, για την ακριβή μέτρηση του χρόνου που παραμένει ένα πακέτο σε ουρές TCP. Αυτό θα βοηθούσε στην καλύτερη κατανόηση φαινομένων όπως ο θορυβώδης γειτονικός κόμβος, δίνοντας την δυνατότητα εντοπισμού της διεργασίας που προκαλεί πραγματικά τις καθυστερήσεις στον χρονοπρογραμματιστή (scheduler).

¹⁸ <https://px.dev/>

¹⁹ <https://cilium.io/>

6.3.2. Μηχανική Χάους σε επίπεδο ασφάλειας

Η αξιοπιστία είναι άρρηκτα συνδεδεμένη με την ασφάλεια ενός συστήματος και η επέκταση της Μηχανικής Χάους σε αυτόν τον τομέα αποτελεί ένα από τα επόμενα βήματα. Οπότε, σαν μελλοντικό ερευνητικό αντικείμενο προτείνεται η εκτέλεση πειραμάτων που δεν στοχεύουν μόνο στην διαθεσιμότητα, αλλά και στην ακεραιότητα των δεδομένων. Μερικά παραδείγματα πειραμάτων είναι η αφαίρεση δικαιωμάτων από ρόλους με τυχαίο τρόπο, η εισαγωγή κακόβουλων rods, η προσομοίωση παραβιασμένων λογαριασμών.

6.3.3. Αυτοματοποίηση μέσω Τεχνητής Νοημοσύνης

Η χρήση Τεχνητής Νοημοσύνης και της Μηχανικής Μάθησης (AI/ML) μπορεί να αυτοματοποιήσει όλες τις διαδικασίες της μεθοδολογίας μηχανικής χάους οι οποίες γίνονται χειροκίνητα από τους διαχειριστές. Ένα σημαντικό πρόβλημα στην μεθοδολογία της μηχανικής χάους είναι ο σωστός καθορισμός της σταθερής κατάστασης (steady state) η οποία δεν είναι ίδια κάθε στιγμή αλλά μεταβάλλεται τόσο κατά την διάρκεια των εποχών, όσο και κατά την διάρκεια της ημέρας. Η χρήση αλγορίθμων μηχανικής μάθησης μπορεί να βοηθήσει ώστε τα εργαλεία μηχανικής χάους να λαμβάνουν υπόψη την εποχικότητα του φορτίου. Επιπλέον, μπορεί να χρησιμοποιηθεί η μηχανική μάθηση για ανάλυση της τοπολογίας ενός συστήματος με αρχιτεκτονική μικροϋπηρεσιών, ώστε να επιλέγεται κατάλληλη εμβέλεια (blast radius) για την εκτέλεση των πειραμάτων. Με αυτόν το τρόπο θα μπορούσε να μειωθεί ο χρόνος για την εύρεση σεναρίων και να αυξηθεί η αποτελεσματικότητα των δοκιμών.

6.3.4. Ανθεκτικότητα σε σύνολα διατήρησης κατάστασης

Τέλος, μία πρόταση για περαιτέρω έρευνα αποτελεί η εφαρμογή πειραμάτων σε σύνολα διατήρησης κατάστασης (StatefulSets), όπως είναι οι βάσεις δεδομένων και οι ουρές μηνυμάτων. Η παρούσα εργασία εστίασε κυρίως σε υπηρεσίες χωρίς διατήρηση κατάστασης (stateless), ωστόσο τα σύνολα διατήρησης κατάστασης είναι εξίσου κρίσιμης σημασίας, αν όχι μεγαλύτερης. Οι δοκιμές αυτές πρέπει να εστιάσουν στην αποτελεσματικότητα των αλγορίθμων συνοχής, όπως ο Raft στο στοιχείο etcd. Στόχος είναι η διαπίστωση όχι απλά του χρόνου ανάκαμψης, αλλά και η διασφάλιση της ακεραιότητας των δεδομένων μετά από κατάρρευση του συστήματος, κατά την επανένταξη των κόμβων στην συστάδα.

Αναφορές

- [1] Ahamed, Z. (2022, May 25). Getting started with Kubernetes Services.
- [2] Ahmad, A. A.-S., Al-Qora'n, L. F., & Zayed, A. (2024). Exploring the impact of chaos engineering with various. *Computing*, 1-38.
- [3] Akuthota, A. (2023). *Chaos Engineering for Microservices*. St. Cloud State University, Department of Computer Science and Information Technology. St. Cloud: St. Cloud State University. Ανάκτηση από https://repository.stcloudstate.edu/csit_etds/42
- [4] Arora, P., & Dixit, A. (2018). Cloud Testing - Proposed Framework with its Scope, Importance, Methodologies, Challenges. *2018 4th International Conference on Computing Communication and Automation (ICCCA)*. Greater Noida, India.
- [5] Badawy, M. (2022, December 30). Make Kubernetes API reachable from anywhere.
- [6] Bernstein, D. (2014, September 30). Containers and Cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Computing*.
- [7] Burns, B., Beda, J., Hightower, K., & Evenson, L. (2022). *Kubernetes Up & Running, Dive into the Future of Infrastructure, Third Edition*. O'Reilly Media.
- [8] Chaos Mesh Authors. (2024). *Chaos Mesh Overview*. Ανάκτηση από Chaos Mesh Documentation: <https://chaos-mesh.org/docs/>
- [9] Chaos Toolkit Contributors. (2024). *Chaos Toolkit Reference*. Ανάκτηση από Chaos Toolkit Documentation: <https://chaostoolkit.org/reference/concepts/>
- [10] Chen, R., Li, S., & Li, Z. (2017). From Monolith to Microservices: A Dataflow-Driven Approach. *24th Asia-Pacific Software Engineering Conference*.
- [11] Extio Technology. (2023, May 22). Demystifying Kubernetes Namespaces: A Deep Dive into Container Segmentation.
- [12] Flora, J., Gonçalves, P., Teixeira, M., & Antunes, N. (2022). A Study on the Aging and Fault Tolerance of Microservices in Kubernetes. *IEEE Access*.
- [13] Ganesh. (2024, July 8). Traditional Method — Virtualization — Containerization.
- [14] Google Cloud Platform. (2025, December 2). *Github*. Ανάκτηση από Microservices Demo: <https://github.com/GoogleCloudPlatform/microservices-demo>
- [15] Grafana Labs. (2025). *Grafana Docs*. Ανάκτηση από Grafana: <https://grafana.com/docs/grafana-cloud/monitor-infrastructure/kubernetes-monitoring/intro-kubernetes-monitoring/>

- [16] Jadhav, Y. C., Sable, A., Suresh, M., & Hanawal, M. K. (2023). Securing Containers: Honey pots for Analysing Container Attacks. *2023 15th International Conference on COMMunication Systems & NETworkS (COMSNETS)* (σ. 3). IEEE.
- [17] Johari, N. (2023, June 28). Conducting chaos engineering experiments on Amazon EBS using AWS Fault Injection Simulator.
- [18] Karn, R. R., Das, R., Pant, D. R., Heikkonen, J., & Kanth, R. (2022). Automated Testing and Resilience of Microservice's Network-link using Istio Service Mesh. *31st Conference of Open Innovations Association (FRUCT)* (σ. 10). Helsinki, Finland: IEEE.
- [19] Kiprotich, I. (2023, February 2). Kubernetes Architecture.
- [20] Krkn Contributors. (2025, December 19). *Krkn aka Kraken*. Ανάκτηση από GitHub: <https://github.com/krkn-chaos/krkn>
- [21] LitmusChaos Authors. (2025). *Litmus Docs*. Ανάκτηση από LitmusChaos: <https://docs.litmuschaos.io/>
- [22] Luksa, M. (2018). *Kubernetes in Action*. Manning.
- [23] Maharjan, Y. (2020, October 25). How Rolling and Rollback Deployments work in Kubernetes.
- [24] Mailewa, A. B., Akuthota, A., & Mohottalalage, T. M. (2025). A Review of Resilience Testing in Microservices Architectures: Implementing Chaos Engineering for Fault Tolerance and System Reliability. *IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*. Las Vegas, NV, USA: IEEE.
- [25] Nandigam, V. D. (2024). Chaos Engineering: Stress-Testing Microservices for Resilience. *International Journal of Research in Computer Applications and Information Technology*.
- [26] Netflix OSS. (2023). *Home*. Ανάκτηση από Chaos Monkey: <https://netflix.github.io/chaosmonkey/>
- [27] Newman, S. (2021). *Building Microservices Designing Fine-Grained Systems, Second Edition*. O'Reilly.
- [28] Oracle Cloud Infrastructure (OCI). (2024, October 17). Differences Between Microservices and Monolithic Architectures.
- [29] Pahl, C. (2015, May - June). Containerization and the PaaS Cloud. *IEEE Cloud Computing*, σσ. 24-31.
- [30] Prasath, A. (χ.χ.). *Micro Frontend Architecture: A Complete Guide to Modern Frontend Development*.

- [31] Prometheus Authors 2014-2025. (2025). *Prometheus Docs*. Ανάκτηση από Prometheus: <https://prometheus.io/docs/introduction/overview/>
- [32] Red Hat Authors. (2024, Απρίλιος 9). *Red Hat Topics*. Ανάκτηση από Red Hat: <https://www.redhat.com/en/topics/virtualization/what-is-a-virtual-machine>
- [33] Rosenthal, C., & Jones, N. (2020). *Chaos Engineering: System Resiliency in Practice*. O'Reilly Media.
- [34] Stanojevic, P., Usorac, S., & Stanojev, N. (2021). Container manager for multiple container. *2021 44th International Convention on Information, Communication and Electronic Technology (MIPRO)*. Opatija, Croatia.
- [35] Steadybit Authors. (2024). *Steadybit*. Ανάκτηση από Steadybit Documentation: <https://docs.steadybit.com/>
- [36] The ChaosBlade Authors. (2025). *ChaosBlade*. Ανάκτηση από ChaosBlade Documentation: <https://chaosblade.io/en/docs>
- [37] The Kubernetes Authors. (2024). *Kubernetes*. Ανάκτηση από Kubernetes Documentation: <https://kubernetes.io/docs>
- [38] Wang, H., Liu, L., Yue, C., Wang, L., Li, B., Chang, J., & Pang, B. (2024). Cloud-native systems resilience assessments based on kubernetes architecture graph. *Service Oriented Computing and Applications*, 1-12.
- [39] Yang, G., Jiang, J., & Huang, J. (2010). System modules interaction based stress testing model. *2010 Second International Conference on Computer Engineering and Applications*. Bali Island.